**Quantum Computing**
Professor Gopal Pandurangan

---

**Scribe Author:** Armaan Amatya

---

# Contents


**Lecture of August 25, 2026** **5**

**1 Efficient Algorithms and Finding Prime Numbers** **5**
1.1 Randomized Algorithms . . . . . 5

**2 Random Sampling** **5**
2.1 Sampling a Random Element of an Array . . . . . 5
2.2 With and Without Replacement . . . . . 6

**3 Factoring** **6**

**4 Search** **7**
4.1 Lower Bound for Deterministic Algorithms . . . . . 7
4.2 Randomized and Quantum Algorithms . . . . . 7

**5 Probability** **8**

**Lecture of September 1, 2026** **9**

**6 Query Complexity for Search** **9**

**7 Coupon Collector Problem** **9**

**8 High Probability Bound** **10**

**9 Lower Bound for Randomized Algorithms** **11**

**10 Chernoff Bounds** **11**

**11 Solution of the Exercise** **12**

**Lecture of September 3, 2026** **15**

**12 Markov's Inequality** **15**

**13 Chebyshev's Inequality** **16**

**14 Chernoff Bound** **16**

**15 Search** **16**

**16 Birthday Paradox** **17**
16.1 Expected Number of Collisions . . . . . 17

**17 Quantum Birthday Paradox** **20**

**Lecture of September 8, 2026** **22**

**18 Distributed Computing in a Complete Network** **22**
18.1 Leader Election . . . . . 22
18.2 Known Bounds . . . . . 24

**19 A Randomized Algorithm for Leader Election** **24**
19.1 The Number of Candidates . . . . . 24
19.2 At Most One Leader . . . . . 25
19.3 Messages and Rounds . . . . . 27

**20 Markov Chains: Random Walks in Undirected Graphs** **27**

**21 The Law of Large Numbers** **29**

**Lecture of September 10, 2026** **30**

**22 Law of Large Numbers** **30**

**23 Random Walks on Graphs** **30**
23.1 The Triangle . . . . . 31
23.2 The Evolution $p_{t+1} = p_t P$ . . . . . 31
23.3 Row Stochastic Matrices . . . . . 31
23.4 The Transition Matrix of a General Graph . . . . . 32

**24 Stationary Distribution** **33**

**25 Convergence to the Stationary Distribution** **34**

**Lecture of September 15, 2026** **36**

**26 Qubits** **36**

**27 Kets, Bras and the Dagger** **37**

**28 Unitary Matrices** **37**

**29 Quantum Circuits and One-Qubit Gates** **39**

**30 The CNOT Gate** **40**
30.1 Classical Gates Are Not Reversible . . . . . 41

**31 Tensor Products and Entanglement** **41**

**Lecture of September 17, 2026** **44**

**32 The Product $Ax$ as a Combination of Columns** **44**

**33 Linear Independence and Rank** **44**

**34 Orthogonality** **46**

**Lecture of September 24, 2026** **48**

**35 Deutsch's Algorithm** **48**
35.1 The Quantum Query $U_f$ . . . 48
35.2 The Circuit . . . 48

**36 Phase Flip** **49**

**37 The $n$-Qubit Hadamard Transform** **50**

**38 Deutsch–Jozsa Algorithm** **50**
38.1 Classical Query Complexity . . . 50
38.2 The Algorithm . . . 51

**Lecture of September 29, 2026** **53**

**39 Recap: Quantum Queries and the Walsh–Hadamard Transform** **53**

**40 Bernstein–Vazirani Algorithm** **54**
40.1 The Algorithm . . . 54

**41 Simon's Algorithm** **55**
41.1 The Circuit . . . 55
41.2 Analysis of One Run . . . 56
41.3 From the Measurements to $s$ . . . 57
41.4 A Deterministic Classical Algorithm . . . 58

**Lecture of October 1, 2026** **60**

**42 Simon's Problem, Classically** **60**
42.1 Deterministic Algorithms . . . 60
42.2 A Randomized Algorithm . . . 60
42.3 Lower Bound for Randomized Algorithms via Yao's Principle . . . 61

**43 Simon's Algorithm Revisited** **65**
43.1 The 1-to-1 Case . . . 65
43.2 The 2-to-1 Case . . . 66
43.3 Solving for $s$ . . . 66

**Lecture of October 6, 2026** **68**

**44 Unstructured Search** **68**

**45 Grover's Algorithm on Two Qubits** **68**

**46 Grover's Algorithm on $n$ Qubits** **69**
46.1 The Operators $Z_f$, $Z_0$ and $D$ . . . 69

**47 Analysis: A Rotation in Two Dimensions** **71**
47.1 How Many Iterations . . . . . 73

**Lecture Date:** August 25, 2026

**Lecture Topic:** Efficient Algorithms, Random Sampling, Lower Bound for Deterministic Search, Probability Basics

In this first lecture, we see why randomness and quantum computation matter for two basic problems, primality/factoring and search. We introduce random sampling as the basic primitive of randomized algorithms, prove a lower bound of $n-1$ queries for deterministic search, and review the basics of probability that we need for the rest of the course.

# 1 Efficient Algorithms and Finding Prime Numbers

Classical computing was deterministic until relatively recently, when randomness was introduced into algorithms. The next step, and the subject of this course, is quantum computing.

Consider the problem of deciding whether a given number $n$ is prime. The input is the binary representation of $n$, so the *input size* is the number of bits

$$N = \log_2 n,$$

and not $n$ itself.

**Definition 1 ▶ Efficient Algorithm**

An algorithm is **efficient** if it runs in time polynomial in the input size, i.e., in time $\mathcal{O}\left(N^k\right)$ for some constant $k$. For primality testing this means time $\mathcal{O}\left((\log_2 n)^k\right)$.

The trivial algorithm, trial division by all numbers up to $\sqrt{n}$, takes $\Theta(\sqrt{n}) = \Theta\left(2^{N/2}\right)$ time, which is exponential in the input size. For a long time no efficient algorithm was known. A deterministic polynomial-time algorithm was eventually found (running in time $\tilde{\mathcal{O}}(N^{12})$, later improved to $\tilde{\mathcal{O}}(N^6)$), but it is not used in practice, both because it is not particularly fast and because there is a better alternative.

## 1.1 Randomized Algorithms

The algorithm used in practice is *randomized*: the Miller–Rabin test decides primality with high probability in $\mathcal{O}(N^3)$ time. The idea is that if $n$ is composite, then at least $3/4$ of the numbers in $\{2, \ldots, n-2\}$ are *witnesses* of the fact that $n$ is composite, while a prime has no witness. Sampling one random number therefore detects a composite $n$ with probability at least $3/4$, and after $k$ independent samples a composite $n$ goes undetected with probability at most $(1/4)^k$. If the test says "composite" it is always right; if it says "prime" it is wrong with probability at most $(1/4)^k$, which can be made as small as we like by increasing $k$.

# 2 Random Sampling

All that a randomized algorithm needs is the ability to *sample*. We assume that flipping a (fair) coin is an $\mathcal{O}(1)$ time primitive.

## 2.1 Sampling a Random Element of an Array

Consider an array with $n$ slots (Figure 1) and assume for simplicity that $n$ is a power of 2. Every index is a bit string of $\log_2 n = \log n$ bits, so we can sample a uniformly random element

by flipping one coin for each bit of the index. This takes $\mathcal{O}(\log n)$ time, and every index $i$ is obtained with probability

$$\mathbb{P}(\text{index } i \text{ is chosen}) = \left(\frac{1}{2}\right)^{\log n} = \frac{1}{2^{\log n}} = \frac{1}{n}.$$

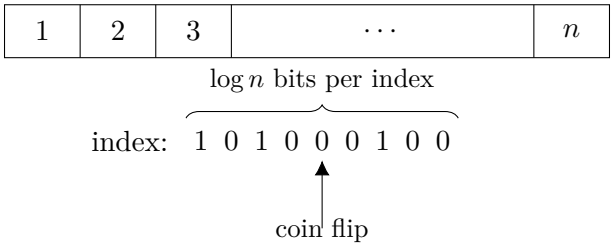



Figure 1: Sampling a random slot of an array with $n$ slots: each of the $\log n$ bits of the index is the result of one coin flip.

We usually abstract away the $\mathcal{O}(\log n)$ and assume that an element can be sampled in constant time. This is only justified when the space we sample from has polynomial size (as a function of $n$), since then the number of coin flips is $\mathcal{O}(\log n)$; for a larger space the sampling time is no longer negligible.

### 2.2 With and Without Replacement

Random sampling can be done *with* or *without* replacement. We will almost always analyze sampling with replacement, because the analysis without replacement is more complex (the samples are no longer independent).

Suppose that we are looking for one particular element of the array. For every sample,

$$\mathbb{P}(\text{element} = \text{sample}) = \frac{1}{n}.$$

If we keep sampling (with replacement) until the element is found, then the expected number of samples, i.e., the number of steps of the algorithm, is

$$\mathbb{E}[\text{samples}] = \frac{1}{\mathbb{P}(\text{element} = \text{sample})} = n.$$

The probability of *not* finding the element is computed directly: a sample misses it with probability $1 - \frac{1}{n}$, and since the samples are independent,

$$\mathbb{P}(\text{not found after } k \text{ samples}) = \left(1 - \frac{1}{n}\right)^k, \qquad \mathbb{P}(\text{found within } k \text{ samples}) = 1 - \left(1 - \frac{1}{n}\right)^k.$$

## 3 Factoring

Factoring is the second motivating problem: given $n = pq$ with $p, q$ prime, find $p$ and $q$.

- Factoring is an *encryption primitive*: the security of widely used encryption schemes rests on it being hard. If factoring can be solved quickly, then the encryption is broken.
- No polynomial-time (in $N = \log n$) classical algorithm is known.
- There is a quantum algorithm that runs in polynomial time, *Shor's factoring algorithm* (time $\mathcal{O}(N^3)$).

- There is *no* known quantum algorithm that solves NP-complete problems in polynomial time.

Recall that NP is the class of problems whose solutions are easy to verify (in polynomial time), and that the NP-complete problems are the NP problems to which every NP problem reduces (equivalently, the problems that are reducible to 3-SAT and vice versa). Whether P = NP, i.e., whether NP-complete problems can be solved in polynomial time, is a famous open problem. A quantum computer can simulate a classical one with polynomial overhead, so if P = NP then quantum computers also solve NP-complete problems in polynomial time; conversely, if one could show that quantum computers cannot solve NP-complete problems in polynomial time, then P $\neq$ NP.

# 4 Search

The third problem, which we will study in detail, is *search*: we are given an (unsorted) array $A$ with $n$ slots, which can only be accessed through *queries* ("what is $A[i]$?"), and we want to find the index of a given element $x$. If the array were sorted, binary search would take $\mathcal{O}(\log n)$ queries; since it is not, linear search makes $\mathcal{O}(n)$ queries. Can we do better?

## 4.1 Lower Bound for Deterministic Algorithms

An upper bound of $\mathcal{O}(f(n))$ queries only needs one algorithm; a lower bound of $\Omega(f(n))$ has to hold for *all* algorithms. The standard tool for this is an *adversary argument*: given any algorithm, the adversary constructs an input on which the algorithm performs badly.

**Theorem 1 ▶ Lower Bound for Deterministic Search**

Every deterministic algorithm for search makes at least $n - 1$ queries in the worst case.

*Proof.* Fix a deterministic algorithm. The adversary does not commit to the input in advance; instead it answers the queries of the algorithm as they come. Whenever the algorithm queries a slot $i$, the adversary answers that $A[i] \neq x$ (and fills $A[i]$ with some other value).

Since the algorithm is deterministic, its $k$th query is determined by the answers to the first $k-1$ queries, so this defines one input for the algorithm. Suppose that the algorithm stops after $k \leqslant n-2$ queries and outputs an index $j$. At least $n-k \geqslant 2$ slots have not been queried, so the adversary can place $x$ in an unqueried slot different from $j$. The resulting array is consistent with all the answers that were given, and the algorithm is wrong on it. Hence a correct algorithm makes at least $n-1$ queries. □

After $n-1$ queries that were all answered "no", only one slot is left and the algorithm knows that $x$ is there without querying it, so the bound $n-1$ is tight.

## 4.2 Randomized and Quantum Algorithms

- The adversary argument breaks down for *randomized* algorithms: the queries are no longer determined by the previous answers, so the adversary cannot predict them. Nevertheless, randomized search needs $\Omega(n)$ expected queries. The proof uses *Yao's min-max principle*, which is a good general tool for proving lower bounds for randomized algorithms (next lecture).
- Grover's quantum algorithm solves search with $\mathcal{O}(\sqrt{n})$ queries (later in the course).

To show a *quantum advantage* (often called quantum supremacy in class) for a problem, we have to show that the problem needs $\Omega(f(n))$ queries classically but only $o(f(n))$ queries quantumly. Since randomized algorithms are the strongest classical algorithms, this means that we need lower bounds for *randomized* algorithms, not only for deterministic ones. This is why lower bounds for randomized algorithms play a bigger role in this course than lower bounds for deterministic algorithms.

# 5 Probability

We review the basic notions of (discrete) probability (Figure 2). A **sample space** $S$ is a finite set of *simple events*, $|S| = n$. A **probability function** $\mathbb{P}\colon S \to [0,1] \subseteq \mathbb{R}$ assigns a probability to every simple event. An **event** is a subset $A \subseteq S$, with probability $\mathbb{P}(A) = \sum_{a \in A} \mathbb{P}(a)$. The probability function satisfies the *axioms* $\mathbb{P}(A) \geqslant 0$ for every event $A$, $\mathbb{P}(S) = 1$, and $\mathbb{P}(A_1 \cup A_2 \cup \cdots \cup A_n) = \sum_{i=1}^{n} \mathbb{P}(A_i)$ for pairwise disjoint events $A_1, \ldots, A_n$ (i.e., $A_i \cap A_j = \emptyset$ for all $i \neq j$). A **random variable** is a function $X\colon S \to V$ from the sample space to a set of values $V$ (usually $V \subseteq \mathbb{R}$), and for a value $r \in V$, $\mathbb{P}(X = r) = \sum_{s \in S\,:\,X(s)=r} \mathbb{P}(s)$.

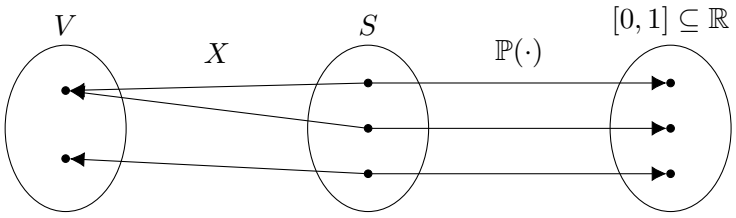



Figure 2: A sample space $S$ of simple events. The probability function $\mathbb{P}(\cdot)$ maps every simple event to a number in $[0,1]$, and a random variable $X$ maps every simple event to a value in $V$.

For example, in random sampling the sample space $S$ is the set of the $n$ possible outcomes of the $\log n$ coin flips, and the random variable $X$ is the index that is sampled. Every $s \in S$ has $\mathbb{P}(s) = 1/n$, and $\mathbb{P}(X = i) = 1/n$ for every index $i$.

**Lecture Date:** September 1, 2026

**Lecture Topic:** Query Complexity of Search, Coupon Collector Problem, Lower Bound for Randomized Algorithms, Chernoff Bounds

In this lecture, we study the query complexity of search for classical randomized algorithms. We analyze random sampling, relate it to the coupon collector problem, state a lower bound for randomized search algorithms using Yao's minmax principle, and introduce Chernoff bounds.

## 6 Query Complexity for Search

Recall the search problem of the last lecture (Figure 1): an array of $n$ slots can only be accessed through *queries*, a query to a slot $x$ returns the value $f(x)$, and the goal is to find the slot $x$ with $f(x) = 1$. The cost of an algorithm is its *query complexity*, the number of queries that it makes.

**Definition 2 ▶ Monte Carlo and Las Vegas Algorithms**

A **Monte Carlo** algorithm is a randomized algorithm whose error probability is $> 0$.
A **Las Vegas** algorithm is a randomized algorithm whose error probability is $= 0$.

Random sampling *with replacement* queries in every step a slot chosen uniformly at random, so every query succeeds with probability $1/n$, and the number of queries until the first success is a *geometric* random variable with parameter $1/n$. Hence

$$\mathbb{E}[\text{queries until find}] = n,$$

as stated in the last lecture. Stopped after a fixed number of queries, random sampling is a Monte Carlo algorithm, since it can fail to find the slot; run until the slot is found, it is a Las Vegas algorithm with $n$ expected queries.

## 7 Coupon Collector Problem

Suppose that we want to distinguish 2 binary arrays (Figure 3). To do this with random sampling, all the $n$ slots have to be accessed. The number of queries needed by random sampling (with replacement) to access all the $n$ slots is the *coupon collector problem*. We show that

$$\mathbb{E}[\text{queries}] \leqslant \mathcal{O}(n \log n) \quad \text{and} \quad \mathbb{E}[\text{queries}] \geqslant \Omega(n \log n),$$

i.e., $\mathbb{E}[\text{queries}] = \Theta(n \log n)$.

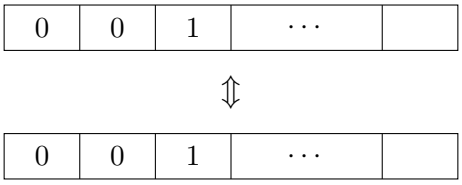



Figure 3: To distinguish 2 binary arrays.

**Theorem 2 ▶ Coupon Collector**

Let $X$ be the number of queries made by random sampling (with replacement) until all the $n$ slots are accessed. Then

$$\mathbb{E}[X] = nH_n = \Theta(n \log n),$$

where $H_n = \sum_{i=1}^{n} \frac{1}{i}$ is the $n$th harmonic number.

*Proof.* Write

$$X = X_1 + X_2 + \cdots + X_n,$$

where $X_i$ is the number of queries made to access a new slot, after $i-1$ different slots have already been accessed. When $i-1$ slots have been accessed, a query accesses a new slot with probability $\frac{n-(i-1)}{n}$. Hence $X_i$ is a geometric random variable,

$$X_i \sim G\left(\frac{n-(i-1)}{n}\right), \qquad \mathbb{E}[X_i] = \frac{n}{n-(i-1)}.$$

By linearity of expectation,

$$\begin{aligned}
\mathbb{E}[X] &= \mathbb{E}[X_1 + X_2 + \cdots + X_n] \\
&= \sum_{i=1}^{n} \mathbb{E}[X_i] \\
&= \sum_{i=1}^{n} \frac{n}{n-(i-1)} \\
&= n \sum_{i=1}^{n} \frac{1}{n-i+1} \\
&= nH_n.
\end{aligned}$$

Since $H_n = \Theta(\log n)$, we get $\mathbb{E}[X] = \Theta(n \log n)$. □

# 8 High Probability Bound

The following was given in class as an exercise. Its solution uses the Chernoff bounds, which are stated at the end of this lecture, so it is given in Section 11.

**Exercise 1.** Show that

$$\mathbb{P}(\text{any slot is accessed} \leqslant c \log n \text{ times}) \leqslant \frac{1}{n^2}.$$

The expectation of the number of queries is $\Theta(n \log n)$. We now show that $3n \log n$ queries are enough with high probability. Here log is the natural logarithm.

**Theorem 3**

If random sampling makes $3n \log n$ queries, then

$$\mathbb{P}(\text{fail}) \leqslant \frac{1}{n^2}, \qquad \text{i.e.,} \qquad \mathbb{P}(\text{Success}) \geqslant 1 - \frac{1}{n^2},$$

where "fail" means that some slot is not accessed.

*Proof.* Let $B_i$ be the event that slot $i$ is not accessed. A query does not access slot $i$ with probability $1 - \frac{1}{n}$. Using $1 - x \leqslant e^{-x}$,

$$\begin{aligned}\mathbb{P}(B_i) = \mathbb{P}(\text{slot } i \text{ not accessed}) &\leqslant \left(1 - \frac{1}{n}\right)^{3n \log n} \\ &\leqslant e^{-\frac{1}{n} \cdot 3n \log n} \\ &= e^{-3 \ln n} \\ &= \frac{1}{n^3}.\end{aligned}$$

By the union bound,

$$\mathbb{P}\left(\bigcup_i B_i\right) \leqslant \sum_{i=1}^{n} \mathbb{P}(B_i) = \frac{n}{n^3} = \frac{1}{n^2}.$$

□

# 9 Lower Bound for Randomized Algorithms

**Theorem 4**

Any Las Vegas randomized algorithm for search needs $\Omega(n)$ expected queries.

The lower bound is shown using Yao's minmax principle.

### Yao's Minmax Principle

Let $A^{\text{rand}}$ be a randomized algorithm, and let $\mathbb{E}\left[A^{\text{rand}}\right]$ be its expected number of queries. Let $\mu$ be a probability distribution over the inputs, and let $D_\mu^{\text{best}}$ be the best deterministic algorithm for inputs that are chosen according to $\mu$. Then

$$\mathbb{E}\left[A^{\text{rand}}\right] \geqslant \max_\mu \mathbb{E}_\mu\left[D_\mu^{\text{best}}\right] \geqslant \Omega(n).$$

The quantity $\mathbb{E}_\mu\left[D_\mu^{\text{best}}\right]$ is called the *distributional complexity*.

# 10 Chernoff Bounds

Let's say that a coin gives heads ($H$) with probability $\frac{1}{2} + \varepsilon$ and tails ($T$) with probability $\frac{1}{2} - \varepsilon$. If we toss the coin $n$ times, then

$$\mathbb{E}[\text{Heads}] = n\left(\frac{1}{2} + \varepsilon\right) = \frac{n}{2} + n\varepsilon.$$

Chernoff bounds show that the number of heads is close to its expectation, so that

$$\mathbb{P}\left(\text{Heads} < \frac{n}{2}\right) \sim \text{very small.}$$

In general, let

$$X = \sum_{i=1}^{n} X_i, \qquad X_i = \begin{cases} 1 & \text{with probability } p_i, \\ 0 & \text{with probability } 1 - p_i, \end{cases}$$

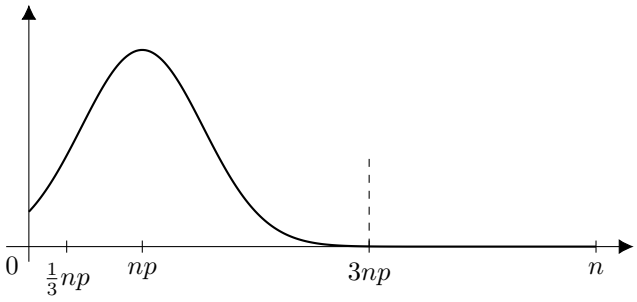



Figure 4: The distribution of $X$ is concentrated around the mean $np$.

where the $X_i$ are independent Bernoulli random variables. If $p_i = p$ for all $i$, then

$$\mathbb{E}[X] = np, \qquad \mathbb{V}[X] = np(1-p).$$

The distribution of $X$ is concentrated around its mean $np$ (Figure 4).

**Theorem 5 ▶ Chernoff Bounds**

Let $X = \sum_{i=1}^{n} X_i$ be as above and let $\mu = \mathbb{E}[X]$.

- Upper tail (UI): for $0 < \delta < 1$,

$$\mathbb{P}(X \geqslant (1+\delta)\mu) \leqslant e^{-\frac{\mu\delta^2}{3}}.$$

- For $R \geqslant 6\mu$,

$$\mathbb{P}(X \geqslant R) \leqslant \frac{1}{2^R}.$$

- Lower tail (LI): for $0 < \delta < 1$,

$$\mathbb{P}(X < (1-\delta)\mu) \leqslant e^{-\frac{\mu\delta^2}{2}}.$$

## 11 Solution of the Exercise

We now solve Exercise 1 with the Chernoff bounds of Theorem 5. The exercise says that the queries of random sampling are spread evenly over the slots: every slot is accessed about $\log n$ times, up to a constant $c$. We prove this in both directions. First, with $3n \log n$ queries no slot is accessed much *more* than the expected $3 \log n$ times (upper tail, $c = 18$). Second, with a constant factor more queries no slot is accessed much *less* than expected (lower tail, $c = 6$); we also explain why the lower tail needs the extra queries. As before, log is the natural logarithm.

Fix a slot $i$ and suppose that random sampling makes $T$ queries. Let

$$Y_i = \sum_{t=1}^{T} Z_t, \qquad Z_t = \begin{cases} 1 & \text{if the } t\text{th query accesses slot } i, \\ 0 & \text{otherwise,} \end{cases}$$

be the number of times that slot $i$ is accessed. Every query accesses slot $i$ with probability $1/n$, and the queries are independent (sampling with replacement), so the $Z_t$ are independent Bernoulli random variables with $p_t = 1/n$, exactly as in Theorem 5, and

$$\mu = \mathbb{E}[Y_i] = \frac{T}{n}.$$

**Lemma 6 ▶ Upper Tail: No Slot Is Accessed Too Often**

If random sampling makes $3n \log n$ queries, then

$$\mathbb{P}(\text{some slot is accessed} \geqslant 18 \log n \text{ times}) \leqslant \frac{1}{n^2}.$$

*Proof.* Fix a slot $i$. Here $T = 3n \log n$, so $\mu = 3 \log n$, and $R = 18 \log n = 6\mu$. The second Chernoff bound of Theorem 5 gives

$$\mathbb{P}(Y_i \geqslant 18 \log n) \leqslant \frac{1}{2^{18 \log n}} = \frac{1}{n^{18 \log 2}} \leqslant \frac{1}{n^3},$$

where $2^{18 \log n} = e^{18 \log 2 \cdot \log n} = n^{18 \log 2}$ and $18 \log 2 > 12 > 3$. Let $C_i$ be the event that slot $i$ is accessed at least $18 \log n$ times. By the union bound,

$$\mathbb{P}\left(\bigcup_{i=1}^{n} C_i\right) \leqslant \sum_{i=1}^{n} \mathbb{P}(C_i) \leqslant n \cdot \frac{1}{n^3} = \frac{1}{n^2}.$$ □

**Lemma 7 ▶ Lower Tail: No Slot Is Accessed Too Rarely**

If random sampling makes $24n \log n$ queries, then

$$\mathbb{P}(\text{some slot is accessed} < 12 \log n \text{ times}) \leqslant \frac{1}{n^2}.$$

In particular, for $n \geqslant 2$, $\mathbb{P}(\text{some slot is accessed} \leqslant 6 \log n \text{ times}) \leqslant 1/n^2$, which is Exercise 1 with $c = 6$.

*Proof.* Fix a slot $i$. Now $T = 24n \log n$, so $\mu = 24 \log n$. Take $\delta = 1/2$ in the lower tail bound of Theorem 5; then $(1 - \delta)\mu = 12 \log n$ and

$$\mathbb{P}(Y_i < 12 \log n) = \mathbb{P}(Y_i < (1 - \delta)\mu) \leqslant e^{-\frac{\mu\delta^2}{2}} = e^{-\frac{24 \log n}{8}} = e^{-3 \log n} = \frac{1}{n^3}.$$

Let $D_i$ be the event that slot $i$ is accessed fewer than $12 \log n$ times. By the union bound,

$$\mathbb{P}\left(\bigcup_{i=1}^{n} D_i\right) \leqslant \sum_{i=1}^{n} \mathbb{P}(D_i) \leqslant n \cdot \frac{1}{n^3} = \frac{1}{n^2}.$$

The lower tail bound is stated for the strict event $X < (1-\delta)\mu$, which is why the lemma has "<". For $n \geqslant 2$ we have $\log n > 0$, so a slot that is accessed at most $6 \log n$ times is accessed fewer than $12 \log n$ times; the event of the second statement is contained in $\bigcup_i D_i$ and has probability at most $1/n^2$ as well. □

**Why the lower tail needs more queries.** With only $T = 3n \log n$ queries we have $\mu = 3 \log n$, and the lower tail bound gives, for every $0 < \delta < 1$,

$$\mathbb{P}(Y_i < (1 - \delta)\, 3 \log n) \leqslant e^{-\frac{3 \log n \cdot \delta^2}{2}} = \frac{1}{n^{3\delta^2/2}} > \frac{1}{n^{3/2}}.$$

After the union bound over the $n$ slots this is more than $n^{-1/2}$, which is far from the $n^{-2}$ that the exercise asks for. In general, with $T = a\, n \log n$ queries the same computation gives $n \cdot n^{-a\delta^2/2}$, which is at most $1/n^2$ as soon as $a \geqslant 6/\delta^2$; Lemma 7 is the choice $\delta = 1/2$, $a = 24$. The upper tail does not have this problem because the bound $2^{-R}$ for $R \geqslant 6\mu$ is much stronger.

**Corollary 8 ▶ Every Slot Is Accessed $\Theta(\log n)$ Times**

If random sampling makes $24n \log n$ queries, then with probability at least $1 - 2/n^2$ every slot is accessed at least $12 \log n$ and fewer than $144 \log n$ times.

*Proof.* The lower bound is Lemma 7. For the upper bound, repeat the proof of Lemma 6 with $\mu = 24 \log n$ and $R = 6\mu = 144 \log n$: $\mathbb{P}(Y_i \geqslant 144 \log n) \leqslant 2^{-144 \log n} \leqslant n^{-3}$, and the union bound gives $1/n^2$. The two failure events together have probability at most $2/n^2$. □

**Lecture Date:** September 3, 2026

**Lecture Topic:** Markov, Chebyshev and Chernoff Inequalities, Search, Birthday Paradox

In this lecture, we state and prove Markov's inequality and Chebyshev's inequality, and we state the Chernoff bound. We then look at the query complexity of search for classical and quantum algorithms, and we analyze the birthday paradox.

## 12 Markov's Inequality

Markov's inequality uses only the expectation $\mathbb{E}[X]$ of the random variable, which is also called the *first moment*.

**Theorem 9 ► Markov's Inequality**

Let $X \geqslant 0$ be a random variable. Then, for any $a > 0$,

$$\mathbb{P}(X \geqslant a\,\mathbb{E}[X]) \leqslant \frac{1}{a}.$$

Equivalently, for any $b > 0$,

$$\mathbb{P}(X \geqslant b) \leqslant \frac{\mathbb{E}[X]}{b}.$$

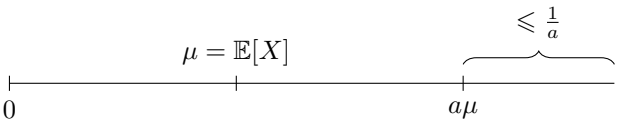



Figure 5: Markov's inequality: the probability that $X$ is at least $a\mu$ is at most $1/a$.

*Proof.* Let

$$I = \begin{cases} 1 & \text{if } X \geqslant b, \\ 0 & \text{otherwise.} \end{cases}$$

Then

$$I \leqslant \frac{X}{b}.$$

This is true for $X \geqslant b$, since then $I = 1 \leqslant \frac{X}{b}$. It is also true for $X < b$, since then $I = 0$ and $X \geqslant 0$. Markov's inequality follows by taking expectations on both sides:

$$\mathbb{E}[I] \leqslant \mathbb{E}\left[\frac{X}{b}\right] = \frac{\mathbb{E}[X]}{b}.$$

Since

$$\mathbb{P}(I = 1) = \mathbb{P}(X \geqslant b) \qquad \text{and} \qquad \mathbb{P}(I = 0) = \mathbb{P}(X < b),$$

we have $\mathbb{E}[I] = \mathbb{P}(I = 1)$, and hence

$$\mathbb{P}(X \geqslant b) \leqslant \frac{\mathbb{E}[X]}{b}.$$

Taking $b = a\,\mathbb{E}[X]$ gives $\mathbb{P}(X \geqslant a\,\mathbb{E}[X]) \leqslant \frac{1}{a}$. □

## 13 Chebyshev's Inequality

Chebyshev's inequality uses the variance of the random variable, which is also called the *second moment*.

**Definition 3 ▶ Variance and Standard Deviation**

The **variance** of a random variable $X$ is

$$\mathbb{V}[X] = \mathbb{E}\Big[(X - \mathbb{E}[X])^2\Big].$$

The **standard deviation** of $X$ is

$$\sigma = \sqrt{\mathbb{V}[X]}.$$

**Theorem 10 ▶ Chebyshev's Inequality**

For any random variable $X$ and any $a > 0$,

$$\mathbb{P}(|X - \mathbb{E}[X]| \geqslant a) \leqslant \frac{\mathbb{V}[X]}{a^2} = \frac{\sigma^2}{a^2}.$$

Here $|X - \mathbb{E}[X]|$ is the deviation of $X$ from its mean.

*Proof.*

$$\mathbb{P}(|X - \mathbb{E}[X]| \geqslant a) = \mathbb{P}\Big((X - \mathbb{E}[X])^2 \geqslant a^2\Big) \leqslant \frac{\mathbb{E}\Big[(X - \mathbb{E}[X])^2\Big]}{a^2} = \frac{\mathbb{V}[X]}{a^2},$$

where the inequality is Markov's inequality applied to the random variable $(X - \mathbb{E}[X])^2 \geqslant 0$. □

## 14 Chernoff Bound

The Chernoff bounds of the last lecture (Theorem 5) were restated in class with the independence assumption made explicit. For a sum $X = \sum_{i=1}^{n} X_i$ of *mutually independent* indicator random variables with $\mathbb{P}(X_i = 1) = p_i$, linearity of expectation gives $\mu = \mathbb{E}[X] = \sum_{i=1}^{n} p_i$, and for $0 < \delta < 1$

$$\mathbb{P}(X \geqslant (1+\delta)\mu) \leqslant e^{-\frac{\mu\delta^2}{3}}, \qquad \mathbb{P}(X < (1-\delta)\mu) \leqslant e^{-\frac{\mu\delta^2}{2}},$$

and $\mathbb{P}(X \geqslant R) \leqslant 2^{-R}$ for $R \geqslant 6\mu$.

## 15 Search

Let $f\colon D \to R$ be a function from a domain $D$ to a range $R$ (Figure 6). Given $y$, we ask: is there an $x$ such that $f(x) = y$? We consider

$$f\colon [1, \ldots, N] \to [1, \ldots, N],$$

and we want to find an $x$ such that $f(x) = k$. The function can be seen as an array with $n$ slots, where $n$ is the size of the domain, and the cost of an algorithm is the number of queries to $f$.

- Randomized classical algorithm: $\Theta(n)$ queries.
- Grover's algorithm: $\mathcal{O}(\sqrt{n})$ queries.

For the random sampling algorithm,

$$\mathbb{E}[\text{queries}] = \mathcal{O}(n), \qquad \mathbb{P}(\text{Success}) = \frac{1}{n}.$$

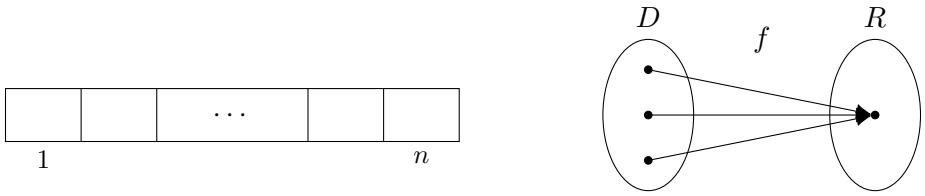



Figure 6: Left: an array with $n$ slots. Right: a function $f\colon D \to R$.

**Probability amplitude amplification.** If the classical success probability is $p$, then in the quantum case it becomes $\sqrt{p}$.

# 16 Birthday Paradox

There are $m$ people and $n$ birthdays ($n = 365$). We are looking for a *collision*, i.e., $x \neq y$ such that

$$f(x) = f(y),$$

where $f(x)$ is the birthday of person $x$.

Let $E_i$ be the event that the $i$th birthday is different from the first $i-1$ birthdays. Then

$$\begin{aligned}
\mathbb{P}\left(\bigcap_{i=1}^{m} E_i\right) &= \mathbb{P}(\text{all birthdays different}) \\
&= \mathbb{P}(E_1) \cdot \mathbb{P}(E_2 \mid E_1) \cdot \mathbb{P}(E_3 \mid E_2 \cap E_1) \cdots \\
&= 1 \cdot \left(1 - \frac{1}{n}\right) \cdot \left(1 - \frac{2}{n}\right) \cdots \left(1 - \frac{m-1}{n}\right) \\
&= \prod_{i=0}^{m-1} \left(1 - \frac{i}{n}\right) \\
&\leqslant \prod_{i=0}^{m-1} e^{-i/n} \\
&= e^{-\sum_{i=0}^{m-1} \frac{i}{n}} \\
&= e^{-\frac{(m-1)m}{2n}}.
\end{aligned}$$

For $m = \sqrt{2n} + 1$,

$$\mathbb{P}(\text{all birthdays different}) \leqslant e^{-\frac{\sqrt{2n}(\sqrt{2n}+1)}{2n}} \leqslant e^{-\frac{2n}{2n}} = \frac{1}{e} \leqslant \frac{1}{2}.$$

Hence $\mathcal{O}(\sqrt{n})$ people are enough to have a constant probability of collision.

## 16.1 Expected Number of Collisions

The birthday paradox can be seen as throwing $m$ balls into $n$ bins (Figure 7). We show that for $m \approx \sqrt{2n}$,

$$\mathbb{E}[\#\text{ collisions}] \geqslant 1.$$

Let $X$ be the number of collisions. $X$ takes values from 0 to $\binom{m}{2}$. Write

$$X = \sum_{i \neq j} X_{ij}, \qquad X_{ij} = \begin{cases} 1 & \text{if } f(i) = f(j), \\ 0 & \text{else,} \end{cases}$$

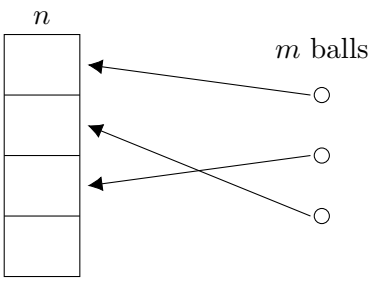



Figure 7: $m$ balls are thrown into $n$ bins.

where the sum is over the $\binom{m}{2}$ pairs $i \neq j$. The random variables $X_{ij}$ are *pairwise independent*. By linearity of expectation,

$$\begin{aligned}\mathbb{E}[X] &= \sum_{i\neq j} \mathbb{E}[X_{ij}] \\ &= \sum_{i\neq j} \mathbb{P}(X_{ij} = 1) \\ &= \sum_{i\neq j} \frac{1}{n} \\ &= \binom{m}{2}\frac{1}{n} = \frac{m(m-1)}{2n} \sim \frac{m^2}{2n} = \Theta\left(\frac{m^2}{n}\right).\end{aligned}$$

**Exercise 2.** Show that

$$\mathbb{P}(X \geqslant 1) \geqslant \Theta(1),$$

i.e., the birthday paradox occurs.

The hint given in class is to compute the variance

$$\mathbb{V}[X] = \mathbb{V}\left[\sum_{i\neq j} X_{ij}\right]$$

and to use Chebyshev's inequality (Theorem 10): if the variance of $X$ is small compared to $\mathbb{E}[X]^2$, then $X$ is rarely far from its mean, and in particular rarely 0. We first prove the claim that the $X_{ij}$ are pairwise independent, which is what makes the variance easy to compute.

**Lemma 11 ▶ Variance of the Number of Collisions**

For any two different pairs $\{i, j\} \neq \{k, l\}$,

$$\mathbb{E}[X_{ij}X_{kl}] = \mathbb{E}[X_{ij}]\,\mathbb{E}[X_{kl}] = \frac{1}{n^2},$$

i.e., $X_{ij}$ and $X_{kl}$ are independent. Consequently

$$\mathbb{V}[X] = \sum_{i\neq j} \mathbb{V}[X_{ij}] = \binom{m}{2}\frac{1}{n}\left(1 - \frac{1}{n}\right) \leqslant \mathbb{E}[X].$$

*Proof.* Since $X_{ij}X_{kl} = 1$ exactly when both indicators are 1,

$$\mathbb{E}[X_{ij}X_{kl}] = \mathbb{P}(f(i) = f(j) \text{ and } f(k) = f(l)).$$

There are two cases.

- The pairs are disjoint, $\{i,j\} \cap \{k,l\} = \emptyset$. Then the two events depend on the birthdays of four different people, which are independent, so the probability is $\mathbb{P}(f(i) = f(j)) \cdot \mathbb{P}(f(k) = f(l)) = 1/n^2$.
- The pairs share one person, say $\{i,j\}$ and $\{i,k\}$ with $j \neq k$. The birthdays $f(j)$ and $f(k)$ are independent of $f(i)$ and of each other, and each equals a given day $b$ with probability $1/n$, so

$$\mathbb{P}(f(j) = f(i) \text{ and } f(k) = f(i)) = \sum_{b=1}^{n} \mathbb{P}(f(i) = b) \cdot \frac{1}{n} \cdot \frac{1}{n} = \frac{1}{n^2}.$$

In both cases $\mathbb{E}[X_{ij}X_{kl}] = 1/n^2 = \mathbb{E}[X_{ij}]\,\mathbb{E}[X_{kl}]$. For two indicator random variables this is the same as independence, since their joint distribution is determined by $\mathbb{P}(X_{ij} = 1)$, $\mathbb{P}(X_{kl} = 1)$ and $\mathbb{P}(X_{ij} = X_{kl} = 1)$.

For the variance, write $X - \mathbb{E}[X] = \sum_{i \neq j}(X_{ij} - \mathbb{E}[X_{ij}])$ and expand the square:

$$\begin{aligned}
\mathbb{V}[X] &= \mathbb{E}\left[\Big(\sum_{i\neq j}(X_{ij} - \mathbb{E}[X_{ij}])\Big)^2\right] \\
&= \sum_{i\neq j} \mathbb{E}\Big[(X_{ij} - \mathbb{E}[X_{ij}])^2\Big] + \sum_{\{i,j\}\neq\{k,l\}} \mathbb{E}[(X_{ij} - \mathbb{E}[X_{ij}])(X_{kl} - \mathbb{E}[X_{kl}])] \\
&= \sum_{i\neq j} \mathbb{V}[X_{ij}] + \sum_{\{i,j\}\neq\{k,l\}} (\mathbb{E}[X_{ij}X_{kl}] - \mathbb{E}[X_{ij}]\,\mathbb{E}[X_{kl}]) = \sum_{i\neq j} \mathbb{V}[X_{ij}],
\end{aligned}$$

where the second sum vanishes by the first part. Finally $X_{ij}$ is an indicator with $\mathbb{P}(X_{ij} = 1) = 1/n$, so $X_{ij}^2 = X_{ij}$ and

$$\mathbb{V}[X_{ij}] = \mathbb{E}\Big[X_{ij}^2\Big] - \mathbb{E}[X_{ij}]^2 = \frac{1}{n} - \frac{1}{n^2} = \frac{1}{n}\left(1 - \frac{1}{n}\right) \leqslant \frac{1}{n}.$$

Summing over the $\binom{m}{2}$ pairs gives $\mathbb{V}[X] \leqslant \binom{m}{2}\frac{1}{n} = \mathbb{E}[X]$. □

**Theorem 12 ▶ The Birthday Paradox Occurs**

If $m \geqslant 2\sqrt{n} + 1$, then

$$\mathbb{P}(X \geqslant 1) \geqslant \frac{1}{2}.$$

In words: with $\mathcal{O}(\sqrt{n})$ people there is a collision with constant probability.

*Proof.* The event $X = 0$ (no collision) is contained in the event $|X - \mathbb{E}[X]| \geqslant \mathbb{E}[X]$, since $X = 0$ means $X - \mathbb{E}[X] = -\mathbb{E}[X]$. By Chebyshev's inequality (Theorem 10) with $a = \mathbb{E}[X]$, and then Lemma 11,

$$\mathbb{P}(X = 0) \leqslant \mathbb{P}(|X - \mathbb{E}[X]| \geqslant \mathbb{E}[X]) \leqslant \frac{\mathbb{V}[X]}{\mathbb{E}[X]^2} \leqslant \frac{\mathbb{E}[X]}{\mathbb{E}[X]^2} = \frac{1}{\mathbb{E}[X]}.$$

For $m \geqslant 2\sqrt{n} + 1$, since $m(m-1)$ grows with $m$,

$$\mathbb{E}[X] = \frac{m(m-1)}{2n} \geqslant \frac{(2\sqrt{n}+1)\cdot 2\sqrt{n}}{2n} = \frac{4n + 2\sqrt{n}}{2n} \geqslant 2,$$

so $\mathbb{P}(X = 0) \leqslant 1/2$, and $\mathbb{P}(X \geqslant 1) = 1 - \mathbb{P}(X = 0) \geqslant 1/2$. □

The direct computation of $\mathbb{P}(\text{all birthdays different})$ above already gave $\mathbb{P}(X = 0) \leqslant 1/e$ for $m = \sqrt{2n} + 1$. The point of this *second moment* argument is that it only uses $\mathbb{E}[X]$ and $\mathbb{V}[X]$, so it also works when the exact probability cannot be computed.

# 17 Quantum Birthday Paradox

The birthday paradox, seen as an algorithm, solves the *collision problem*: given query access to a function $f$, find $x \neq y$ with $f(x) = f(y)$. Querying $m = \sqrt{2n} + 1$ random points finds a collision with probability at least $1/2$, so $\mathcal{O}(\sqrt{n})$ queries suffice classically. The quantum version of the problem is studied later in the course; here we show that classically $\sqrt{n}$ queries are also necessary. An algorithm makes $k$ queries

$$f(x_1), \ldots, f(x_k),$$

where the point $x_s$ may depend on the earlier answers $f(x_1), \ldots, f(x_{s-1})$ and on random coins.

**Exercise 3.** Show a $\sqrt{n}$ lower bound in the classical case.

We use the same model as in the birthday paradox: $f\colon D \to [n]$, where $[n] = \{1, \ldots, n\}$, is a *random function*, i.e., every value $f(x)$ is uniform in $[n]$ and independent of all the other values (the birthday of person $x$). The algorithm *finds a collision* if it outputs $x \neq y$ with $f(x) = f(y)$. We may assume that the algorithm queries the two points that it outputs: if it does not, it can do so with at most 2 extra queries, which does not change the bound below. We may also assume that the $k$ queried points are different, since repeating a query gives no new information.

**Theorem 13 ▶ Classical Lower Bound for Collision Finding**

Let $f\colon D \to [n]$ be a random function. Every classical algorithm (deterministic or randomized, adaptive) that makes $k$ queries finds a collision with probability at most

$$\frac{\binom{k}{2}}{n} = \frac{k(k-1)}{2n}.$$

In particular, to find a collision with probability at least $1/2$ it needs $k > \sqrt{n}$ queries, so the classical query complexity of the collision problem is $\Omega(\sqrt{n})$.

*Proof.* Since the algorithm queries the points that it outputs, it can only find a collision if two of its queried values agree. Let $E_s$ be the event that the $s$th query collides with an earlier one, i.e., $f(x_s) \in \{f(x_1), \ldots, f(x_{s-1})\}$. A collision among the $k$ queries means that some $E_s$ occurs, so by the union bound

$$\mathbb{P}(\text{collision}) \leqslant \sum_{s=1}^{k} \mathbb{P}(E_s).$$

We claim that $\mathbb{P}(E_s) \leqslant \frac{s-1}{n}$. Consider the moment just before the $s$th query. The algorithm chooses $x_s$ from the $s-1$ answers seen so far and from its random coins; condition on all of these. Then $x_s$ is a fixed point that has not been queried before, so its value $f(x_s)$ is still a uniformly random element of $[n]$, independent of the $s-1$ values $f(x_1), \ldots, f(x_{s-1})$, which are now fixed. The probability that $f(x_s)$ equals one of these $s-1$ fixed values is therefore at most $\frac{s-1}{n}$, no matter how the algorithm chose $x_s$. Since this holds for every outcome of the first $s-1$ steps, it also holds without conditioning, which proves the claim. Hence

$$\mathbb{P}(\text{collision}) \leqslant \sum_{s=1}^{k} \frac{s-1}{n} = \frac{0 + 1 + \cdots + (k-1)}{n} = \frac{k(k-1)}{2n}.$$

If the algorithm succeeds with probability at least $1/2$, then $\frac{k(k-1)}{2n} \geqslant \frac{1}{2}$, i.e., $k^2 > k(k-1) \geqslant n$, so $k > \sqrt{n}$. □

**Worst case inputs.** Theorem 13 is a statement about a *random* input $f$: the distributional complexity of the collision problem under the uniform distribution on functions is $\Omega(\sqrt{n})$. By Yao's minmax principle from the lecture of September 1 (in its version for algorithms that are allowed to err), this is also a lower bound for every randomized algorithm on its worst case input. Together with the birthday paradox, which finds a collision with $\mathcal{O}(\sqrt{n})$ queries, the classical query complexity of finding a collision is $\Theta(\sqrt{n})$.

**Lecture Date:** September 8, 2026

**Lecture Topic:** Distributed Computing, Randomized Leader Election in a Complete Network, Random Walks, Law of Large Numbers

In this lecture, we turn to distributed computing. We define the synchronous complete network model, study the leader election problem, and analyze a randomized algorithm that elects a leader in $\mathcal{O}(1)$ rounds with only $\mathcal{O}\left(\sqrt{n}\log^{3/2} n\right)$ messages. We then take a first look at random walks on undirected graphs, and we prove the weak law of large numbers with Chebyshev's inequality.

# 18 Distributed Computing in a Complete Network

The network is a *complete graph* $G$ on $n$ nodes (Figure 8): every pair of nodes is joined by a bidirectional edge, so there are $\binom{n}{2} = \frac{n(n-1)}{2}$ edges in total and every node has $n-1$ incident edges. The nodes are computers, and an edge is a communication link on which a message can be sent in either direction.

The network is *synchronous*: all nodes share a clock, and the computation proceeds in *rounds* (also called steps). In one round, every node

$$\text{receives} \longrightarrow \text{computes} \longrightarrow \text{sends},$$

i.e., it receives the messages that were sent to it in the previous round, does an arbitrary local computation, and sends messages on some of its edges.

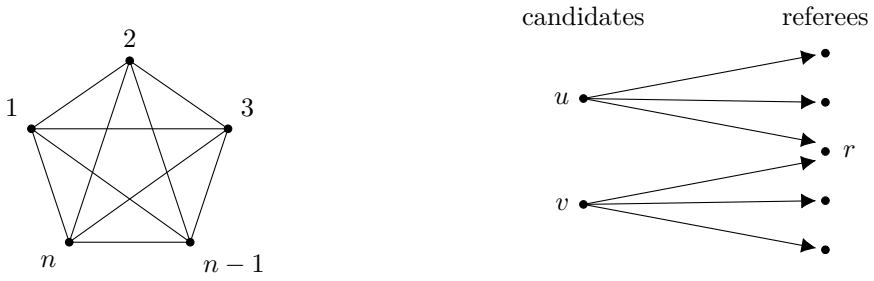



Figure 8: Left: the complete network on $n$ nodes. Right: two candidates $u$ and $v$ send their ids to their referees; the common referee $r$ sees both ids and declares the smaller one the winner.

## 18.1 Leader Election

**Problem 1 ▶ Leader Election**

All nodes run the same algorithm. At the end, exactly one node must be in the state *leader*, and all other nodes must be in the state *non-leader*.

There are two goals.

- Finish fast: minimize the number of rounds.
- Less communication: minimize the total number of messages sent.

The nodes are either *labelled*, i.e., every node has a unique id, or *anonymous*, i.e., the nodes have no ids and cannot be told apart.

**The trivial algorithm.** With unique ids, every node sends its id to all other nodes in the first round, and the node with the minimum id becomes the leader. Every node learns all $n$ ids, so exactly one node sees that its own id is the minimum. Messages are sent in 1 round only (the ids are received and compared at the start of round 2), so this takes $\mathcal{O}(1)$ rounds, and each of the $n$ nodes sends $n-1$ messages, so the total number of messages is $n(n-1) = \Theta(n^2)$: two on every edge.

**Anonymous nodes.** For anonymous nodes there is no deterministic algorithm. To state this precisely we have to say how a node addresses its edges: it has $n-1$ *ports*, numbered $1, \ldots, n-1$, one per incident edge, and it can send a message on a given port and sees on which port a message arrives. The node does not know which node is at the other end of a port, and the port numbering is not under the control of the algorithm (we may think of it as chosen by an adversary).

**Theorem 14 ▶ No Deterministic Leader Election for Anonymous Nodes**

Let $n \geqslant 2$. In an anonymous complete network in which all nodes start in the same state, there is no deterministic algorithm that elects a leader for every port numbering.

*Proof.* Fix a deterministic algorithm. We choose the following symmetric port numbering. Identify the nodes with $\mathbb{Z}_n = \{0, 1, \ldots, n-1\}$, with addition modulo $n$, and let port $j$ of node $i$ lead to node $i+j$, for $j = 1, \ldots, n-1$. This is a valid port numbering: the nodes $i+1, \ldots, i+(n-1)$ are exactly the $n-1$ nodes other than $i$. The edge between $i$ and $i+j$ is port $j$ at $i$ and port $n-j$ at $i+j$, since $(i+j)+(n-j) = i$ in $\mathbb{Z}_n$. Hence a message that some node sends on port $j$ arrives at its destination on port $n-j$.

We show by induction on $t$ that at the end of round $t$ all nodes are in the same state, and that in round $t$ all nodes send the same message on port $j$, for every $j$ (where "no message" counts as a message). For $t = 0$ all nodes are in the same initial state, and nothing has been sent. Assume that the claim holds for round $t-1$, and let $m_j$ be the message that every node sent on port $j$ in round $t-1$. In round $t$, node $i$ receives on port $q$ the message that node $i+q$ sent on its port $n-q$, which is $m_{n-q}$. This does not depend on $i$, so all nodes receive the same messages on the same ports. They were in the same state, and the algorithm is deterministic, so they compute the same new state and send the same messages on the same ports. This proves the claim.

Whether a node declares itself the leader is a function of its state. Therefore, at every time, either all $n \geqslant 2$ nodes are leaders or no node is a leader, and there is never exactly one leader. □

A randomized algorithm can break this symmetry: if every node picks an id uniformly at random in $\{1, \ldots, n^4\}$, then two given nodes pick the same id with probability $1/n^4$, so by the union bound over the $\binom{n}{2} \leqslant n^2/2$ pairs all ids are different with probability at least $1 - \frac{1}{2n^2}$. From now on we assume that the nodes are non-anonymous and have unique ids.

**Complexity classes.** The same three kinds of algorithms (deterministic, randomized, quantum) appear in the complexity classes

$$\mathrm{P} \subseteq \mathrm{BPP} \subseteq \mathrm{BQP},$$

where P is deterministic polynomial time, BPP is bounded-error probabilistic polynomial time, and BQP is bounded-error quantum polynomial time. The first inclusion holds because a deterministic algorithm is a randomized algorithm that ignores its coins; the second is recorded without proof.

### 18.2 Known Bounds

For leader election in a complete network of $n$ nodes with unique ids, the following bounds were listed in class.

| Messages | Rounds | Algorithm |
|---|---|---|
| $\Theta(n^2)$ | $\mathcal{O}(1)$ | trivial, deterministic |
| $\mathcal{O}(n \log n)$ | $\mathcal{O}(\log n)$ | deterministic |
| $\mathcal{O}\left(\sqrt{n} \log^{3/2} n\right) = \widetilde{O}(\sqrt{n})$ | $\mathcal{O}(1)$ | randomized (this lecture) |
| $\widetilde{\Omega}(\sqrt{n})$ | | classical lower bound |
| $\widetilde{O}(n^{1/3})$ | | quantum |

Here $\widetilde{O}$ and $\widetilde{\Omega}$ hide factors that are polynomial in $\log n$. The first row is the trivial algorithm above, and the third row is the algorithm of the next section. The other rows are recorded as stated, without proof. In particular, by the fourth row the randomized algorithm is optimal among classical algorithms up to logarithmic factors, and by the fifth a quantum algorithm gets below this classical lower bound.

## 19 A Randomized Algorithm for Leader Election

Throughout, log is the natural logarithm, and

$$p = \frac{2 \log n}{n}, \qquad k = \left\lceil \sqrt{n \log n} \right\rceil.$$

We have $p \leqslant 1$ and $k \leqslant n$ for every $n \geqslant 1$, since $2 \log n \leqslant n$. The algorithm has four steps.

1. **Subsample candidates.** Every node becomes a *candidate* with probability $p$, independently of all other nodes.
2. **Candidates contact referees.** Every candidate sends its id to $k$ random nodes, its *referees*.
3. **Referee round.** Every node that received at least one id declares the minimum id among the received messages the *winner*, and replies with the winner's id to all candidates that contacted it.
4. **Decision.** A candidate becomes the leader if it is the winner at all of its referees. All other nodes are non-leaders.

**The model for the random choices.** We fix the following precise model, which the analysis below uses. Every node $u$ draws a coin $C_u \in \{0, 1\}$ with $\mathbb{P}(C_u = 1) = p$ ($u$ is a candidate if $C_u = 1$) and a *referee set* $R_u$, which is a uniformly random set of $k$ distinct nodes among all $n$ nodes. All the $C_u$ and all the $R_u$ are mutually independent. Only candidates use their referee sets. A candidate may pick itself as a referee; then it plays the role of this referee itself, and no message is needed.

### 19.1 The Number of Candidates

Let $X = \sum_u C_u$ be the number of candidates. By linearity of expectation,

$$\mathbb{E}[X] = \frac{2 \log n}{n} \cdot n = 2 \log n.$$

**Exercise 4.** Show that, with high probability, there is at least 1 leader.

The solution is Theorem 15.

**Theorem 15 ▶ At Least One Leader**

There is at least one leader if and only if there is at least one candidate, and

$$\mathbb{P}(\text{no leader}) = \mathbb{P}(X = 0) \leqslant \frac{1}{n^2}, \qquad \text{i.e.,} \qquad \mathbb{P}(\text{at least 1 leader}) \geqslant 1 - \frac{1}{n^2}.$$

*Proof.* Only candidates become leaders, so if $X = 0$ there is no leader. Conversely, assume $X \geqslant 1$, and let $w$ be the candidate with the minimum id among all candidates (it is unique, since the ids are unique). Let $r$ be any referee of $w$. The ids that $r$ receives are ids of candidates, so all of them are at least the id of $w$, and the id of $w$ is one of them. Hence $r$ declares $w$ the winner. This holds for every referee of $w$, so $w$ is a leader.

It remains to bound $\mathbb{P}(X = 0)$. The coins are independent, so, using $1 - x \leqslant e^{-x}$,

$$\mathbb{P}(X = 0) = \left(1 - \frac{2\log n}{n}\right)^n \leqslant e^{-\frac{2\log n}{n} \cdot n} = e^{-2\log n} = \frac{1}{n^2}.$$

Here we used $0 \leqslant 1 - \frac{2\log n}{n}$, so that taking the $n$th power preserves the inequality. □

**Remark (the Chernoff computation).** In class, $\mathbb{P}(X < 1)$ was bounded "from Chernoff bounds", with $(1-\delta)\cdot 2\log n = 1$, i.e., $\delta = 1 - \frac{1}{2\log n}$. The lower tail of Theorem 5 is an inequality with a factor $1/2$ in the exponent, and with $\mu = 2\log n$ it gives

$$\mathbb{P}(X < 1) \leqslant e^{-\frac{\mu\delta^2}{2}} = e^{-\log n\left(1-\frac{1}{2\log n}\right)^2} = \frac{e}{n} \cdot e^{-\frac{1}{4\log n}} \leqslant \frac{e}{n}$$

for $n \geqslant 2$. So the lower tail bound in this form only gives $\mathcal{O}(1/n)$ here; the value $1/n^2$ recorded in class is the one that the direct computation above gives.

## 19.2 At Most One Leader

**Exercise 5.** Show that, with probability tending to 1, there is at most 1 leader.

We prove this in Theorem 18, using Lemmas 16 and 17. In the notes this is written "almost 1 leader"; we read it as "at most 1 leader". Read literally, as "almost surely exactly 1 leader", it is Theorem 19 below.

Call a pair of different nodes $\{u, v\}$ a *bad pair* if both are candidates and they have no common referee, i.e., $C_u = C_v = 1$ and $R_u \cap R_v = \emptyset$.

**Lemma 16 ▶ Two Leaders Form a Bad Pair**

If two candidates have a common referee, then at most one of them is a leader. Consequently, if there is more than one leader, then there is a bad pair.

*Proof.* Let $u \neq v$ be candidates with a common referee $r$, and assume that the id of $u$ is smaller than the id of $v$ (Figure 8). The referee $r$ receives both ids (if $r$ is $u$ or $v$ itself, it knows its own id), so the minimum id that it receives is at most the id of $u$, which is smaller than the id of $v$. Hence $r$ does not declare $v$ the winner, and $v$ is not a leader. Consequently, two different leaders are two candidates without a common referee, i.e., a bad pair. □

**Lemma 17 ▶ Two Referee Sets Rarely Miss Each Other**

For any two different nodes $u$ and $v$,

$$\mathbb{P}(R_u \cap R_v = \emptyset) \leqslant \left(1 - \frac{k}{n}\right)^k \leqslant e^{-k^2/n} \leqslant \frac{1}{n}.$$

*Proof.* Condition on $R_u = S$, where $S$ is any set of $k$ nodes. Since $R_v$ is independent of $R_u$, it is still a uniformly random set of $k$ nodes. If $2k > n$, two sets of $k$ nodes cannot be disjoint, the probability is 0, and there is nothing to show. So let $2k \leqslant n$. We draw the $k$ nodes of $R_v$ one after the other, each uniformly among the nodes not drawn yet. Assume that the first $i$ nodes drawn are outside $S$. Then $n - i$ nodes are left, and $k$ of them are in $S$, so the next node is outside $S$ with probability $\frac{n-k-i}{n-i}$. Hence

$$\mathbb{P}(R_v \cap S = \emptyset) = \prod_{i=0}^{k-1} \frac{n-k-i}{n-i} = \prod_{i=0}^{k-1} \left(1 - \frac{k}{n-i}\right) \leqslant \left(1 - \frac{k}{n}\right)^k,$$

because every factor satisfies $0 \leqslant 1 - \frac{k}{n-i} \leqslant 1 - \frac{k}{n}$ (the first inequality holds since $i \leqslant k-1$ and $2k \leqslant n$ give $n - i \geqslant k$). This bound does not depend on $S$, so it also holds without conditioning. Finally, $1 - x \leqslant e^{-x}$ and $k^2 \geqslant n \log n$ give

$$\left(1 - \frac{k}{n}\right)^k \leqslant e^{-\frac{k}{n} \cdot k} = e^{-k^2/n} \leqslant e^{-\frac{n \log n}{n}} = e^{-\log n} = \frac{1}{n}.$$ □

With $k = \sqrt{n \log n}$, the last display is the computation from class. In the notes its left-hand side is labelled as the probability of at most one leader; the quantity that it bounds is the probability that one given pair of candidates has no common referee. The number of pairs of candidates is random, so we finish with an expectation over all pairs of nodes.

**Theorem 18 ▶ At Most One Leader**

$$\mathbb{P}(\text{more than 1 leader}) \leqslant \frac{2 \log^2 n}{n}.$$

*Proof.* Let $B$ be the number of bad pairs. For a fixed pair $\{u, v\}$, the coins $C_u$, $C_v$ and the pair of sets $(R_u, R_v)$ are independent, so by Lemma 17

$$\mathbb{P}(\{u, v\} \text{ is a bad pair}) = \mathbb{P}(C_u = 1) \cdot \mathbb{P}(C_v = 1) \cdot \mathbb{P}(R_u \cap R_v = \emptyset) \leqslant p^2 \cdot \frac{1}{n}.$$

By linearity of expectation over the $\binom{n}{2}$ pairs of nodes,

$$\mathbb{E}[B] \leqslant \binom{n}{2} \frac{p^2}{n} = \frac{n(n-1)}{2} \cdot \frac{4 \log^2 n}{n^2} \cdot \frac{1}{n} = \frac{2(n-1) \log^2 n}{n^2} \leqslant \frac{2 \log^2 n}{n}.$$

If there is more than one leader, then $B \geqslant 1$ by Lemma 16. Since $B \geqslant 0$, Markov's inequality (Theorem 9) with $b = 1$ gives

$$\mathbb{P}(\text{more than 1 leader}) \leqslant \mathbb{P}(B \geqslant 1) \leqslant \mathbb{E}[B] \leqslant \frac{2 \log^2 n}{n}.$$ □

**Theorem 19 ▶ Exactly One Leader**

The algorithm elects exactly one leader with probability at least

$$1 - \frac{1}{n^2} - \frac{2 \log^2 n}{n}.$$

*Proof.* The algorithm fails if there is no leader or more than one leader. By the union bound and Theorems 15 and 18, this has probability at most $\frac{1}{n^2} + \frac{2\log^2 n}{n}$. □

**Remark (the failure probability).** The bound $\frac{2\log^2 n}{n}$ tends to 0, but it is not of the form $1/n^c$ with $c \geqslant 1$. This is easy to repair: let every candidate contact $k' = \lceil 2\sqrt{n \log n} \rceil$ referees instead of $k$ (for $n \geqslant 9$ we have $4 \log n \leqslant n$ and hence $k' \leqslant n$). The proof of Lemma 17 goes through with $k'$ in place of $k$ and gives $e^{-k'^2/n} \leqslant e^{-4\log n} = 1/n^4$, so that $\mathbb{E}[B] \leqslant \binom{n}{2}\frac{p^2}{n^4} \leqslant \frac{2\log^2 n}{n^4}$. With $k'$ referees the algorithm therefore fails with probability at most $\frac{1}{n^2} + \frac{2\log^2 n}{n^4} \leqslant \frac{2}{n^2}$ (using $2\log^2 n \leqslant n^2$), and the number of messages only doubles.

### 19.3 Messages and Rounds

**Theorem 20 ▶ Complexity of the Algorithm**

The algorithm takes 3 rounds, and messages are sent only in the first two. The expected number of messages is at most $4k \log n = \mathcal{O}\left(\sqrt{n}\log^{3/2} n\right)$. Moreover, with probability at least $1 - 1/n^8$, the number of messages is at most $24k \log n = \mathcal{O}\left(\sqrt{n}\log^{3/2} n\right)$.

*Proof. Rounds.* In round 1 every node draws its coin and its referee set, and, if it is a candidate, sends its id to its referees. In round 2 the referees receive the ids, compute the minimum, and send the replies. In round 3 the candidates receive the replies and decide; no message is sent.

*Messages.* Every candidate sends at most $k$ messages and gets at most $k$ replies, so the number of messages is at most $2kX$. Its expectation is at most $2k\,\mathbb{E}[X] = 4k \log n$, and since $k \leqslant \sqrt{n \log n} + 1$, this is $\mathcal{O}\left(\sqrt{n}\log^{3/2} n\right)$.

For the second statement, $X$ is a sum of independent indicator random variables with mean $\mu = 2\log n$, so Theorem 5 with $R = 12\log n = 6\mu$ gives

$$\mathbb{P}(X \geqslant 12\log n) \leqslant 2^{-12\log n} = e^{-12\log 2 \cdot \log n} = n^{-12\log 2} \leqslant \frac{1}{n^8},$$

since $12\log 2 \geqslant 8.3$. If $X < 12\log n$, the number of messages is at most $2kX \leqslant 24k\log n$. □

## 20 Markov Chains: Random Walks in Undirected Graphs

**Definition 4 ▶ Random Walk**

Let $G$ be an undirected graph in which every node has at least one neighbour. A **random walk** on $G$ is the following process: a walker starts at some node, and in every step it moves from its current node $u$ to a neighbouring node chosen with uniform probability, i.e., to each neighbour of $u$ with probability $\frac{1}{\deg(u)}$, independently of the earlier steps.

We describe the walk by the row vector $p_t = (p_t^1, \ldots, p_t^n)$, where $p_t^i$ is the probability that the walker is at node $i$ after $t$ steps (this $p_t$ is not related to the candidate probability $p$ of the last section). Since the walker is at exactly one node, the entries of $p_t$ sum to 1. If the walker is at node $j$ after $t$ steps, it moves to a given neighbour $i$ of $j$ with probability $\frac{1}{\deg(j)}$, so by the law of total probability

$$p_{t+1}^i = \sum_{j\,:\,j \text{ is a neighbour of } i} \frac{p_t^j}{\deg(j)}. \tag{1}$$

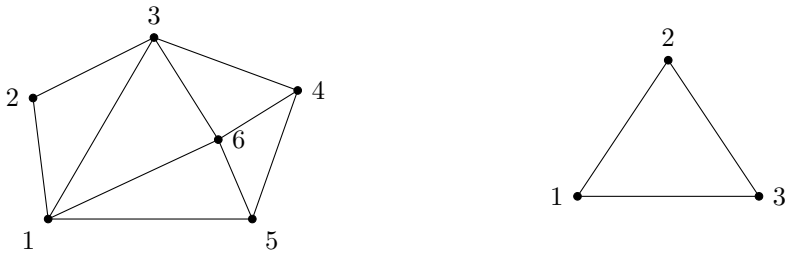



Figure 9: Left: a graph with 6 nodes and 10 edges. Right: the triangle. In both examples the walk starts at node 1.

**Example 1.** In the graph on the left of Figure 9, let the walker start at node 1, so that

$$p_0 = (1, 0, 0, 0, 0, 0).$$

Node 1 has the 4 neighbours 2, 3, 5, 6, each of which is chosen with probability 1/4, so

$$p_1 = \left(0, \tfrac{1}{4}, \tfrac{1}{4}, 0, \tfrac{1}{4}, \tfrac{1}{4}\right).$$

**Example 2.** In the triangle on the right of Figure 9, every node has degree 2. Starting at node 1,

$$p_0 = (1, 0, 0), \qquad p_1 = \left(0, \tfrac{1}{2}, \tfrac{1}{2}\right), \qquad p_2 = \left(\tfrac{1}{2}, \tfrac{1}{4}, \tfrac{1}{4}\right), \qquad p_t = \left(p_t^1, p_t^2, p_t^3\right).$$

For instance, by Equation (1), $p_2^1 = \frac{1}{2}p_1^2 + \frac{1}{2}p_1^3 = \frac{1}{2}$ and $p_2^2 = \frac{1}{2}p_1^1 + \frac{1}{2}p_1^3 = \frac{1}{4}$.

**Exercise 6.** Compute $p_t = (p_t^1, p_t^2, p_t^3)$ for all $t \geqslant 0$ for the random walk on the triangle that starts at node 1.

The solution is Lemma 21.

**Lemma 21 ▶ Random Walk on the Triangle**

For the random walk on the triangle that starts at node 1, for all $t \geqslant 0$,

$$p_t^1 = \frac{1}{3} + \frac{2}{3}\left(-\frac{1}{2}\right)^t, \qquad p_t^2 = p_t^3 = \frac{1}{3} - \frac{1}{3}\left(-\frac{1}{2}\right)^t.$$

In particular, $p_t$ converges to $\left(\frac{1}{3}, \frac{1}{3}, \frac{1}{3}\right)$ as $t \to \infty$.

*Proof.* Let $i$ be any of the three nodes, and let $j$ and $l$ be the other two. Both are neighbours of $i$ and have degree 2, so by Equation (1), and since $p_t^i + p_t^j + p_t^l = 1$,

$$p_{t+1}^i = \frac{p_t^j + p_t^l}{2} = \frac{1 - p_t^i}{2}, \qquad \text{and hence} \qquad p_{t+1}^i - \frac{1}{3} = \frac{1 - p_t^i}{2} - \frac{1}{3} = -\frac{1}{2}\left(p_t^i - \frac{1}{3}\right).$$

So the difference $p_t^i - 1/3$ is multiplied by $-1/2$ in every step, and by induction on $t$

$$p_t^i = \frac{1}{3} + \left(p_0^i - \frac{1}{3}\right)\left(-\frac{1}{2}\right)^t.$$

With $p_0^1 = 1$ and $p_0^2 = p_0^3 = 0$ this gives the two formulas. Since $\left(-\frac{1}{2}\right)^t \to 0$, every entry converges to 1/3. □

For $t = 0, 1, 2$ the formulas give $(1, 0, 0)$, $\left(0, \frac{1}{2}, \frac{1}{2}\right)$ and $\left(\frac{1}{2}, \frac{1}{4}, \frac{1}{4}\right)$, in agreement with the example. The next lecture writes Equation (1) as a multiplication by a matrix, shows in general that $p_t$ stays a probability vector (Theorem 25), and finds the distribution that the walk does not change: it is proportional to the degrees (Theorem 26), which for the triangle is the uniform distribution found here.

## 21 The Law of Large Numbers

**Exercise 7.** Show the "law of large numbers" with Chebyshev's inequality.

The law of large numbers says that the average of many independent samples of a random variable is close to its expectation. The version that follows from Chebyshev's inequality is the *weak* law, Theorem 22.

**Theorem 22 ▶ Weak Law of Large Numbers**

Let $X_1, X_2, \ldots$ be pairwise independent random variables with the same expectation $\mathbb{E}[X_i] = \mu$ and the same finite variance $\mathbb{V}[X_i] = \sigma^2$ (for example, independent and identically distributed with finite variance), and let $\overline{X}_n = \frac{1}{n}\sum_{i=1}^{n} X_i$. Then, for every $\varepsilon > 0$,

$$\mathbb{P}\Big(\Big|\overline{X}_n - \mu\Big| \geqslant \varepsilon\Big) \leqslant \frac{\sigma^2}{n\varepsilon^2} \longrightarrow 0 \qquad \text{as } n \to \infty.$$

*Proof.* By linearity of expectation,

$$\mathbb{E}\Big[\overline{X}_n\Big] = \frac{1}{n}\sum_{i=1}^{n} \mathbb{E}[X_i] = \frac{n\mu}{n} = \mu.$$

For the variance, write $Y_i = X_i - \mu$, so that $\mathbb{E}[Y_i] = 0$, $\mathbb{E}[Y_i^2] = \sigma^2$, and $\overline{X}_n - \mu = \frac{1}{n}\sum_{i=1}^{n} Y_i$. Expanding the square as in the proof of Lemma 11,

$$\mathbb{V}\Big[\overline{X}_n\Big] = \mathbb{E}\left[\Big(\frac{1}{n}\sum_{i=1}^{n} Y_i\Big)^2\right] = \frac{1}{n^2}\sum_{i=1}^{n} \mathbb{E}\Big[Y_i^2\Big] + \frac{1}{n^2}\sum_{i \neq j} \mathbb{E}[Y_i Y_j].$$

For $i \neq j$, the variables $Y_i$ and $Y_j$ are independent, so $\mathbb{E}[Y_i Y_j] = \mathbb{E}[Y_i]\,\mathbb{E}[Y_j] = 0$, and the second sum vanishes. Hence

$$\mathbb{V}\Big[\overline{X}_n\Big] = \frac{1}{n^2} \cdot n\sigma^2 = \frac{\sigma^2}{n}.$$

Chebyshev's inequality (Theorem 10), applied to the random variable $\overline{X}_n$ with $a = \varepsilon$, now gives

$$\mathbb{P}\Big(\Big|\overline{X}_n - \mu\Big| \geqslant \varepsilon\Big) = \mathbb{P}\Big(\Big|\overline{X}_n - \mathbb{E}\Big[\overline{X}_n\Big]\Big| \geqslant \varepsilon\Big) \leqslant \frac{\mathbb{V}\Big[\overline{X}_n\Big]}{\varepsilon^2} = \frac{\sigma^2}{n\varepsilon^2},$$

which tends to 0 as $n \to \infty$ for every fixed $\varepsilon > 0$. □

The statement that $\mathbb{P}\Big(\Big|\overline{X}_n - \mu\Big| \geqslant \varepsilon\Big) \to 0$ for every $\varepsilon > 0$ is called *convergence in probability* of $\overline{X}_n$ to $\mu$, and this is what "weak" refers to.

**Lecture Date:** September 10, 2026

**Lecture Topic:** Law of Large Numbers, Random Walks on Graphs, Transition Matrix, Stationary Distribution and Convergence

In this lecture, we state the law of large numbers and return to random walks on graphs. We describe a random walk by its transition matrix $P$, show that the distribution of the token evolves as $p_{t+1} = p_t P$, and study the stationary distribution $\pi = \pi P$ and when the walk converges to it.

## 22 Law of Large Numbers

Let $X_1, X_2, \ldots, X_n$ be independent, identically distributed (i.i.d.) random variables, each distributed as a random variable $X$ with finite expectation $\mathbb{E}[X]$. The law of large numbers says that the average of the samples approaches the expectation.

**Theorem 23 ▶ Law of Large Numbers, as Stated in Class**

Let $X_1, X_2, \ldots$ be i.i.d. random variables distributed as $X$, where $\mathbb{E}[X]$ is finite. Then, with probability 1,

$$\lim_{n\to\infty} \frac{\sum_{i=1}^{n} X_i}{n} = \mathbb{E}[X].$$

**Remark.** There are two laws of large numbers, and they differ in the kind of convergence. The statement above, convergence with probability 1, is the *strong* law; we do not prove it here. The *weak* law is convergence in probability, which is implied by convergence with probability 1. The weak law is the one that we proved from Chebyshev's inequality in Theorem 22, under the additional assumption that $\mathbb{V}[X]$ is finite. In the notes the heading is "weak law" and the statement written under it is the strong one.

## 23 Random Walks on Graphs

As in the last lecture, a *random walk* on a finite undirected graph $G = (V, E)$ moves a token in every step from its current vertex $u$ to a uniformly random neighbour of $u$. We take $V = \{1, \ldots, n\}$, write $m = |E|$ for the number of edges, $u \sim v$ if $\{u, v\} \in E$, and $\deg(u)$ (or $d_u$) for the number of neighbours of $u$, and we assume throughout that $G$ has no isolated vertices. Let $X_t$ be the vertex of the token at time $t = 0, 1, 2, \ldots$.

**Definition 5 ▶ Distribution at Time $t$, Transition Matrix**

The **distribution at time** $t$ is the row vector

$$p_t = \left(p_t^1, p_t^2, \ldots, p_t^n\right), \qquad p_t^i = \mathbb{P}(X_t = i),$$

i.e., $p_t^i$ is the probability that the token is at vertex $i$ at time $t$ (the subscript is the time, the superscript is the vertex). The **transition matrix** is the $n \times n$ matrix $P$ with entries

$$P_{uv} = \mathbb{P}(u \to v) = \mathbb{P}(X_{t+1} = v \mid X_t = u) = \begin{cases} \frac{1}{\deg(u)} & \text{if } u \sim v, \\ 0 & \text{otherwise.} \end{cases}$$

The entries $P_{uv}$ do not depend on $t$, and the next vertex depends only on the current one. A process with these two properties is a (time-homogeneous) *Markov chain*.

## 23.1 The Triangle

Consider the triangle on the vertices $1, 2, 3$ with the token starting at vertex 1 (the right half of Figure 9). Every vertex has degree 2, so the token moves to each of the two other vertices with probability $1/2$ and

$$P = \begin{pmatrix} 0 & 1/2 & 1/2 \\ 1/2 & 0 & 1/2 \\ 1/2 & 1/2 & 0 \end{pmatrix}.$$

## 23.2 The Evolution $p_{t+1} = p_t P$

**Lemma 24 ▶ Evolution of the Distribution**

For every $t \geqslant 0$,

$$p_{t+1} = p_t P, \qquad \text{and hence} \qquad p_t = p_0 P^t.$$

*Proof.* Fix a vertex $v$. The events $\{X_t = u\}$, $u \in V$, are disjoint and one of them occurs, so

$$p^v_{t+1} = \mathbb{P}(X_{t+1} = v) = \sum_{u \in V} \mathbb{P}(X_t = u \text{ and } X_{t+1} = v) = \sum_{u \in V} p^u_t P_{uv} = (p_t P)_v.$$

Here $\mathbb{P}(X_t = u \text{ and } X_{t+1} = v) = p^u_t P_{uv}$ by the definition of $P_{uv}$ as a conditional probability when $p^u_t > 0$, and both sides are 0 when $p^u_t = 0$. The last expression is the $v$th entry of the product of the row vector $p_t$ with the matrix $P$. This holds for every $v$, so $p_{t+1} = p_t P$. The formula $p_t = p_0 P^t$ follows by induction on $t$: it is true for $t = 0$ since $P^0 = I$, and if $p_t = p_0 P^t$ then $p_{t+1} = p_t P = p_0 P^{t+1}$. □

For the triangle we can verify this directly. Since $p_0 = (1, 0, 0)$, the product $p_0 P$ is the first row of $P$, and

$$\begin{aligned} p_1 &= p_0 P = (0, 1/2, 1/2), \\ p_2 &= p_1 P = p_0 P^2 = \tfrac{1}{2}(1/2, 0, 1/2) + \tfrac{1}{2}(1/2, 1/2, 0) = (1/2, 1/4, 1/4), \end{aligned}$$

which are the vectors computed in the last lecture. The closed form of $p_t$ for all $t$ is Lemma 21.

## 23.3 Row Stochastic Matrices

Every row of the triangle's matrix $P$ has nonnegative entries that sum to 1; in other words, the $L_1$ norm of every row is 1.

**Definition 6 ▶ Probability Vector, Row Stochastic Matrix**

A vector $p = (p^1, \ldots, p^n)$ is a **probability vector** if $p^u \geqslant 0$ for all $u$ and $\sum_{u=1}^n p^u = 1$. An $n \times n$ matrix $P$ is **row stochastic** if every row of $P$ is a probability vector:

$$P_{uv} \geqslant 0 \text{ for all } u, v, \qquad \text{and} \qquad \sum_{v=1}^{n} P_{uv} = 1 \text{ for all } u.$$

The transition matrix of a random walk on any graph is row stochastic: its entries are 0 or $1/\deg(u)$, and row $u$ has exactly $\deg(u)$ entries equal to $1/\deg(u)$. The nonnegativity is part of the definition and is needed below.

**Exercise 8.** Show that $p_{t+1}$ is a probability vector, assuming that $p_t$ is.

The solution is Theorem 25.

**Theorem 25 ▶ A Row Stochastic Matrix Preserves Probability Vectors**

Let $P$ be a row stochastic $n \times n$ matrix and let $p$ be a probability vector. Then $pP$ is a probability vector. Consequently, if $p_0$ is a probability vector and $p_{t+1} = p_t P$ for all $t \geqslant 0$, then $p_t = p_0 P^t$ is a probability vector for every $t \geqslant 0$.

*Proof.* Let $q = pP$, so that $q^v = \sum_{u=1}^{n} p^u P_{uv}$ for every $v$.

*Nonnegativity.* Every term $p^u P_{uv}$ is a product of two nonnegative numbers, since $p^u \geqslant 0$ ($p$ is a probability vector) and $P_{uv} \geqslant 0$ ($P$ is row stochastic). A sum of nonnegative numbers is nonnegative, so $q^v \geqslant 0$ for every $v$.

*The entries sum to* 1. We have

$$\sum_{v=1}^{n} q^v = \sum_{v=1}^{n} \sum_{u=1}^{n} p^u P_{uv} = \sum_{u=1}^{n} \sum_{v=1}^{n} p^u P_{uv} = \sum_{u=1}^{n} p^u \left( \sum_{v=1}^{n} P_{uv} \right) = \sum_{u=1}^{n} p^u \cdot 1 = 1.$$

The second equality exchanges the order of two finite sums (both sides add up the same $n^2$ numbers $p^u P_{uv}$), the fourth uses that row $u$ of $P$ sums to 1, and the last that the entries of $p$ sum to 1. Hence $q = pP$ is a probability vector.

*All times.* We prove by induction on $t$ that $p_t$ is a probability vector. For $t = 0$ this is the assumption on $p_0$. If $p_t$ is a probability vector, then by the first part (with $p = p_t$) so is $p_t P = p_{t+1}$, which is exactly the statement of Exercise 8. □

Both halves of "row stochastic" are used: the nonnegativity of the entries of $P$ gives $p_{t+1}^v \geqslant 0$, and the unit row sums give $\sum_v p_{t+1}^v = 1$.

### 23.4 The Transition Matrix of a General Graph

Let $A$ be the *adjacency matrix* of $G$, i.e., $A_{uv} = 1$ if $u \sim v$ and $A_{uv} = 0$ otherwise, and let $D$ be the diagonal matrix of the degrees. Then

$$D = \begin{pmatrix} d_1 & & \\ & \ddots & \\ & & d_n \end{pmatrix}, \qquad D^{-1} = \begin{pmatrix} 1/d_1 & & \\ & \ddots & \\ & & 1/d_n \end{pmatrix}, \qquad P = D^{-1}A.$$

Indeed, multiplying $A$ from the left by the diagonal matrix $D^{-1}$ multiplies row $u$ of $A$ by $1/d_u$, so $(D^{-1}A)_{uv} = A_{uv}/\deg(u)$, which is $P_{uv}$ of Definition 5. The inverse $D^{-1}$ exists because there are no isolated vertices. (The diagonal matrix drawn in the notes next to $D^{-1}$ shows the entries $d_1, \ldots, d_n$, i.e., $D$ itself.)

If $G$ is *$d$-regular*, i.e., every vertex has the same degree $d$, then $D = dI$ and the formula becomes

$$P = \frac{A}{d}.$$

This is the regular case only; the triangle is 2-regular, and its $P$ above is $A/2$.

# 24 Stationary Distribution

**Definition 7 ▶ Stationary Distribution**

A probability vector $\pi$ is a **stationary distribution** of the walk with transition matrix $P$ if

$$\pi P = \pi \cdot 1 = \pi,$$

i.e., $\pi$ is a left eigenvector of $P$ with eigenvalue 1.

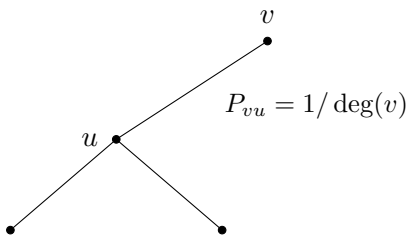



Figure 10: A vertex $u$ and its neighbours. Each neighbour $v$ contributes $\pi_v/\deg(v)$ to $(\pi P)_u$.

By Lemma 24, if $p_t = \pi$ then $p_{t+1} = \pi P = \pi$: a walk whose distribution is $\pi$ keeps the distribution $\pi$ forever. To find $\pi$ we write out the $u$th entry of $\pi P$. Only the neighbours $v$ of $u$ have $P_{vu} \neq 0$, and for them $P_{vu} = 1/\deg(v)$ (Figure 10), so

$$(\pi P)_u = \sum_{v \in V} \pi_v P_{vu} = \sum_{v \sim u} \frac{\pi_v}{\deg(v)}. \tag{2}$$

(The notes stop at $(\pi P)_u = \sum_{v \sim u} \pi_v$, in which the factor $1/\deg(v)$ is missing.) In words, the probability of being at $u$ after one step is collected from the neighbours of $u$, and a neighbour $v$ sends to $u$ a $1/\deg(v)$ fraction of its own probability.

**Theorem 26 ▶ Stationary Distribution of a Random Walk on a Graph**

Let $G$ be a graph with $m$ edges and no isolated vertices. Then

$$\pi_v = \frac{\deg(v)}{2m} \qquad (v \in V)$$

is a stationary distribution of the random walk on $G$.

*Proof.* *$\pi$ is a probability vector.* Since there are no isolated vertices and $V$ is not empty, we have $m \geqslant 1$, so $\pi$ is well defined, and $\pi_v \geqslant 0$. Every edge $\{u, v\}$ is counted exactly twice in $\sum_{v \in V} \deg(v)$, once in $\deg(u)$ and once in $\deg(v)$, so $\sum_{v \in V} \deg(v) = 2m$ (the handshake lemma). Hence $\sum_{v \in V} \pi_v = 2m/2m = 1$.

$\pi P = \pi$. For every vertex $u$, by Equation (2),

$$(\pi P)_u = \sum_{v \sim u} \frac{\pi_v}{\deg(v)} = \sum_{v \sim u} \frac{\deg(v)}{2m} \cdot \frac{1}{\deg(v)} = \sum_{v \sim u} \frac{1}{2m} = \frac{\deg(u)}{2m} = \pi_u,$$

where we used $\deg(v) \geqslant 1$ to cancel, and that the sum has one term for each of the $\deg(u)$ neighbours of $u$. □

For a $d$-regular graph on $n$ vertices we have $2m = nd$, so $\pi_v = d/nd = 1/n$: the stationary distribution is uniform. For the triangle ($m = 3$, all degrees 2) it is $\pi = (1/3, 1/3, 1/3)$, which is the limit of the closed form of Lemma 21 as $t \to \infty$.

## 25 Convergence to the Stationary Distribution

The reason to study $\pi$ is that, as $t \to \infty$, the distribution $p_t$ of the walk approaches it: $p_t \to \pi$. Here convergence of vectors is entry by entry. Two conditions on the graph were given in class: (1) the graph should be connected, and (2) it should be non-bipartite. These are the conditions for the walk to converge to one and the same limit from every start; a stationary distribution itself exists for every graph without isolated vertices, by Theorem 26.

**Theorem 27 ▶ Convergence of Random Walks**

Let $G$ be a connected, non-bipartite graph with $m$ edges. Then for every starting distribution $p_0$,

$$p_t = p_0 P^t \longrightarrow \pi \qquad \text{as } t \to \infty, \qquad \text{where } \pi_v = \frac{\deg(v)}{2m}.$$

In particular, $\pi$ is the only stationary distribution of the walk.

We do not prove the convergence. It is the fundamental theorem of Markov chains (the convergence theorem for irreducible and aperiodic chains); see Levin, Peres and Wilmer, *Markov Chains and Mixing Times*, Chapter 4. It can also be derived from the Perron–Frobenius theorem. A random walk on a graph is irreducible exactly when the graph is connected, and for a connected graph it is aperiodic exactly when the graph is not bipartite. The uniqueness follows at once from the convergence: if $\pi'$ is any stationary distribution and we start the walk with $p_0 = \pi'$, then $p_t = \pi'$ for all $t$, and $p_t \to \pi$ forces $\pi' = \pi$. The triangle is connected and not bipartite (it is an odd cycle), and Lemma 21 shows the convergence there explicitly. The two graphs of Figure 11 show that neither condition can be dropped.

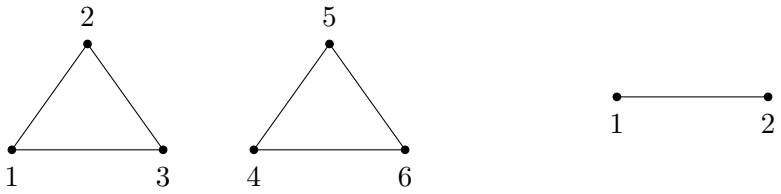



Figure 11: Left: two disjoint triangles (not connected). Right: a single edge (bipartite).

**Lemma 28 ▶ Both Conditions Are Necessary**

Neither condition of Theorem 27 can be dropped.

1. (Not connected.) Let $G$ be two disjoint triangles, on the vertices $1, 2, 3$ and $4, 5, 6$. This graph is not bipartite. The walk converges from every starting vertex, but the limit depends on the start, and the stationary distribution is not unique.

2. (Bipartite.) Let $G$ be a single edge on the vertices $1, 2$. This graph is connected. The walk started at vertex 1 does not converge, although $\pi = (1/2, 1/2)$ is stationary.

*Proof. Part 1.* The graph contains a triangle, which is an odd cycle, so it is not bipartite. There is no edge between $\{1, 2, 3\}$ and $\{4, 5, 6\}$, so $P_{uv} = 0$ whenever $u$ and $v$ are in different triangles. Start the token at vertex 1. Then $p_t^4 = p_t^5 = p_t^6 = 0$ for all $t$, by induction on $t$: it holds for $t = 0$, and if it holds for $t$ then for $v \in \{4, 5, 6\}$ every term of $p_{t+1}^v = \sum_u p_t^u P_{uv}$ is 0, because $p_t^u = 0$ for $u \in \{4, 5, 6\}$ and $P_{uv} = 0$ for $u \in \{1, 2, 3\}$. So the token stays in the first triangle. Every vertex of that triangle has degree 2 in $G$, so the rows and columns $1, 2, 3$ of $P$ form exactly the

transition matrix of the triangle, and the walk is the random walk on a triangle started at vertex 1. By Lemma 21,

$$p_t \longrightarrow (1/3, 1/3, 1/3, 0, 0, 0) =: \pi^{(1)}.$$

By the same argument the walk started at vertex 4 converges to $\pi^{(2)} = (0, 0, 0, 1/3, 1/3, 1/3)$, and by symmetry the walk from any other vertex converges to the uniform distribution on its own triangle. Since $\pi^{(1)} \neq \pi^{(2)}$, there is no single limit that is reached from every start. What fails here is the independence of the start, not the convergence.

Both limits are stationary: $\pi^{(1)}$ is 0 on the second triangle and uniform on the first, and the uniform distribution is stationary for the triangle, so $\pi^{(1)}P = \pi^{(1)}$; similarly for $\pi^{(2)}$. So the stationary distribution is not unique.

*Part 2.* A single edge is connected, and it is bipartite with the two sides $\{1\}$ and $\{2\}$. Both degrees are 1, so

$$P = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad (a, b)P = (b, a).$$

Starting from $p_0 = (1, 0)$ we get $p_t = (1, 0)$ for even $t$ and $p_t = (0, 1)$ for odd $t$, by induction on $t$. The first entry is the sequence $1, 0, 1, 0, \ldots$, which has no limit, so $p_t$ does not converge. On the other hand $(1/2, 1/2)P = (1/2, 1/2)$, so $\pi = (1/2, 1/2)$ is stationary (it is $\deg(v)/2m$ with $m = 1$): a stationary distribution exists, but the walk started at a vertex does not approach it. □

The second example shows what goes wrong in every bipartite graph: if all edges go between two sides $L$ and $R$, then a walk started at a vertex of $L$ is in $L$ at all even times and in $R$ at all odd times, so the probability $\sum_{v \in L} p_t^v$ of the side $L$ alternates between 1 and 0.

**Lecture Date:** September 15, 2026

**Lecture Topic:** Qubits and Dirac Notation, Unitary Matrices, Quantum Gates and Circuits, Tensor Products, Entanglement

In this lecture, we introduce the language of quantum computing: qubits and Dirac (bra-ket) notation, unitary matrices, and quantum circuits. We meet the gates $X$, $H$ and CNOT, define the tensor product that combines several qubits into one state, and show that some two-qubit states are entangled.

## 26 Qubits

A classical *bit* takes one of the two values 0 and 1. A quantum bit can be in a combination of both.

**Definition 8 ▶ Qubit, $n$-Qubit State**

The state of one **qubit** is a unit vector in $\mathbb{C}^2$,

$$\begin{pmatrix}\alpha\\ \beta\end{pmatrix} = \alpha\begin{pmatrix}1\\ 0\end{pmatrix} + \beta\begin{pmatrix}0\\ 1\end{pmatrix} = \alpha|0\rangle + \beta|1\rangle, \qquad |\alpha|^2 + |\beta|^2 = 1,$$

where $|0\rangle = \begin{pmatrix}1\\ 0\end{pmatrix}$ and $|1\rangle = \begin{pmatrix}0\\ 1\end{pmatrix}$ form the **computational basis**. The complex numbers $\alpha, \beta$ are the **amplitudes**. More generally, the state of $n$ qubits is a unit vector in $\mathbb{C}^{2^n}$,

$$|\phi\rangle = \sum_{x\in\{0,1\}^n} \alpha_x|x\rangle, \qquad \sum_{x\in\{0,1\}^n} |\alpha_x|^2 = 1,$$

where $|x\rangle$ is the standard basis vector with a 1 in the position indexed by the bit string $x$ (the $2^n$ strings are listed in increasing binary order) and 0 elsewhere. Measuring the state gives the outcome $x$ with probability $|\alpha_x|^2$.

For one qubit, a measurement gives 0 with probability $|\alpha|^2$ and 1 with probability $|\beta|^2$; the normalization says exactly that these probabilities sum to 1. In short, "amplitude = $\sqrt{\text{probability}}$": the modulus of an amplitude is the square root of the corresponding probability.

**Remark.** On the board the normalization was written as $\sqrt{\alpha^2 + \beta^2} = 1$. Since the amplitudes are complex, the squares must be squared moduli, as in Definition 8.

**Two qubits.** The state of two qubits has four amplitudes, one for each of $00, 01, 10, 11$:

$$\begin{pmatrix}\alpha\\ \beta\\ \gamma\\ \delta\end{pmatrix} = \alpha\begin{pmatrix}1\\ 0\\ 0\\ 0\end{pmatrix} + \beta\begin{pmatrix}0\\ 1\\ 0\\ 0\end{pmatrix} + \gamma\begin{pmatrix}0\\ 0\\ 1\\ 0\end{pmatrix} + \delta\begin{pmatrix}0\\ 0\\ 0\\ 1\end{pmatrix} = \alpha|00\rangle + \beta|01\rangle + \gamma|10\rangle + \delta|11\rangle,$$

with $|\alpha|^2 + |\beta|^2 + |\gamma|^2 + |\delta|^2 = 1$.

**Exercise 9.** What is the third entry of the column vector $|00\rangle$?

**Solution**. It is 0. The four entries of a two-qubit column vector are the amplitudes of $|00\rangle, |01\rangle, |10\rangle, |11\rangle$, in this order, so the third entry is the amplitude of $|10\rangle$. The state $|00\rangle = 1 \cdot |00\rangle + 0 \cdot |01\rangle + 0 \cdot |10\rangle + 0 \cdot |11\rangle$ has amplitude 0 on $|10\rangle$, and $|00\rangle = (1, 0, 0, 0)^T$. The expansion above confirms this: comparing third entries for $\alpha = 1$, $\beta = \gamma = \delta = 0$ gives 0 on the left-hand side, so the first basis column has third entry 0. □

Writing a column vector as a combination of the kets $|x\rangle$ is *Dirac notation*, after Paul Dirac. Its advantage shows as soon as there are more qubits: the state of four qubits is a column vector with $2^4 = 16$ entries, but a state such as

$$\frac{1}{\sqrt{2}}|0000\rangle + \frac{1}{\sqrt{2}}|1111\rangle$$

is written with two terms. As a column vector it has $1/\sqrt{2}$ in the first and in the last position and 0 in the other fourteen (the board sketched these two entries as 1).

## 27 Kets, Bras and the Dagger

A *ket* $|\phi\rangle$ is a column vector. The corresponding *bra* is the row vector

$$\langle\phi| = (|\phi\rangle)^\dagger,$$

where † ("dagger") is the *conjugate transpose*: transpose the vector or matrix and replace every entry by its complex conjugate. For $|\phi\rangle = \begin{pmatrix}\alpha\\ \beta\end{pmatrix}$ we get $\langle\phi| = \begin{pmatrix}\overline{\alpha} & \overline{\beta}\end{pmatrix}$.

A bra times a ket is a number, the *inner product*: for $|x\rangle, |y\rangle \in \mathbb{C}^n$ with entries $x_i$, $y_i$,

$$\langle x|y\rangle = \langle x||y\rangle = \sum_{i=1}^{n} \overline{x_i}\, y_i.$$

With this convention the inner product is linear in the second argument and conjugate-linear in the first, and $\langle y|x\rangle = \overline{\langle x|y\rangle}$. The (Euclidean) norm of a vector is given by

$$\||y\rangle\|^2 = \langle y|y\rangle = \sum_{i=1}^{n} |y_i|^2.$$

We will use two rules for the dagger: $(A^\dagger)^\dagger = A$, and $(AB)^\dagger = B^\dagger A^\dagger$ for matrices $A$, $B$ whose product is defined (the same rule as for the transpose, and conjugation is entry by entry). In particular, for a matrix $U$ and a ket $|x\rangle$,

$$(U|x\rangle)^\dagger = \langle x|U^\dagger.$$

**Remark.** The notes annotate the dagger as "transpose inverse". The dagger is the conjugate transpose; it equals the inverse only for unitary matrices, which are defined next.

## 28 Unitary Matrices

**Definition 9 ▶ Unitary Matrix**

An $n \times n$ complex matrix $U$ is **unitary** if $U^\dagger U = I$.

**Exercise 10.** Show that $\|U|x\rangle\|_2 = \||x\rangle\|_2$ for all $|x\rangle \in \mathbb{C}^n$ if and only if $U$ is a unitary matrix ($U^\dagger U = I$).

The solution is Theorem 29; we prove both directions.

**Theorem 29 ▶ Norm Preserving $\iff$ Unitary**

Let $U$ be an $n \times n$ complex matrix. Then $\|U|x\rangle\| = \||x\rangle\|$ for all $|x\rangle \in \mathbb{C}^n$ if and only if $U^\dagger U = I$.

*Proof. Unitary implies norm preserving.* Assume $U^\dagger U = I$, let $|x\rangle \in \mathbb{C}^n$ and let $|y\rangle = U|x\rangle$. Then

$$\||y\rangle\|^2 = \langle y|y\rangle = (U|x\rangle)^\dagger U|x\rangle = \langle x|U^\dagger U|x\rangle = \langle x|I|x\rangle = \langle x|x\rangle = \||x\rangle\|^2.$$

Norms are non-negative, so taking square roots gives $\|U|x\rangle\| = \||x\rangle\|$.

*Norm preserving implies unitary.* Assume $\|U|x\rangle\| = \||x\rangle\|$ for all $|x\rangle \in \mathbb{C}^n$, and let

$$M = U^\dagger U - I.$$

We have to show $M = 0$. We use two properties of $M$.

1. $M$ is Hermitian, i.e., $M^\dagger = M$: $M^\dagger = (U^\dagger U)^\dagger - I^\dagger = U^\dagger (U^\dagger)^\dagger - I = U^\dagger U - I = M$.
2. For every $|x\rangle$, by the computation in the first part,

$$\langle x|M|x\rangle = \langle x|U^\dagger U|x\rangle - \langle x|x\rangle = \|U|x\rangle\|^2 - \||x\rangle\|^2 = 0.$$

Now fix $|x\rangle, |y\rangle \in \mathbb{C}^n$ and let $z = \langle x|M|y\rangle$, a complex number. A number is its own transpose, so its conjugate equals its dagger, and by property 1

$$\overline{z} = (\langle x|M|y\rangle)^\dagger = \langle y|M^\dagger|x\rangle = \langle y|M|x\rangle.$$

Apply property 2 to the vector $|x\rangle + |y\rangle$, whose bra is $\langle x| + \langle y|$, and expand:

$$\begin{aligned} 0 &= (\langle x| + \langle y|)M(|x\rangle + |y\rangle) = \langle x|M|x\rangle + \langle x|M|y\rangle + \langle y|M|x\rangle + \langle y|M|y\rangle \\ &= 0 + z + \overline{z} + 0 = 2\operatorname{Re} z, \end{aligned}$$

where the first and last terms vanish by property 2. Next apply property 2 to the vector $|x\rangle + i|y\rangle$. Because the dagger conjugates, its bra is $\langle x| - i\langle y|$, and

$$\begin{aligned} 0 &= (\langle x| - i\langle y|)M(|x\rangle + i|y\rangle) = \langle x|M|x\rangle + i\langle x|M|y\rangle - i\langle y|M|x\rangle + (-i)(i)\langle y|M|y\rangle \\ &= 0 + iz - i\overline{z} + 0 = i(z - \overline{z}) = i \cdot 2i \operatorname{Im} z = -2 \operatorname{Im} z. \end{aligned}$$

Hence $\operatorname{Re} z = 0$ and $\operatorname{Im} z = 0$, i.e., $\langle x|M|y\rangle = 0$ for all $|x\rangle, |y\rangle \in \mathbb{C}^n$. Finally take $|x\rangle = e_j$ and $|y\rangle = e_k$, the $j$th and $k$th standard basis vectors. Then $Me_k$ is the $k$th column of $M$ and $e_j^\dagger$ picks out its $j$th entry, so $0 = e_j^\dagger M e_k = M_{jk}$. This holds for all $j, k$, so $M = 0$, i.e., $U^\dagger U = I$. □

**Remark.** For a square matrix, $U^\dagger U = I$ also gives $UU^\dagger = I$. Indeed, if $U|x\rangle = 0$ then $|x\rangle = U^\dagger U|x\rangle = 0$, so $U$ is injective, and an injective square matrix is invertible. Multiplying $U^\dagger U = I$ by $U^{-1}$ on the right gives $U^\dagger = U^{-1}$, hence $UU^\dagger = UU^{-1} = I$. So "unitary" is unambiguous, every unitary matrix is invertible, and its inverse $U^\dagger$ is again unitary.

Theorem 29 is the reason quantum gates are unitary matrices. A gate is a linear map that has to send every state, i.e., every unit vector, to a state. A linear map that preserves the norm of unit vectors preserves the norm of all vectors (scale the vector to norm 1), so by the theorem it is unitary; conversely, every unitary matrix maps states to states.

## 29 Quantum Circuits and One-Qubit Gates

A *quantum circuit* is drawn with one horizontal wire per qubit; time runs from left to right, and a box on a wire is a gate, i.e., a unitary matrix applied to that qubit (Figure 12).

**Columns of a matrix.** We recall one fact from linear algebra. Let $A$ be an $m \times n$ matrix with columns $a_1, \dots, a_n$, and let $x = (x_1, \dots, x_n)^T$. Then

$$y = Ax = x_1 a_1 + x_2 a_2 + \cdots + x_n a_n$$

is a linear combination of the columns of $A$ (equivalently, the $i$th entry of $y$ is the $i$th row of $A$ times $x$). In particular $Ae_j = a_j$: *the $j$th column of a gate's matrix is the image of the $j$th basis state.* So a gate is determined by what it does to the basis states, and we can read off its matrix from them.

**The $X$ gate.** The *Pauli $X$ gate* is the quantum NOT gate. It swaps the two basis states, $X|0\rangle = |1\rangle$ and $X|1\rangle = |0\rangle$, so its columns are $|1\rangle$ and $|0\rangle$:

$$X = \sigma_x = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}\begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

By linearity it swaps the amplitudes of a general state:

$$X(\alpha|0\rangle + \beta|1\rangle) = \alpha|1\rangle + \beta|0\rangle.$$

$X$ is unitary: it is real and symmetric, so $X^\dagger = X$, and $X^\dagger X = X^2 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = I$.

**Definition 10 ▶ Hadamard Gate**

The **Hadamard gate** $H$ is the one-qubit gate with

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle), \qquad H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle), \qquad H = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}. \tag{3}$$

The matrix agrees with the action on the basis states, since its columns are their images:

$$H\begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \end{pmatrix}, \qquad H\begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 1/\sqrt{2} \\ -1/\sqrt{2} \end{pmatrix}.$$

The Hadamard gate turns a basis state into an equal superposition: measuring $H|0\rangle$ or $H|1\rangle$ gives 0 and 1 with probability $1/2$ each. The two states differ only in the sign of the amplitude of $|1\rangle$, which a measurement in the computational basis does not see, but a second Hadamard gate does.

**Lemma 30 ▶ Hadamard Twice Is the Identity**

$H^2 = I$. Consequently $H$ is unitary, and

$$H\,\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = |0\rangle, \qquad H\,\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = |1\rangle.$$

*Proof.* By matrix multiplication,

$$H^2 = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \cdot \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} = \frac{1}{2}\begin{pmatrix} 1+1 & 1-1 \\ 1-1 & 1+1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I.$$

$H$ is real and symmetric, so $H^\dagger = H$ and $H^\dagger H = H^2 = I$, i.e., $H$ is unitary. Finally, by Equation (3), $H\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = HH|0\rangle = |0\rangle$ and $H\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = HH|1\rangle = |1\rangle$. □

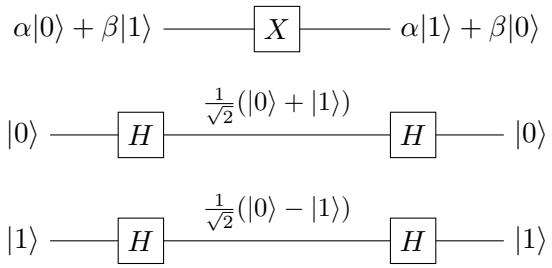



Figure 12: One-qubit circuits. Top: the $X$ gate. Below: the Hadamard gate applied twice gives back the input, since $H^2 = I$ (Lemma 30).

## 30 The CNOT Gate

**Definition 11 ▶ CNOT Gate**

The **controlled-NOT gate** (CNOT) is the two-qubit gate that maps the basis states by

$$|a\rangle|b\rangle \longmapsto |a\rangle|b \oplus a\rangle, \qquad a, b \in \{0, 1\},$$

where $\oplus$ is XOR (addition modulo 2). The first qubit $|a\rangle$ is the **control** qubit and is left unchanged; the second qubit $|b\rangle$ is the **target** qubit and is flipped exactly when $a = 1$. See Figure 13.

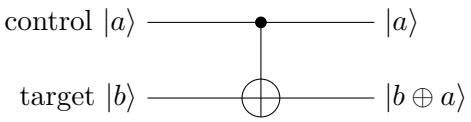



Figure 13: The CNOT gate: a filled dot on the control qubit and $\oplus$ on the target qubit.

The four basis states are mapped as follows (for example $a = 1$, $b = 1$ gives $b \oplus a = 0$):

| $\lvert ab\rangle$ | $\lvert 00\rangle$ | $\lvert 01\rangle$ | $\lvert 10\rangle$ | $\lvert 11\rangle$ |
|---|---|---|---|---|
| $\lvert a, b \oplus a\rangle$ | $\lvert 00\rangle$ | $\lvert 01\rangle$ | $\lvert 11\rangle$ | $\lvert 10\rangle$ |

**Lemma 31 ▶ Matrix of the CNOT Gate**

The matrix of the CNOT gate, with rows and columns indexed by $00, 01, 10, 11$, is the permutation matrix

$$U_{\text{CNOT}} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

It is unitary.

*Proof.* The columns of the matrix of a gate are the images of the basis states. By the table, the

images of $|00\rangle, |01\rangle, |10\rangle, |11\rangle$ are $|00\rangle, |01\rangle, |11\rangle, |10\rangle$, i.e., the column vectors

$$\begin{pmatrix}1\\0\\0\\0\end{pmatrix}, \quad \begin{pmatrix}0\\1\\0\\0\end{pmatrix}, \quad \begin{pmatrix}0\\0\\0\\1\end{pmatrix}, \quad \begin{pmatrix}0\\0\\1\\0\end{pmatrix},$$

which are the four columns of $U_{\text{CNOT}}$, in this order.

The map $\pi\colon (a,b) \mapsto (a, b \oplus a)$ on $\{0,1\}^2$ is its own inverse, since $\pi(\pi(a,b)) = (a, b \oplus a \oplus a) = (a,b)$. Hence $\pi$ is a bijection, i.e., a permutation of the four basis states, and $U_{\text{CNOT}}$ is a *permutation matrix*: every column is a standard basis vector, and different columns are different basis vectors.

Every permutation matrix $P$ is unitary. The $(j,k)$ entry of $P^T P$ is the inner product of the $j$th and the $k$th column of $P$. These columns are standard basis vectors, equal if $j = k$ and different if $j \neq k$, so the entry is 1 if $j = k$ and 0 otherwise. Thus $P^T P = I$, and since $P$ is real, $P^\dagger P = P^T P = I$. □

In block form, $U_{\text{CNOT}} = \begin{pmatrix} I & 0 \\ 0 & X \end{pmatrix}$: if the control is 0 (first two rows and columns) nothing happens, and if it is 1 the gate $X$ is applied to the target.

### 30.1 Classical Gates Are Not Reversible

By Theorem 29 and the remark after it, every quantum gate is unitary and therefore invertible: the computation can be undone by applying $U^\dagger$. Classical gates such as AND, OR and XOR, which take two bits to one bit, are *not bijective*, hence *not reversible*, hence *not unitary*. (Some classical gates, such as NOT, are reversible; the statement is about gates that lose information.)

**Lemma 32 ▶ XOR Is Not Reversible**

The classical gate $\text{XOR}\colon \{0,1\}^2 \to \{0,1\}$, $(a,b) \mapsto a \oplus b$, is not injective, so its input cannot be recovered from its output. The linear map $T\colon \mathbb{C}^4 \to \mathbb{C}^2$ with $T|ab\rangle = |a \oplus b\rangle$ does not preserve norms, so it is not unitary. The same holds for every gate with two input bits and one output bit.

*Proof.* $\text{XOR}(0,0) = 0 = \text{XOR}(1,1)$, so two different inputs have the same output, and no function of the output can return the input in both cases. For the linear map, $T(|00\rangle - |11\rangle) = |0\rangle - |0\rangle = 0$, although $\||00\rangle - |11\rangle\| = \sqrt{2} \neq 0$. For a general gate $g\colon \{0,1\}^2 \to \{0,1\}$ there are four inputs and only two outputs, so by the pigeonhole principle two inputs $u \neq v$ have $g(u) = g(v)$, and the same argument applies with $|u\rangle - |v\rangle$. (For AND, take $u = 00$ and $v = 01$; for OR, $u = 01$ and $v = 11$.) □

The CNOT gate is the reversible version of XOR: it computes $b \oplus a$ on the target wire, but it also keeps a copy of $a$ on the control wire, and from $(a, b \oplus a)$ we recover $b = (b \oplus a) \oplus a$. The example written in red in the notes and drawn on the board is the collision of XOR: the inputs 00 and 11 give the same output 0. In contrast, CNOT maps $|00\rangle$ to $|00\rangle$ and $|11\rangle$ to $|10\rangle$, two different outputs.

## 31 Tensor Products and Entanglement

The state of two qubits is built from the states of the single qubits by the tensor product.

**Definition 12 ▶ Tensor Product**

Let $A = (a_{ij})$ be an $m \times n$ matrix and let $B$ be a $p \times q$ matrix. The **tensor product** (Kronecker product) $A \otimes B$ is the $mp \times nq$ block matrix

$$A \otimes B = \begin{pmatrix} a_{11}B & a_{12}B & \cdots & a_{1n}B \\ a_{21}B & a_{22}B & \cdots & a_{2n}B \\ \vdots & & & \vdots \\ a_{m1}B & a_{m2}B & \cdots & a_{mn}B \end{pmatrix}.$$

Column vectors are matrices with one column, so in particular

$$\begin{pmatrix} \alpha \\ \beta \end{pmatrix} \otimes \begin{pmatrix} \gamma \\ \delta \end{pmatrix} = \begin{pmatrix} \alpha\gamma \\ \alpha\delta \\ \beta\gamma \\ \beta\delta \end{pmatrix}.$$

For basis states this gives, for example, $|0\rangle \otimes |0\rangle = (1,0,0,0)^T = |00\rangle$ and $|1\rangle \otimes |0\rangle = (0,0,1,0)^T = |10\rangle$, and in general

$$|a\rangle|b\rangle = |a\rangle \otimes |b\rangle = |ab\rangle, \qquad a, b \in \{0,1\};$$

the three ways of writing are used interchangeably. In terms of kets, the formula for two vectors reads

$$(\alpha|0\rangle + \beta|1\rangle) \otimes (\gamma|0\rangle + \delta|1\rangle) = \alpha\gamma|00\rangle + \alpha\delta|01\rangle + \beta\gamma|10\rangle + \beta\delta|11\rangle,$$

i.e., the tensor product is multiplied out like a product of sums. Gates combine in the same way: if $A$ is applied to the first qubit and $B$ to the second, the two-qubit gate is $A \otimes B$, and

$$(A \otimes B)(|x\rangle \otimes |y\rangle) = A|x\rangle \otimes B|y\rangle. \tag{4}$$

Indeed, if $x_1, \ldots, x_n$ are the entries of $|x\rangle$, then $|x\rangle \otimes |y\rangle$ consists of the blocks $x_1|y\rangle, \ldots, x_n|y\rangle$, so the $i$th block of the left-hand side is $\sum_j a_{ij}B(x_j|y\rangle) = \left(\sum_j a_{ij}x_j\right)B|y\rangle$, which is the $i$th block of the right-hand side.

**Definition 13 ▶ Product State, Entangled State**

A two-qubit state $|\psi\rangle$ is a **product state** if $|\psi\rangle = |\phi_1\rangle \otimes |\phi_2\rangle$ for some one-qubit states $|\phi_1\rangle$, $|\phi_2\rangle$. Otherwise $|\psi\rangle$ is **entangled**.

In a product state each qubit has a state of its own. This is not always the case: the two qubits of the *Bell state* $\frac{1}{\sqrt{2}}|00\rangle + \frac{1}{\sqrt{2}}|11\rangle$ are entangled.

**Exercise 11.** Show that $\frac{1}{\sqrt{2}}|00\rangle + \frac{1}{\sqrt{2}}|11\rangle \neq |\alpha\rangle|\beta\rangle$ for all one-qubit states $|\alpha\rangle$, $|\beta\rangle$.

The solution is Theorem 33.

**Theorem 33 ▶ The Bell State Is Entangled**

The state $\frac{1}{\sqrt{2}}|00\rangle + \frac{1}{\sqrt{2}}|11\rangle$ is not a product state: there are no vectors $|\phi_1\rangle, |\phi_2\rangle \in \mathbb{C}^2$ with

$$\frac{1}{\sqrt{2}}|00\rangle + \frac{1}{\sqrt{2}}|11\rangle = |\phi_1\rangle \otimes |\phi_2\rangle.$$

*Proof.* Suppose there are, and write $|\phi_1\rangle = \begin{pmatrix}\alpha\\ \beta\end{pmatrix}$ and $|\phi_2\rangle = \begin{pmatrix}\gamma\\ \delta\end{pmatrix}$. By Definition 12, comparing the four entries of the two sides gives

$$\alpha\gamma = \frac{1}{\sqrt{2}}, \qquad \alpha\delta = 0, \qquad \beta\gamma = 0, \qquad \beta\delta = \frac{1}{\sqrt{2}}.$$

From $\alpha\delta = 0$ we get $\alpha = 0$ or $\delta = 0$. If $\alpha = 0$ then $\alpha\gamma = 0 \neq 1/\sqrt{2}$, and if $\delta = 0$ then $\beta\delta = 0 \neq 1/\sqrt{2}$. Both cases are contradictions. □

**Lecture Date:** September 17, 2026

**Lecture Topic:** Linear Algebra Review: Column Space, Rank, Linear Independence, Orthogonality

In this lecture, we review the linear algebra that the quantum part of the course rests on. We read the product $Ax$ as a combination of the columns of $A$, define linear independence, the column rank and the row rank of a matrix, and show that a nonzero vector orthogonal to a set of independent vectors is independent of them.

## 32 The Product $Ax$ as a Combination of Columns

In the previous lecture we saw that $y = Ax$ is a linear combination of the columns of $A$, with the entries of $x$ as coefficients. We work this out on the example

$$A = \begin{bmatrix} 2 & 1 & 3 \\ 3 & 1 & 4 \\ 5 & 7 & 12 \end{bmatrix} \in \mathbb{R}^{3\times 3}, \qquad x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix},$$

whose columns we call $v_1 = (2,3,5)^T$, $v_2 = (1,1,7)^T$ and $v_3 = (3,4,12)^T$. Then

$$Ax = \begin{bmatrix} 2x_1 + x_2 + 3x_3 \\ 3x_1 + x_2 + 4x_3 \\ 5x_1 + 7x_2 + 12x_3 \end{bmatrix} = x_1 \begin{bmatrix} 2 \\ 3 \\ 5 \end{bmatrix} + x_2 \begin{bmatrix} 1 \\ 1 \\ 7 \end{bmatrix} + x_3 \begin{bmatrix} 3 \\ 4 \\ 12 \end{bmatrix} = x_1 v_1 + x_2 v_2 + x_3 v_3. \tag{5}$$

In this example the third column is the sum of the first two, $v_3 = v_1 + v_2$, because $2 + 1 = 3$, $3 + 1 = 4$ and $5 + 7 = 12$. Substituting this into Equation (5) gives

$$Ax = x_1 v_1 + x_2 v_2 + x_3(v_1 + v_2) = (x_1 + x_3)\, v_1 + (x_2 + x_3)\, v_2, \tag{6}$$

so every output $Ax$ is already a combination of $v_1$ and $v_2$ alone.

The matrix $A$ defines a linear map $T\colon \mathbb{R}^3 \to \mathbb{R}^3$, $T(x) = Ax$. In general, an $m \times n$ matrix $A$ with columns $a_1, \dots, a_n$ defines the linear map $T\colon \mathbb{R}^n \to \mathbb{R}^m$, $T(x) = Ax = x_1 a_1 + \cdots + x_n a_n$, and its *image* is the set of all outputs,

$$\mathrm{Im}(A) = \{Ax \mid x \in \mathbb{R}^n\} = \mathrm{span}(a_1, \dots, a_n).$$

The second equality is exactly the statement that $Ax$ is a combination of the columns: the image of $A$ is its *column space*.

## 33 Linear Independence and Rank

**Definition 14 ▶ Linear Independence**

Vectors $v_1, \dots, v_k$ are **linearly independent** if

$$c_1 v_1 + c_2 v_2 + \cdots + c_k v_k = 0 \quad \Longrightarrow \quad c_1 = c_2 = \cdots = c_k = 0,$$

i.e., the only combination of them that gives the zero vector is the trivial one. Otherwise they are **linearly dependent**. If $v_1, \dots, v_k$ are linearly independent, we say that a vector $v$ is *independent of* them if $v, v_1, \dots, v_k$ are linearly independent.

**Definition 15 ▶ Column Rank and Row Rank**

The **column rank** $\operatorname{rank}_c(A)$ of a matrix $A$ is the maximum number of linearly independent columns of $A$. The **row rank** $\operatorname{rank}_r(A)$ is the maximum number of linearly independent rows of $A$.

We use one fact about a subspace $W$ spanned by $r$ vectors $z_1, \dots, z_r$: any $r+1$ vectors $y_1, \dots, y_{r+1}$ of $W$ are linearly dependent. Indeed, write $y_j = \sum_{l=1}^{r} t_{lj} z_l$. The homogeneous system $\sum_{j=1}^{r+1} t_{lj} c_j = 0$ $(1 \leqslant l \leqslant r)$ has $r$ equations in $r+1$ unknowns, so it has a solution $(c_1, \dots, c_{r+1}) \neq 0$ (see below), and then $\sum_j c_j y_j = \sum_l \big(\sum_j t_{lj} c_j\big) z_l = 0$. Hence two bases of $W$ cannot have different sizes (the larger one would consist of more vectors than span $W$, and so be dependent); the common size is the *dimension* $\dim(W)$.

That a homogeneous system of $r$ linear equations in more than $r$ unknowns has a nonzero solution follows by induction on $r$. For $r = 0$ there are no equations and any nonzero vector is a solution. For $r \geqslant 1$, if no equation contains $c_1$, set $c_1 = 1$ and all other unknowns to 0. Otherwise solve one equation that contains $c_1$ for $c_1$ and substitute into the others: this leaves $r-1$ equations in the remaining unknowns, of which there are more than $r-1$, so by induction they have a nonzero solution, and the solved equation then gives the value of $c_1$. The column rank is the dimension of the image:

**Lemma 34 ▶ Column Rank Is the Dimension of the Image**

Let $A$ be an $m \times n$ matrix with $r = \operatorname{rank}_c(A) \geqslant 1$. Then any $r$ linearly independent columns of $A$ form a basis of $\operatorname{Im}(A)$. In particular,

$$\operatorname{rank}_c(A) = \dim(\operatorname{Im}(A)).$$

*Proof.* Let $w_1, \dots, w_r$ be independent columns of $A$ and let $a$ be any column of $A$. By the definition of $r$, the vectors $a, w_1, \dots, w_r$ are dependent (trivially so if $a$ is one of the $w_l$): $\beta a + \beta_1 w_1 + \cdots + \beta_r w_r = 0$ with coefficients that are not all zero. Here $\beta \neq 0$, since otherwise this would be a nontrivial relation among $w_1, \dots, w_r$. Hence $a = -\frac{1}{\beta}(\beta_1 w_1 + \cdots + \beta_r w_r) \in \operatorname{span}(w_1, \dots, w_r)$. So every column of $A$, and therefore every combination of columns, lies in $\operatorname{span}(w_1, \dots, w_r)$: the independent vectors $w_1, \dots, w_r$ span $\operatorname{Im}(A)$ and form a basis of it. □

**Exercise 12.** For the matrix $A$ of Equation (5), find $\operatorname{rank}_c(A)$, $\operatorname{Im}(A)$ and $\operatorname{rank}_r(A)$, and express every row of $A$ as a combination of [1 0 1] and [0 1 1].

**Solution**. *Columns.* The columns $v_1, v_2$ are independent: $av_1 + bv_2 = 0$ reads $2a + b = 0$, $3a + b = 0$, $5a + 7b = 0$; subtracting the first equation from the second gives $a = 0$, and then $b = 0$. All three columns are dependent, since $v_1 + v_2 - v_3 = 0$. Hence $\operatorname{rank}_c(A) = 2$, and by Lemma 34,

$$\operatorname{Im}(A) = \operatorname{span}(v_1, v_2), \qquad \dim(\operatorname{Im}(A)) = 2.$$

*Rows.* Let $u_1 = [1\ 0\ 1]$ and $u_2 = [0\ 1\ 1]$, so that $au_1 + bu_2 = [a \quad b \quad a+b]$. Then

$$[2\ 1\ 3] = 2u_1 + 1u_2, \qquad [3\ 1\ 4] = 3u_1 + 1u_2, \qquad [5\ 7\ 12] = 5u_1 + 7u_2.$$

So all three rows lie in $\operatorname{span}(u_1, u_2)$, and any three vectors in the span of two vectors are dependent: $\operatorname{rank}_r(A) \leqslant 2$. The first two rows are independent: $a\,[2\ 1\ 3] + b\,[3\ 1\ 4] = 0$ gives $a + b = 0$ from the second entry and $2a + 3b = 0$ from the first, so $b = 0$ and $a = 0$. Hence $\operatorname{rank}_r(A) = 2$. □

In the example the two ranks agree. This is not a coincidence.

**Exercise 13.** Show that $\operatorname{rank}_r(A) = \operatorname{rank}_c(A)$ for every matrix $A$.

The solution is Theorem 35.

**Theorem 35 ▶ Row Rank Equals Column Rank**

For every $m \times n$ matrix $A$, $\operatorname{rank}_r(A) = \operatorname{rank}_c(A)$. This common value is called the **rank** of $A$.

*Proof.* We first show $\operatorname{rank}_r(A) \leqslant \operatorname{rank}_c(A)$. Let $r = \operatorname{rank}_c(A)$. If $r = 0$, then every column of $A$ is zero (a single nonzero vector is independent), so $A = 0$ and $\operatorname{rank}_r(A) = 0$ as well. Let $r \geqslant 1$, and let $w_1, \dots, w_r$ be independent columns of $A$; write $w_{il}$ for the $i$th entry of $w_l$, so that $W = (w_{il})$ is the $m \times r$ matrix with columns $w_1, \dots, w_r$. By Lemma 34, every column $a_j$ of $A$ is a combination of them: there are numbers $s_{lj}$ with

$$a_j = \sum_{l=1}^{r} s_{lj}\, w_l \qquad (1 \leqslant j \leqslant n).$$

Let $S = (s_{lj})$, an $r \times n$ matrix with rows $\rho_1, \dots, \rho_r$. Taking the $i$th entry on both sides, the entries of $A$ are $a_{ij} = \sum_{l=1}^{r} w_{il}\, s_{lj}$, that is, $A = WS$. Now fix $i$ and let $j$ run from 1 to $n$: the same equation says that

$$\text{row } i \text{ of } A \;=\; \sum_{l=1}^{r} w_{il}\, \rho_l.$$

So every row of $A$ lies in $\operatorname{span}(\rho_1, \dots, \rho_r)$. Any $r+1$ vectors in the span of $r$ vectors are dependent, so $A$ has at most $r$ independent rows: $\operatorname{rank}_r(A) \leqslant r = \operatorname{rank}_c(A)$.

This holds for every matrix, in particular for the transpose $A^T$, whose rows are the columns of $A$ and whose columns are the rows of $A$. Hence

$$\operatorname{rank}_c(A) = \operatorname{rank}_r(A^T) \leqslant \operatorname{rank}_c(A^T) = \operatorname{rank}_r(A),$$

and the two inequalities together give $\operatorname{rank}_r(A) = \operatorname{rank}_c(A)$. □

For the example, with $w_1 = v_1$ and $w_2 = v_2$, the rows of $S$ are exactly the two row vectors [1 0 1] and [0 1 1] from above.

## 34 Orthogonality

The results of this section hold over $\mathbb{R}$ and over $\mathbb{C}$. For $a, b \in \mathbb{R}^n$ the inner product is $\langle a, b\rangle = a \cdot b = \sum_{i=1}^{n} a_i b_i$. For $a, b \in \mathbb{C}^n$ it is $\langle a, b\rangle = \sum_{i=1}^{n} \overline{a_i}\, b_i$, which is linear in $b$. In both cases $\langle a, a\rangle = \sum_{i=1}^{n} |a_i|^2 = \|a\|^2$, which is 0 only for $a = 0$.

**Definition 16 ▶ Orthogonality**

Two vectors $a$ and $b$ are **orthogonal** if $\langle a, b\rangle = 0$; for real vectors, $a \cdot b = 0$.

**Exercise 14.** Let $v_1, \dots, v_k$ be linearly independent vectors. Show that if a nonzero vector $v$ is orthogonal to $v_1, v_2, \dots, v_k$, then $v$ is independent of $v_1, v_2, \dots, v_k$.

The solution is Theorem 36; the other reading of the line on the board is Corollary 37.

**Theorem 36 ▶ Orthogonal Implies Independent**

Let $v_1, \dots, v_k$ be linearly independent vectors in $\mathbb{R}^n$ or $\mathbb{C}^n$, and let $v \neq 0$ be orthogonal to each of them. Then $v, v_1, \dots, v_k$ are linearly independent. In particular, $v \notin \operatorname{span}(v_1, \dots, v_k)$.

*Proof.* Suppose that $c\,v + c_1 v_1 + \cdots + c_k v_k = 0$. Taking the inner product of $v$ with both sides, and using linearity in the second argument and $\langle v, v_i \rangle = 0$ for all $i$,

$$0 = \langle v,\ c\,v + c_1 v_1 + \cdots + c_k v_k \rangle = c\,\langle v, v \rangle + \sum_{i=1}^{k} c_i\,\langle v, v_i \rangle = c\,\|v\|^2.$$

Since $v \neq 0$, we have $\|v\|^2 > 0$, so $c = 0$. The relation becomes $c_1 v_1 + \cdots + c_k v_k = 0$, and the independence of $v_1, \dots, v_k$ gives $c_1 = \cdots = c_k = 0$. So all coefficients vanish and $v, v_1, \dots, v_k$ are independent. If we had $v = \sum_i \alpha_i v_i$, then $1 \cdot v - \sum_i \alpha_i v_i = 0$ would be a nontrivial relation, so $v \notin \operatorname{span}(v_1, \dots, v_k)$. □

**Remark.** The assumption $v \neq 0$ is needed: the zero vector is orthogonal to every vector, but no set of vectors that contains it is independent ($1 \cdot 0 = 0$ is a nontrivial relation).

If $v_1, \dots, v_k$ are instead only assumed to be nonzero and pairwise orthogonal, then the implication $c_1 v_1 + \cdots + c_k v_k = 0 \Rightarrow c_1 = \cdots = c_k = 0$ becomes the conclusion:

**Corollary 37 ▶ Orthogonal Vectors are Independent**

Let $v_1, \dots, v_k$ be nonzero vectors that are pairwise orthogonal, i.e., $\langle v_i, v_j \rangle = 0$ for all $i \neq j$. Then $v_1, \dots, v_k$ are linearly independent.

*Proof.* Suppose that $c_1 v_1 + \cdots + c_k v_k = 0$ and fix $j$. Taking the inner product of $v_j$ with both sides, all terms with $i \neq j$ vanish, and

$$0 = \sum_{i=1}^{k} c_i\,\langle v_j, v_i \rangle = c_j\,\|v_j\|^2.$$

Since $v_j \neq 0$, this gives $c_j = 0$. This holds for every $j$, so the vectors are independent. □

In particular, the vectors of an orthonormal set, such as the basis states of a quantum system, are always linearly independent, and $n$ of them in an $n$-dimensional space form a basis.

**Lecture Date:** September 24, 2026

**Lecture Topic:** Deutsch's Algorithm, Phase Flip, Walsh–Hadamard Transform, Deutsch–Jozsa Algorithm

In this lecture, we see the first quantum algorithms that beat every classical algorithm in the query model. We study Deutsch's algorithm, which decides with a single query whether a function $f\colon \{0,1\} \to \{0,1\}$ is constant or balanced, the *phase flip* form of a quantum query, the $n$-qubit Hadamard (Walsh–Hadamard) transform, and the Deutsch–Jozsa algorithm, which does the same for $f\colon \{0,1\}^n \to \{0,1\}$, again with a single query.

# 35 Deutsch's Algorithm

**Problem 2 ▶ Deutsch's Problem**

Given a function $f\colon \{0,1\} \to \{0,1\}$ as a black box, determine whether $f$ is *constant* ($f(0) = f(1)$) or *balanced* ($f(0) \neq f(1)$).

Classically we need 2 queries: after one query, say $f(0)$, both answers are still possible. Quantumly, Deutsch's algorithm does it with 1 query.

## 35.1 The Quantum Query $U_f$

A quantum circuit is reversible, but $f$ need not be (a constant $f$ forgets its input). We therefore convert $f$ into a reversible *quantum version* $U_f$: a controlled-$f$ gate modeled on the CNOT gate of Definition 11. Where CNOT XORs the control bit $a$ into the target, $U_f$ XORs $f(a)$ into it (Figure 14):

$$U_f\colon |a\rangle|b\rangle \longmapsto |a\rangle|b \oplus f(a)\rangle.$$

The first qubit (the input $a$) is left unchanged, and $f(a)$ is XORed into the second qubit. Applying $U_f$ twice gives back $|a\rangle|b\rangle$, so $U_f$ is reversible. One application of $U_f$ counts as one query to $f$.

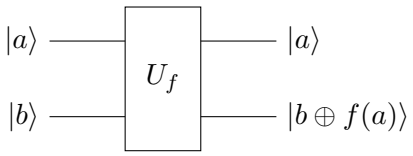



Figure 14: The quantum query $U_f$, a controlled-$f$ gate (compare the CNOT gate of Figure 13).

## 35.2 The Circuit

Deutsch's algorithm is usually given in the form of Figure 15: the first qubit starts in $|0\rangle$ and the second in $|1\rangle$; both go through a Hadamard gate, then through $U_f$, then through a Hadamard gate again, and are measured. We will show that the measurement of the first qubit gives $|0\rangle$ if $f$ is constant and $|1\rangle$ if $f$ is balanced (the second qubit always gives $|1\rangle$).

The action of the Hadamard gate on $|0\rangle$ and $|1\rangle$ (Equation (3)) can be written in one formula: for $a \in \{0,1\}$,

$$H|a\rangle = \frac{1}{\sqrt{2}}(|0\rangle + (-1)^a|1\rangle) = \frac{1}{\sqrt{2}} \sum_{b\in\{0,1\}} (-1)^{ab}|b\rangle. \tag{7}$$

Since $H^2 = I$ (Lemma 30), applying $H$ to the right-hand side of Equation (7) gives back $|a\rangle$.

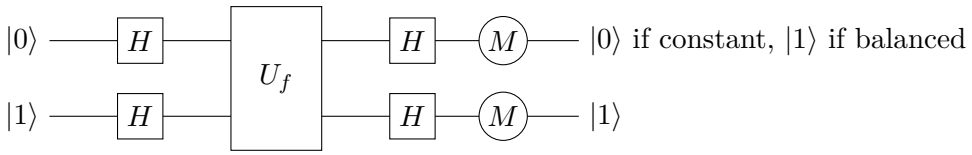



Figure 15: Deutsch's algorithm. It makes one query ($U_f$); the first measurement decides whether $f$ is constant or balanced.

**Theorem 38 ▶ Deutsch's Algorithm**

The circuit of Figure 15 makes one query to $f$, and measuring its first qubit gives $f(0) \oplus f(1)$ with probability 1. Hence the outcome is 0 if $f$ is constant and 1 if $f$ is balanced.

*Proof.* After the first two Hadamard gates the state is (by Equation (4))

$$|\psi_1\rangle = H|0\rangle \otimes H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$
$$= \frac{1}{2}(|0\rangle(|0\rangle - |1\rangle) + |1\rangle(|0\rangle - |1\rangle)).$$

Applying $U_f$ to each term, and using that $|0 \oplus c\rangle - |1 \oplus c\rangle = (-1)^c(|0\rangle - |1\rangle)$ for $c \in \{0,1\}$ (for $c = 1$ the two kets are swapped),

$$|\psi_2\rangle = U_f|\psi_1\rangle = \frac{1}{2}(|0\rangle(|0 \oplus f(0)\rangle - |1 \oplus f(0)\rangle) + |1\rangle(|0 \oplus f(1)\rangle - |1 \oplus f(1)\rangle))$$
$$= \frac{1}{2}\Big(|0\rangle(-1)^{f(0)}(|0\rangle - |1\rangle) + |1\rangle(-1)^{f(1)}(|0\rangle - |1\rangle)\Big)$$
$$= \frac{1}{\sqrt{2}}\Big((-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle\Big) \otimes \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$
$$= (-1)^{f(0)}\frac{1}{\sqrt{2}}\Big(|0\rangle + (-1)^{f(0)\oplus f(1)}|1\rangle\Big) \otimes \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle),$$

where in the last step we used $(-1)^{f(1)} = (-1)^{f(0)}(-1)^{f(0)\oplus f(1)}$. The first qubit is now exactly the right-hand side of Equation (7) with $a = f(0) \oplus f(1)$, up to the sign $(-1)^{f(0)}$. Applying $H$ to it therefore gives

$$|\psi_3\rangle = (-1)^{f(0)}|f(0) \oplus f(1)\rangle \otimes \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle).$$

The factor $(-1)^{f(0)}$ is a global phase and does not affect the measurement, so measuring the first qubit gives $f(0) \oplus f(1)$ with probability 1. (Applying $H$ to the second qubit gives $|1\rangle$, which is why the second measurement in Figure 15 always gives 1.) □

## 36 Phase Flip

The computation above shows a general trick. If the second (target) qubit of $U_f$ is in the state $\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$, then for every input $x$

$$U_f\,|x\rangle\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = (-1)^{f(x)}\,|x\rangle\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle),$$

i.e., the target qubit is unchanged and the value $f(x)$ shows up as a sign in front of $|x\rangle$. Since the target qubit never changes, we drop it and write the query in the short-hand *phase flip* form

$$U_f\colon |x\rangle \longmapsto (-1)^{f(x)}|x\rangle,$$

instead of the original form $U_f\colon |a\rangle|b\rangle \mapsto |a\rangle|b \oplus f(a)\rangle$. By linearity, $U_f$ maps a superposition $\sum_x \alpha_x|x\rangle$ to $\sum_x \alpha_x(-1)^{f(x)}|x\rangle$: one query flips the sign of every $|x\rangle$ with $f(x) = 1$ at once.

## 37 The $n$-Qubit Hadamard Transform

To generalize Deutsch's algorithm to $n$ bits we need the Hadamard gate applied to each of $n$ qubits. Start with $n = 2$ and let $|x\rangle = |x_1x_2\rangle = |x_1\rangle|x_2\rangle$. Using Equation (4) and then Equation (7) on each qubit,

$$\begin{aligned}(H \otimes H)|x\rangle &= H|x_1\rangle \otimes H|x_2\rangle \\ &= \left(\frac{1}{\sqrt{2}} \sum_{y_1 \in \{0,1\}} (-1)^{x_1 y_1} |y_1\rangle\right)\left(\frac{1}{\sqrt{2}} \sum_{y_2 \in \{0,1\}} (-1)^{x_2 y_2} |y_2\rangle\right) \\ &= \frac{1}{2} \sum_{y \in \{0,1\}^2} (-1)^{x_1 y_1 + x_2 y_2} |y\rangle = \frac{1}{2} \sum_{y \in \{0,1\}^2} (-1)^{x \cdot y} |y\rangle,\end{aligned}$$

where $x \cdot y = x_1 y_1 \oplus x_2 y_2$ is the inner product of the bit strings modulo 2 (only the parity of the exponent matters). In general, for $x, y \in \{0,1\}^n$ we write

$$x \cdot y = x_1 y_1 \oplus x_2 y_2 \oplus \cdots \oplus x_n y_n.$$

**Lemma 39 ▶ Walsh–Hadamard Transform**

The $n$-qubit Hadamard transform $H^{\otimes n} = H \otimes H \otimes \cdots \otimes H$ (the *Walsh–Hadamard transform*) acts on a basis state $|x\rangle = |x_1 x_2 \cdots x_n\rangle$ as

$$H^{\otimes n}|x\rangle = \frac{1}{\sqrt{2^n}} \sum_{y \in \{0,1\}^n} (-1)^{x \cdot y} |y\rangle.$$

*Proof.* Exactly as for $n = 2$: apply Equation (7) to each of the $n$ qubits and multiply out the $n$ sums. The exponent is $x_1y_1 + \cdots + x_ny_n$, and $(-1)$ to this power only depends on its parity $x \cdot y$. □

The most important special case is $x = 0^n = 00\cdots0$: then $x \cdot y = 0$ for every $y$, and

$$H^{\otimes n}|00\cdots0\rangle = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle$$

is the uniform superposition of all $2^n$ bit strings.

## 38 Deutsch–Jozsa Algorithm

**Problem 3 ▶ Deutsch–Jozsa Problem**

Given $f\colon \{0,1\}^n \to \{0,1\}$ as a black box, with the *promise* that $f$ is either constant or balanced (i.e., $f(x) = 0$ for exactly half of the $2^n$ inputs), determine which is the case.

This is the same problem as Deutsch's, but on $n$ bits. It is a *promise problem*: we are told in advance that $f$ is one of the two kinds. Let $N = 2^n$ be the number of inputs.

### 38.1 Classical Query Complexity

- *Deterministic:* in the worst case $2^{n-1} + 1 = N/2 + 1$ queries are needed. If an algorithm has seen only $N/2$ values and they are all equal, then $f$ may still be either constant or balanced. The same holds for a randomized algorithm that has to succeed with probability 1: in the worst case it needs $2^{n-1} + 1$ queries.

- *Randomized (Monte Carlo):* query $f$ on $\alpha$ independent uniformly random inputs. If two of the answers differ, output "balanced"; otherwise output "constant". If $f$ is constant, we are always right. If $f$ is balanced, every answer is 0 or 1 with probability $1/2$ each, so all $\alpha$ answers agree, and we fail, with probability

$$2 \cdot \left(\frac{1}{2}\right)^{\alpha} = \left(\frac{1}{2}\right)^{\alpha-1}.$$

  A constant number of queries gives a constant error probability, and $\alpha = \mathcal{O}(\log N)$ queries bring the error probability down to $1/N$.

- *Quantum:* the Deutsch–Jozsa algorithm needs 1 query (and $n$ measurements) and makes no error.

## 38.2 The Algorithm

The circuit (Figure 16) is Deutsch's circuit with $n$ input qubits: $n$ qubits in $|0\rangle$ and one qubit in $|1\rangle$ go through Hadamard gates, then through $U_f$ (one query), then the $n$ input qubits go through Hadamard gates again and are measured. If any of the $n$ measurements gives 1, then $f$ is balanced; if they all give 0, then $f$ is constant.

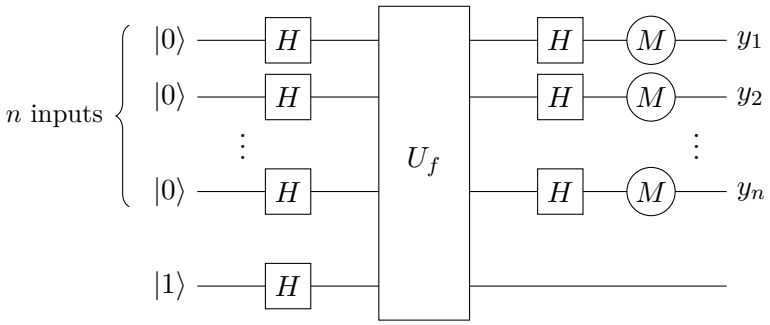



Figure 16: The Deutsch–Jozsa algorithm. It makes one query and $n$ measurements; the last qubit is not needed after the query and is not measured.

**Theorem 40 ▶ Deutsch–Jozsa Algorithm**

In the circuit of Figure 16, measuring the $n$ input qubits gives $y = 0^n$ with probability 1 if $f$ is constant, and with probability 0 if $f$ is balanced. Hence, if any measurement gives 1, then $f$ is balanced; otherwise $f$ is constant.

*Proof.* We use the phase flip form of $U_f$: the last qubit is $\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$ throughout and is dropped. The $n$ input qubits start in $|00\cdots0\rangle$. After the first layer of Hadamard gates,

$$H^{\otimes n}|00\cdots0\rangle = \frac{1}{\sqrt{2^n}} \sum_{x\in\{0,1\}^n} |x\rangle.$$

The query $U_f$ flips the sign of every $|x\rangle$ with $f(x) = 1$:

$$\frac{1}{\sqrt{2^n}} \sum_{x\in\{0,1\}^n} (-1)^{f(x)}|x\rangle.$$

After the second layer of Hadamard gates, by Lemma 39,

$$\frac{1}{\sqrt{2^n}} \sum_{x\in\{0,1\}^n} (-1)^{f(x)} \frac{1}{\sqrt{2^n}} \sum_{y\in\{0,1\}^n} (-1)^{x\cdot y}|y\rangle = \frac{1}{2^n} \sum_{y\in\{0,1\}^n} \sum_{x\in\{0,1\}^n} (-1)^{f(x)+x\cdot y}|y\rangle.$$

Look at the amplitude of $|y\rangle$ for $y = 0^n$. Since $x \cdot 0^n = 0$ for every $x$, it is

$$\frac{1}{2^n} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} = \begin{cases} \pm 1 & \text{if } f \text{ is constant (all } 2^n \text{ terms are equal),} \\ 0 & \text{if } f \text{ is balanced (half the terms are } +1\text{, half are } -1\text{).} \end{cases}$$

If $f$ is constant, the state is $\pm|00\cdots0\rangle$ (all other amplitudes must be 0, since the amplitudes squared sum to 1), so the measurement gives $0^n$ with probability 1. If $f$ is balanced, the amplitude of $|00\cdots0\rangle$ is 0, so the measurement gives $0^n$ with probability 0, i.e., at least one of the $n$ measurements gives 1. □

For $n = 1$ this is exactly Deutsch's algorithm. In both cases a single query suffices, whereas any classical algorithm with zero error needs $2^{n-1} + 1$ queries.

**Lecture Date:** September 29, 2026

**Lecture Topic:** Walsh–Hadamard Transform Recap, Bernstein–Vazirani Algorithm, Simon's Algorithm

In this lecture, we first recall the quantum query $U_f$, the Walsh–Hadamard transform and the analysis of the Deutsch–Jozsa algorithm from the last lecture. We then see two more problems that a quantum algorithm solves with far fewer queries than any classical algorithm: the *Bernstein–Vazirani* problem (one quantum query instead of $n$ classical queries) and *Simon's problem* ($\mathcal{O}(n)$ quantum queries instead of $\Omega\left(\sqrt{2^n}\right)$ classical queries). Simon's algorithm is the first one where the quantum speed-up is exponential.

## 39 Recap: Quantum Queries and the Walsh–Hadamard Transform

We recall the tools of the last lecture that both algorithms below use; the proofs are in the scribe of September 24. A function $f\colon \{0,1\}^n \to \{0,1\}^m$ is given as a black box, and in the quantum query model the black box is the unitary

$$U_f\colon |x\rangle|b\rangle \longmapsto |x\rangle|b \oplus f(x)\rangle,$$

which XORs $f(x)$ into the *target* register and leaves the *control* register $|x\rangle$ unchanged (Figure 14). Every classical circuit can be made into a reversible quantum circuit, so whenever $f$ has a classical circuit, $U_f$ has a quantum circuit of about the same size. One application of $U_f$ is one query. The Hadamard gate acts as $H|a\rangle = \frac{1}{\sqrt{2}}\sum_{y\in\{0,1\}}(-1)^{ay}|y\rangle$, and on $n$ qubits, with $x \cdot y = x_1y_1 \oplus \cdots \oplus x_ny_n$ the inner product of bit strings modulo 2, the *Walsh–Hadamard transform* (Lemma 39) is

$$H^{\otimes n}|x\rangle = \frac{1}{\sqrt{2^n}} \sum_{y\in\{0,1\}^n} (-1)^{x\cdot y}|y\rangle, \qquad \text{in particular} \qquad H^{\otimes n}|0^n\rangle = \frac{1}{\sqrt{2^n}} \sum_{y\in\{0,1\}^n} |y\rangle.$$

It is the *Fourier transform* over the group $\{0,1\}^n$ (with $\oplus$ as the group operation): the sign $(-1)^{x\cdot y}$ plays the role that the root of unity $e^{2\pi i\, xy/N}$ plays in the usual discrete Fourier transform. We will use the following sum many times; it is the Fourier-transform fact that "a nonzero character sums to zero".

**Lemma 41 ▶ Sum of Signs**

For every $z \in \{0,1\}^n$,

$$\sum_{x\in\{0,1\}^n} (-1)^{z\cdot x} = \begin{cases} 2^n & \text{if } z = 0^n, \\ 0 & \text{if } z \neq 0^n. \end{cases}$$

*Proof.* If $z = 0^n$, every term is $(-1)^0 = 1$ and there are $2^n$ terms. If $z \neq 0^n$, pick an index $i$ with $z_i = 1$ and let $e_i$ be the string with a single 1 in position $i$. The map $x \mapsto x \oplus e_i$ pairs every $x$ with a different string (applying it twice gives $x$ back), so it splits $\{0,1\}^n$ into $2^{n-1}$ disjoint pairs $\{x, x \oplus e_i\}$. The inner product is linear modulo 2, so $z\cdot(x\oplus e_i) = z\cdot x \oplus z\cdot e_i = z\cdot x \oplus z_i = z\cdot x \oplus 1$. So the two strings of every pair contribute $(-1)^{z\cdot x} + (-1)^{z\cdot x\oplus 1} = 0$, and the whole sum is 0. □

In the Deutsch–Jozsa circuit (Figure 16) the $n$ control qubits start in $|0^n\rangle$ and the target qubit in $|1\rangle$. After the first layer of Hadamard gates the target is $\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$, so the query acts

on the control register as the *phase flip* $|k\rangle \mapsto (-1)^{f(k)}|k\rangle$ and the target never changes again. After the second layer of Hadamard gates the control register is in the state

$$\sum_{y\in\{0,1\}^n} \alpha_y |y\rangle, \qquad \alpha_y = \frac{1}{2^n} \sum_{k\in\{0,1\}^n} (-1)^{f(k)+k\cdot y}, \tag{8}$$

and $|\alpha_{0^n}|^2 = \left|\frac{1}{2^n}\sum_k (-1)^{f(k)}\right|^2$ is 1 if $f$ is constant and 0 if $f$ is balanced, which is Theorem 40. This computation was redone on the board; Equation (8) is all that the next section needs.

# 40 Bernstein–Vazirani Algorithm

**Problem 4 ▶ Bernstein–Vazirani Problem**

Given $f\colon \{0,1\}^n \to \{0,1\}$ as a black box, with the *promise* that

$$f(x) = s \cdot x = s_1x_1 \oplus s_2x_2 \oplus \cdots \oplus s_nx_n$$

for some unknown $n$-bit string $s$, find $s$.

Again this is a promise problem: we are told that $f$ has this form. Classically, $n$ queries suffice and are needed: querying the unit string $e_i$ (a single 1 in position $i$) returns $f(e_i) = s_i$, one bit of $s$ per query.

## 40.1 The Algorithm

The quantum algorithm uses exactly the Deutsch–Jozsa circuit (Figure 16), with the same analysis. It makes one query.

**Theorem 42 ▶ Bernstein–Vazirani Algorithm**

Run the Deutsch–Jozsa circuit on $f(x) = s \cdot x$. Then measuring the $n$ control qubits gives $y = s$ with probability 1. Hence $s$ is found with a single query.

*Proof.* By Equation (8) (with the summation index $k$ renamed $x$), before the measurement the control register is in the state $\sum_y \alpha_y|y\rangle$ with

$$\alpha_y = \frac{1}{2^n} \sum_{x\in\{0,1\}^n} (-1)^{s\cdot x \oplus y\cdot x} = \frac{1}{2^n} \sum_{x\in\{0,1\}^n} (-1)^{(s\oplus y)\cdot x},$$

where we used that only the parity of the exponent matters and that $s\cdot x \oplus y\cdot x = (s\oplus y)\cdot x$ (the inner product is bilinear over $\{0,1\}$, i.e., modulo 2). By Lemma 41 applied with $z = s \oplus y$,

$$\alpha_y = \begin{cases} 1 & \text{if } y = s, \\ 0 & \text{if } y \neq s. \end{cases}$$

So the state is exactly $|s\rangle$, and the measurement gives $s$ with probability 1. □

The Deutsch–Jozsa algorithm and the Bernstein–Vazirani algorithm are the same circuit; the difference is only in what the promise says about $f$ and in how we read the measured string $y$.

**Query complexity versus running time.** The query model only counts the calls to $U_f$. The running time of the algorithm in the *quantum Turing machine* (or quantum circuit) model counts all the gates, and the two measures are not always the same: an algorithm can make few queries but need many other gates. For Deutsch–Jozsa and Bernstein–Vazirani the circuit has, besides the single query, only $2n + 1$ Hadamard gates and $n$ measurements, so here both measures are small.

# 41 Simon's Algorithm

**Problem 5 ▶ Simon's Problem**

Given $f\colon \{0,1\}^n \to \{0,1\}^n$ as a black box, with the *promise* that there is a string $s \in \{0,1\}^n$ such that for all $x, y$,

$$f(x) = f(y) \iff x = y \text{ or } x = y \oplus s \qquad \text{(equivalently } s = x \oplus y\text{)},$$

find $s$.

If $s \neq 0^n$, then $f$ is a 2-*to*-1 function: the inputs come in pairs $\{x, x \oplus s\}$, and the two inputs of a pair map to the same output while different pairs map to different outputs (Figure 17). If $s = 0^n$, then $f(x) = f(y)$ only for $x = y$, i.e., $f$ is 1-*to*-1. So the problem contains the decision problem "is $f$ 1-to-1 or 2-to-1?" and, in the second case, asks for the period $s$ of $f$. We write $N = 2^n$.

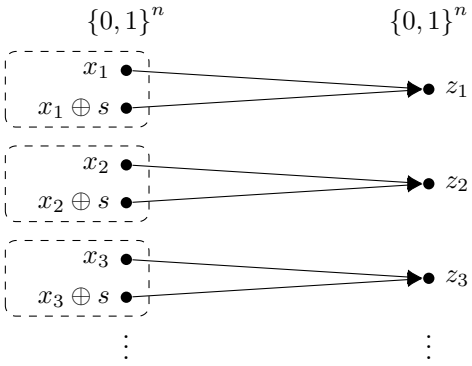



Figure 17: Simon's promise for $s \neq 0^n$: $f$ is 2-to-1, and the two inputs of every pair $\{x, x \oplus s\}$ map to the same output. The range of $f$ has $2^{n-1}$ elements.

**Query complexity.** Classically, Simon's problem needs $\Omega\left(\sqrt{N}\right) = \Omega\left(\sqrt{2^n}\right)$ queries, even for randomized algorithms (next lecture). Simon's quantum algorithm uses $\mathcal{O}(n)$ queries. The only expensive classical part of the algorithm is a Gaussian elimination on an $n \times n$ system of linear equations over $\{0,1\}$, which takes $\mathcal{O}(n^3)$ time. This is an exponential speed-up.

## 41.1 The Circuit

The circuit (Figure 18) has two registers of $n$ qubits each, all starting in $|0\rangle$. The control register goes through Hadamard gates, then both registers go through $U_f$ (one query), then the control register goes through Hadamard gates again and is measured, giving a string $y \in \{0,1\}^n$. Since $f$ has $n$ output bits, we use $U_f$ in its original form $|x\rangle|b\rangle \mapsto |x\rangle|b \oplus f(x)\rangle$ and not the phase flip form.

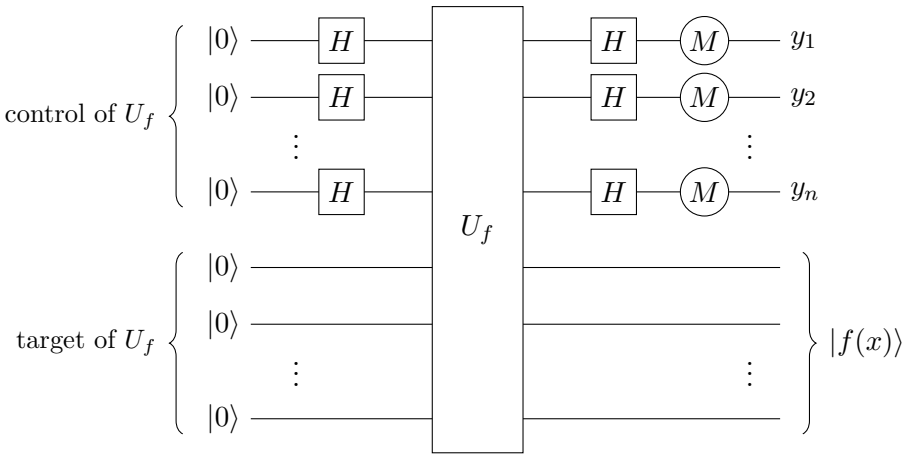



Figure 18: Simon's circuit. It makes one query and measures the $n$ control qubits, giving a string $y$. The target register is not measured.

## 41.2 Analysis of One Run

For a state $\sum_{y,z} \alpha_{y,z}|y\rangle|z\rangle$ of the two registers, measuring only the first register gives $y$ with probability $\sum_z |\alpha_{y,z}|^2$ (the probabilities of all the basis states $|y\rangle|z\rangle$ with this $y$ add up).

**Theorem 43 ▶ One Run of Simon's Circuit**

Let $y$ be the string measured by the circuit of Figure 18.

1. If $f$ is 2-to-1 with period $s \neq 0^n$, then $y$ is uniformly distributed over the $2^{n-1}$ strings $y$ with $s \cdot y = 0$: each of them has probability $1/2^{n-1}$, and every $y$ with $s \cdot y = 1$ has probability 0.
2. If $f$ is 1-to-1, then $y$ is uniformly distributed over all of $\{0,1\}^n$: each string has probability $1/2^n$.

*Proof.* We follow the state through the circuit. After the first layer of Hadamard gates, by Lemma 39,

$$\frac{1}{\sqrt{2^n}} \sum_{x\in\{0,1\}^n} |x\rangle|0^n\rangle.$$

After the query $U_f$,

$$\frac{1}{\sqrt{2^n}} \sum_{x\in\{0,1\}^n} |x\rangle|f(x)\rangle.$$

After the second layer of Hadamard gates on the control register, by Lemma 39 applied to every $|x\rangle$,

$$\frac{1}{\sqrt{2^n}} \sum_{x\in\{0,1\}^n} \frac{1}{\sqrt{2^n}} \sum_{y\in\{0,1\}^n} (-1)^{x\cdot y}|y\rangle|f(x)\rangle = \sum_{y\in\{0,1\}^n} \frac{1}{2^n} \sum_{x\in\{0,1\}^n} (-1)^{x\cdot y}|y\rangle|f(x)\rangle. \tag{9}$$

The target register now carries $f(x)$, so to read off the amplitude of a basis state $|y\rangle|z\rangle$ we have to collect the $x$ with $f(x) = z$.

*Case 2: $f$ is 1-to-1.* Then the strings $f(x)$, $x \in \{0,1\}^n$, are all different, so every basis state $|y\rangle|f(x)\rangle$ in Equation (9) appears exactly once, with amplitude $\frac{1}{2^n}(-1)^{x\cdot y}$. Measuring the first register gives $y$ with probability

$$\sum_{x\in\{0,1\}^n} \left|\frac{1}{2^n}(-1)^{x\cdot y}\right|^2 = 2^n \cdot \frac{1}{2^{2n}} = \frac{1}{2^n}.$$

Every string $y$ is equally likely, and these $2^n$ probabilities add up to 1.

*Case 1: $f$ is 2-to-1 with period $s \neq 0^n$.* Let $R$ be the range of $f$; it has $|R| = 2^n/2 = 2^{n-1}$ elements. For every $z \in R$ pick one of its two preimages and call it $x_z$; the other preimage is $x'_z = x_z \oplus s$. Collecting in Equation (9) the two $x$ with $f(x) = z$, the state is

$$\sum_{y\in\{0,1\}^n} \frac{1}{2^n} \sum_{z\in R} \Big( (-1)^{x_z\cdot y} + (-1)^{(x_z\oplus s)\cdot y} \Big) |y\rangle|z\rangle = \sum_{y\in\{0,1\}^n} \frac{1}{2^n} \sum_{z\in R} (-1)^{x_z\cdot y} (1 + (-1)^{s\cdot y}) |y\rangle|z\rangle,$$

since $(x_z \oplus s)\cdot y = x_z \cdot y \oplus s \cdot y$. The amplitude of $|y\rangle|z\rangle$ is therefore $\frac{1}{2^n}(-1)^{x_z\cdot y}(1 + (-1)^{s\cdot y})$, whose absolute value does not depend on the sign $(-1)^{x_z\cdot y}$, and measuring the first register gives $y$ with probability

$$\sum_{z\in R} \left| \frac{1}{2^n}(1 + (-1)^{s\cdot y}) \right|^2 = 2^{n-1} \cdot \frac{(1 + (-1)^{s\cdot y})^2}{2^{2n}} = \begin{cases} 2^{n-1} \cdot \dfrac{4}{2^{2n}} = \dfrac{2}{2^n} = \dfrac{1}{2^{n-1}} & \text{if } s\cdot y = 0, \\ 0 & \text{if } s\cdot y = 1. \end{cases}$$

The strings $y$ with $s \cdot y = 0$ form the solution set of one nontrivial linear equation over $\{0,1\}$, so there are $2^{n-1}$ of them (the pairing $y \mapsto y \oplus e_i$ of the proof of Lemma 41, with $s_i = 1$, matches every $y$ with $s \cdot y = 0$ to a $y'$ with $s \cdot y' = 1$), and their probabilities add up to $2^{n-1} \cdot 1/2^{n-1} = 1$. □

Note that the second register was needed to *entangle* $x$ with $f(x)$, so that only the two preimages of the same $z$ interfere; we never look at it.

### 41.3 From the Measurements to $s$

Each run of the circuit gives one piece of information about $s$: a random $y$ with $s \cdot y = 0$, i.e., one linear equation

$$s_1y_1 \oplus s_2y_2 \oplus \cdots \oplus s_ny_n = 0$$

for the unknown bits $s_1, \ldots, s_n$. (In class the count was "$n$ measurements give $n$ equations in $n$ unknowns"; since $s$ itself satisfies every equation, at most $n-1$ of them can be independent, and $n-1$ independent ones are enough, so we run the circuit $n-1$ times. One more run would not hurt.) Running the circuit $n-1$ times gives $n-1$ equations

$$y_1 \cdot s = 0, \quad y_2 \cdot s = 0, \quad \ldots, \quad y_{n-1} \cdot s = 0$$

in the $n$ unknowns $s_1, \ldots, s_n$, a homogeneous system of linear equations over the field $\{0,1\}$ (addition is $\oplus$). If the $y_1, \ldots, y_{n-1}$ are *linearly independent* over $\{0,1\}$, the system has rank $n-1$ and its solution space has dimension $n - (n-1) = 1$, i.e., exactly two solutions: $0^n$ and one nonzero string $s'$. Both the test for linear independence and the solution $s'$ are computed by Gaussian elimination over $\{0,1\}$ in $\mathcal{O}(n^3)$ time. Since $s \cdot y_i = 0$ for every $i$, the true $s$ is always a solution; so if the $y_i$ are independent and $f$ is 2-to-1, then $s' = s$.

**Algorithm 1** Simon's algorithm

```
run the circuit of Figure 18 n − 1 times; let y_1, ..., y_{n−1} be the measured strings
if y_1, ..., y_{n−1} are linearly dependent over {0,1}:    ▷ Gaussian elimination
    return FAIL
let s' be the unique nonzero solution of y_1 · s' = ··· = y_{n−1} · s' = 0
if f(s') = f(0^n):    ▷ two classical queries
    return s'    ▷ f is 2-to-1 with period s'
else:
    return 0^n    ▷ f is 1-to-1
```

The probability that the $y_i$ are linearly independent will be shown in the next lecture (Theorem 53) to be, in the 2-to-1 case,

$$\left(1-\frac{1}{2}\right)\left(1-\frac{1}{2^2}\right)\left(1-\frac{1}{2^3}\right)\cdots\left(1-\frac{1}{2^{n-1}}\right) \geqslant \frac{1}{4},$$

and even larger (at least 1/2) in the 1-to-1 case. This gives:

**Theorem 44 ▶ Simon's Algorithm**

Algorithm 1 makes $n+1$ queries ($n-1$ quantum, 2 classical) and $\mathcal{O}(n^3)$ other operations. It never outputs a wrong answer, and it outputs the correct $s$ with probability at least 1/4. Repeating it until it does not fail, $t$ repetitions fail with probability at most $(3/4)^t$; so $\mathcal{O}(n)$ queries find $s$ with constant probability and $\mathcal{O}(n \log n)$ queries find it with probability $1 - 1/n$.

*Proof.* Suppose the algorithm does not fail, so $y_1, \ldots, y_{n-1}$ are independent and $s'$ is the unique nonzero solution of the system. If $f$ is 2-to-1 with period $s$, then $s$ is a nonzero solution, so $s' = s$, and the test $f(s') = f(0^n) = f(0^n \oplus s)$ succeeds: the output $s' = s$ is correct. If $f$ is 1-to-1, then $f(s') \neq f(0^n)$ because $s' \neq 0^n$, and the output $0^n = s$ is correct. So the output is never wrong.

The algorithm fails only if the $y_i$ are dependent. The $n-1$ runs of the circuit are independent (the register is re-initialized to $|0^n\rangle|0^n\rangle$ for every run). If $f$ is 2-to-1, by Theorem 43 the $y_i$ are independent uniform samples from the $2^{n-1}$ strings with $s \cdot y = 0$, and Theorem 53 shows that they are linearly independent with probability at least 1/4. If $f$ is 1-to-1, the $y_i$ are independent uniform samples from all of $\{0,1\}^n$, and Theorem 53 shows that they are linearly independent with probability at least 1/2. In both cases the success probability is at least 1/4, and $t$ independent repetitions all fail with probability at most $(3/4)^t$. With $t = \left\lceil \log_{4/3} n \right\rceil$ repetitions this is at most $1/n$, and the number of queries is $t(n+1) = \mathcal{O}(n \log n)$. □

## 41.4 A Deterministic Classical Algorithm

The following was given as an exercise in class. It shows that, classically, $\mathcal{O}\left(\sqrt{2^n}\right)$ queries suffice even without randomness; the next lecture shows that $\Omega\left(\sqrt{2^n}\right)$ queries are also necessary, so the exercise is the best possible classical algorithm up to a constant factor.

**Exercise 15.** Find a deterministic classical algorithm for Simon's problem that makes $\mathcal{O}\left(\sqrt{2^n}\right)$ queries.

**Solution**. The idea is to query a set $S$ of inputs such that every nonzero string $s$ is the XOR of two elements of $S$. Then, if $f$ is 2-to-1 with period $s$, those two elements collide ($f(x) = f(x \oplus s)$) and the collision reveals $s$.

Let $m = \lceil n/2 \rceil$ and split every $n$-bit string $x$ as $x = (u, v)$ with $u \in \{0,1\}^m$ its first $m$ bits and $v \in \{0,1\}^{n-m}$ its last $n-m$ bits. Let

$$S_1 = \{(u, 0^{n-m}) \mid u \in \{0,1\}^m\}, \qquad S_2 = \left\{(0^m, v) \mid v \in \{0,1\}^{n-m}\right\}, \qquad S = S_1 \cup S_2.$$

Then, since $n - m \leqslant m \leqslant (n+1)/2$, $|S| \leqslant 2^m + 2^{n-m} \leqslant 2 \cdot 2^m \leqslant 2 \cdot 2^{(n+1)/2} = 2\sqrt{2} \cdot \sqrt{2^n}$.

*Claim: every $s \neq 0^n$ is the XOR of two different elements of $S$.* Write $s = (u, v)$. Then $(u, 0^{n-m}) \in S_1$, $(0^m, v) \in S_2$ and $(u, 0^{n-m}) \oplus (0^m, v) = (u, v) = s$. The two elements are different, since otherwise $u = 0^m$ and $v = 0^{n-m}$, i.e., $s = 0^n$.

*The algorithm.* Query $f$ on every $x \in S$ ($|S|$ queries). If there are $x \neq x'$ in $S$ with $f(x) = f(x')$, output $s = x \oplus x'$; otherwise output $s = 0^n$.

*Correctness.* If $f$ is 2-to-1 with period $s \neq 0^n$, by the claim there are $x \neq x'$ in $S$ with $x \oplus x' = s$, i.e., $x' = x \oplus s$, so $f(x) = f(x')$ and the algorithm finds a collision. Any collision $f(x) = f(x')$ with $x \neq x'$ satisfies $x' = x \oplus s$ by the promise, so the output $x \oplus x' = s$ is correct whichever collision is found. If $f$ is 1-to-1 there is no collision, and the output $0^n = s$ is correct.

The algorithm is deterministic, always correct, and makes $|S| \leqslant 2\sqrt{2} \cdot \sqrt{2^n} = \mathcal{O}\left(\sqrt{2^n}\right)$ queries. □

**Lecture Date:** October 1, 2026

**Lecture Topic:** Classical Query Complexity of Simon's Problem, Yao's Minmax Principle, Linear Independence in Simon's Algorithm

In this lecture, we determine the *classical* query complexity of Simon's problem. A randomized algorithm that queries random inputs and waits for a collision uses $\mathcal{O}(\sqrt{N \log N})$ queries, where $N = 2^n$, and we show with Yao's minmax principle that every randomized algorithm needs $\Omega\left(\sqrt{N}\right)$ queries. We then go through Simon's quantum algorithm once more, compute the measurement probabilities in both cases, and prove that the measured strings are linearly independent with probability at least $1/4$, which completes the analysis of the last lecture.

# 42 Simon's Problem, Classically

Recall Simon's problem: $f\colon \{0,1\}^n \to \{0,1\}^n$ is given as a black box, and we are promised that $f$ is either 1-to-1, or 2-to-1 with a period $s \neq 0^n$, i.e.,

$$f(x) = f(y) \iff x = y \quad \text{or} \quad x \oplus y = s \text{ (i.e., } y = x \oplus s).$$

We have to find $s$ (with $s = 0^n$ meaning that $f$ is 1-to-1). Let $N = 2^n$. A classical algorithm learns $f$ only through queries $f(q_1), f(q_2), \ldots$, and we may assume that it never queries the same input twice, since a repeated query gives no new information. The algorithm can find $s$ only by finding a *collision*, two queries $q_i \neq q_j$ with $f(q_i) = f(q_j)$: by the promise this gives $s = q_i \oplus q_j$ immediately. Conversely, as long as all answers are distinct, the algorithm has no direct evidence of $s$.

## 42.1 Deterministic Algorithms

The exercise of the last lecture (Exercise 15) gives a deterministic algorithm with $\mathcal{O}\left(\sqrt{N}\right)$ queries. In class it was stated that this is optimal: every deterministic algorithm needs $\Omega\left(\sqrt{2^n}\right)$ queries. This is a special case of the lower bound for randomized algorithms proved below (Corollary 50).

## 42.2 A Randomized Algorithm

The natural randomized algorithm is the birthday paradox of the lecture of September 3: query random inputs and wait for a collision.

**Algorithm 2** Randomized classical algorithm for Simon's problem

choose $k$ distinct inputs $q_1, \ldots, q_k$ uniformly at random (without replacement) and query them
**if** $f(q_i) = f(q_j)$ for some $i \neq j$:
    **return** $q_i \oplus q_j$
**else**:
    **return** $0^n$

**Theorem 45 ▶ Randomized Upper Bound**

If $f$ is 1-to-1, Algorithm 2 is always correct. If $f$ is 2-to-1 with period $s$, it is wrong with probability at most $e^{-k(k-1)/(2N)}$. In particular, with $k \geqslant \sqrt{2N \ln N} + 1$ queries (an integer at most $N$ once $n \geqslant 3$) the algorithm is correct with probability at least $1 - 1/N$, and with $k \geqslant \sqrt{2N} + 1$, i.e., $\mathcal{O}\left(\sqrt{N}\right)$, queries with probability at least $1 - 1/e$.

*Proof.* If $f$ is 1-to-1 there is no collision, and the output $0^n$ is correct. Let $f$ be 2-to-1 with period $s \neq 0^n$. Any collision $f(q_i) = f(q_j)$ gives $q_i \oplus q_j = s$ by the promise, so the algorithm is wrong only if no collision occurs among the $k$ queries. Call $q_j \oplus s$ the *mirror* of $q_j$; a collision occurs at step $i$ exactly when $q_i$ is the mirror of an earlier query $q_j$, $j < i$.

Suppose that no collision occurred among $q_1, \ldots, q_{i-1}$. The $i-1$ mirrors $q_1 \oplus s, \ldots, q_{i-1} \oplus s$ are distinct (because the $q_j$ are distinct), and none of them is among $q_1, \ldots, q_{i-1}$ (that would be a collision). So all $i-1$ mirrors are among the $N - (i-1)$ inputs that have not been queried, and $q_i$ is uniform among those. Hence

$$\mathbb{P}(\text{collision at step } i \mid \text{no collision before}) = \frac{i-1}{N-i+1} \geqslant \frac{i-1}{N},$$

and the probability of no collision at step $i$, given none before, is at most $1 - (i-1)/N$. These are the factors $1, 1 - 1/N, 1 - 2/N, \ldots$ in the notes. Multiplying over the $k$ steps and using $1 - x \leqslant e^{-x}$,

$$\begin{aligned}\mathbb{P}(\text{no collision among } q_1, \ldots, q_k) &\leqslant \prod_{i=1}^{k} \left(1 - \frac{i-1}{N}\right) \leqslant \prod_{i=1}^{k} e^{-(i-1)/N} \\ &= e^{-\sum_{i=1}^{k}(i-1)/N} = e^{-k(k-1)/(2N)}.\end{aligned}$$

With $k - 1 \geqslant \sqrt{2N \ln N}$ this is at most $e^{-(k-1)^2/(2N)} \leqslant e^{-\ln N} = 1/N$, and with $k - 1 \geqslant \sqrt{2N}$ it is at most $e^{-1}$. □

So $\mathcal{O}\left(\sqrt{N}\right)$ queries give a constant success probability and $\mathcal{O}(\sqrt{N \log N})$ queries give success probability $1 - 1/N$ ("with high probability"). The next theorem shows that randomness cannot do much better than this.

## 42.3 Lower Bound for Randomized Algorithms via Yao's Principle

**Theorem 46 ▶ Randomized Lower Bound**

Let $n \geqslant 2$. Every randomized classical algorithm that solves Simon's problem with success probability at least $2/3$ on every input makes more than $\sqrt{N}/2$ queries on some input. Hence every such algorithm needs $\Omega\left(\sqrt{N}\right) = \Omega\left(\sqrt{2^n}\right)$ queries.

The proof uses Yao's minmax principle from the lecture of September 1: to lower bound randomized algorithms on their worst input, it is enough to pick a *distribution* $\mu$ on the inputs and to lower bound the error of every *deterministic* algorithm on an input drawn from $\mu$. On September 1 the principle was stated for the expected number of queries of Las Vegas algorithms; here we need the version for the error probability of algorithms with a fixed number of queries, and only its easy direction, which is an averaging argument:

**Lemma 47 ▶ Yao's Minmax Principle, Easy Direction**

Let $\mu$ be a probability distribution on the inputs. If every deterministic algorithm that makes at most $k$ queries errs with probability greater than $\delta$ on an input drawn from $\mu$, then every randomized algorithm that makes at most $k$ queries errs with probability greater than $\delta$ on some input.

*Proof.* A randomized algorithm $R$ is a deterministic algorithm $A_r$ together with a random string $r$: once $r$ is fixed, $A_r$ is deterministic and makes at most $k$ queries. Let $\mathrm{err}(A, x) \in \{0, 1\}$ indicate that $A$ is wrong on input $x$. Then

$$\max_x \mathbb{P}_r(\mathrm{err}(A_r, x) = 1) \geqslant \mathbb{E}_{x\sim\mu}[\mathbb{P}_r(\mathrm{err}(A_r, x) = 1)] = \mathbb{E}_r[\mathbb{P}_{x\sim\mu}(\mathrm{err}(A_r, x) = 1)] > \delta,$$

since a maximum is at least an average, the order of the two expectations can be exchanged, and every inner probability on the right is greater than $\delta$ by assumption. So on the input $x$ achieving the maximum, $R$ errs with probability greater than $\delta$. □

**The input distribution.** We use the distribution $\mu$ of Figure 19:

- with probability $1/2$, $f$ is a uniformly random 1-to-1 function, i.e., the table of values $(f(0^n), \ldots, f(1^n))$ is a uniformly random permutation of $\{0, 1\}^n$ (call this distribution $\mathcal{P}$);
- with probability $1/2$, $f$ is drawn from the distribution $\mathcal{D}$ of random 2-to-1 functions: choose $s$ uniformly among the $N - 1$ nonzero strings, which splits $\{0, 1\}^n$ into $N/2$ pairs $\{x, x \oplus s\}$; then assign to the pairs distinct values, uniformly at random among all such assignments, and let $f(x) = f(x \oplus s)$ be the value of the pair of $x$.

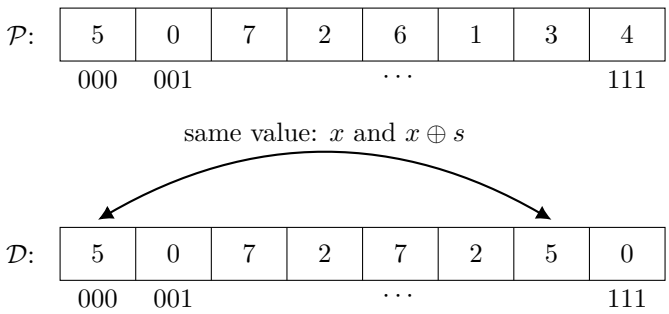



Figure 19: The two halves of the input distribution $\mu$ for $n = 3$ (values written as integers). Under $\mathcal{P}$ the table is a random permutation. Under $\mathcal{D}$ a random nonzero $s$ (here $s = 110$) pairs up the inputs, and the two inputs of a pair get the same random value, different pairs different values.

**Principle of deferred decisions.** We do not have to choose the whole table in advance. We can generate the random input lazily, while the algorithm runs, and only decide the values that the algorithm asks for; the distribution of the answers is the same.

- Under $\mathcal{P}$: when the algorithm queries a new input $q_i$, the answer $f(q_i)$ is a uniformly random value among the $N - (i - 1)$ values not used by the earlier answers.
- Under $\mathcal{D}$: choose $s$ uniformly among the nonzero strings and keep it hidden. When the algorithm queries $q_i$: if $q_i = q_j \oplus s$ for an earlier query $q_j$ (a *collision*: $q_i$ is the mirror of $q_j$), the answer is the earlier answer $f(q_j)$; otherwise $q_i$ lies in a new pair, and the answer is a uniformly random value among the values not used so far ($N - (i - 1)$ of them as long as no collision has occurred).

Both procedures produce exactly the distributions $\mathcal{P}$ and $\mathcal{D}$ on the queried values, because a uniformly random injective assignment of values can be built one value at a time, each new value uniform among the unused ones.

The lower bound has two steps: (1) under $\mathcal{D}$, an algorithm with few queries is unlikely to see a collision; (2) as long as it sees no collision, its answers look exactly as under $\mathcal{P}$, so it cannot tell the two cases apart. Step (1) is the computation done in class:

**Lemma 48 ▶ Few Queries Rarely Collide**

Let $n \geqslant 2$, let $0 < \epsilon \leqslant 1/2$, and let $A$ be a deterministic algorithm that makes at most $k \leqslant \epsilon\sqrt{N}$ queries. Under $\mathcal{D}$, the probability that two of the queries of $A$ form a pair $\{x, x \oplus s\}$ (a collision) is at most

$$\sum_{i=1}^{k} \frac{i-1}{N-1-\binom{i-1}{2}} \leqslant \frac{k(k-1)}{N} < \epsilon^2.$$

*Proof.* Let $Q_i$ be the event that there is no collision among the first $i$ queries, and let $C_i$ be the event that the $i$th query collides with an earlier one, i.e., $q_i \oplus s \in \{q_1, \ldots, q_{i-1}\}$. A collision among the $k$ queries means that some $C_i$ occurs, and if $C_i$ is the first one then $Q_{i-1}$ holds. So by the union bound

$$\mathbb{P}(\text{collision}) \leqslant \sum_{i=1}^{k} \mathbb{P}(C_i \cap Q_{i-1}).$$

Fix $i$ and condition on $Q_{i-1}$ and on the answers $f(q_1), \ldots, f(q_{i-1})$ seen so far (they are distinct, as $Q_{i-1}$ holds). What does the algorithm know about $s$? Since $A$ is deterministic, the queries $q_1, \ldots, q_{i-1}$ are the same for every $s$ that leads to these answers. By the lazy generation above, the answers were $i-1$ fresh random values, and this particular sequence of answers has probability $\prod_{j=1}^{i-1} 1/(N-j+1)$, the same for every $s$ that is not of the form $q_j \oplus q_l$ with $j < l \leqslant i-1$; for an $s$ of this form, the queries $q_j$ and $q_l$ would have collided, contradicting $Q_{i-1}$, so the probability is 0. By Bayes' rule, given the answers, $s$ is uniformly distributed over the nonzero strings that are not among the at most $\binom{i-1}{2}$ strings $q_j \oplus q_l$, a set of at least $N-1-\binom{i-1}{2}$ candidates. The next query $q_i$ is determined by the answers ($A$ is deterministic), and $C_i$ occurs exactly when $s \in \{q_i \oplus q_1, \ldots, q_i \oplus q_{i-1}\}$, a set of $i-1$ strings. Therefore

$$\mathbb{P}(C_i \mid Q_{i-1}, \text{answers}) \leqslant \frac{i-1}{N-1-\binom{i-1}{2}},$$

and the same bound holds for $\mathbb{P}(C_i \cap Q_{i-1})$ after averaging over the answers.

For the final estimate, $\binom{i-1}{2} \leqslant \binom{k}{2} \leqslant k^2/2 \leqslant \epsilon^2 N/2 \leqslant N/8$, so the denominator is at least $N-1-N/8 \geqslant N/2$ for $N \geqslant 4$. Hence

$$\mathbb{P}(\text{collision}) \leqslant \sum_{i=1}^{k} \frac{2(i-1)}{N} = \frac{2}{N} \cdot \frac{k(k-1)}{2} = \frac{k(k-1)}{N} < \frac{k^2}{N} \leqslant \epsilon^2. \qquad \square$$

Step (2) says that, without a collision, the algorithm sees the same thing under both distributions.

**Lemma 49 ▶ Indistinguishability**

Let $A$, $k$ and $\epsilon$ be as in Lemma 48. For every sequence $t$ of answers (the *transcript* $T = (f(q_1), \ldots, f(q_k))$ of $A$),

$$\mathbb{P}_{\mathcal{D}}(T = t) \geqslant \mathbb{P}_{\mathcal{D}}(T = t \text{ and no collision}) \geqslant (1-\epsilon^2)\,\mathbb{P}_{\mathcal{P}}(T = t).$$

*Proof.* If $t$ has two equal entries then $\mathbb{P}_{\mathcal{P}}(T = t) = 0$ and there is nothing to show. Let $t = (a_1, \ldots, a_k)$ have distinct entries. Under $\mathcal{P}$ every answer is uniform among the unused values, so

$$\mathbb{P}_{\mathcal{P}}(T = t) = \prod_{i=1}^{k} \frac{1}{N - i + 1}.$$

Under $\mathcal{D}$, write $p_i$ for the probability of a collision at step $i$ given $Q_{i-1}$ and the answers $a_1, \ldots, a_{i-1}$; by the proof of Lemma 48, $p_i \leqslant \pi_i := (i-1)/(N - 1 - \binom{i-1}{2})$. If there is no collision at step $i$, the answer is a fresh value, uniform among the $N - i + 1$ unused ones, so it equals $a_i$ with probability $1/(N - i + 1)$. Multiplying over the steps,

$$\begin{aligned}\mathbb{P}_{\mathcal{D}}(T = t \text{ and no collision}) &= \prod_{i=1}^{k}(1 - p_i)\frac{1}{N - i + 1} \geqslant \prod_{i=1}^{k}(1 - \pi_i) \cdot \mathbb{P}_{\mathcal{P}}(T = t) \\ &\geqslant \left(1 - \sum_{i=1}^{k} \pi_i\right) \mathbb{P}_{\mathcal{P}}(T = t),\end{aligned}$$

using $\prod(1 - \pi_i) \geqslant 1 - \sum \pi_i$ for $\pi_i \in [0, 1]$ (Lemma 52 below; $\pi_i \leqslant 2(i-1)/N \leqslant 1$ by the estimate in the proof of Lemma 48). By Lemma 48, $\sum_i \pi_i < \epsilon^2$. □

*Proof of Theorem 46.* Let $A$ be a deterministic algorithm with at most $k \leqslant \sqrt{N}/2$ queries, i.e., $\epsilon = 1/2$ in the lemmas. We may assume that $A$ makes exactly $k$ queries (extra queries whose answers are ignored change nothing), so its transcript $T$ has length $k$. The output of $A$ is a function of $T$. Let $\mathcal{T}$ be the set of transcripts on which $A$ outputs $0^n$ ("$f$ is 1-to-1"), and let $p = \mathbb{P}_{\mathcal{P}}(T \in \mathcal{T})$ be the probability that $A$ is correct on a random 1-to-1 function. Under $\mathcal{D}$, $A$ can only be correct if it does not output $0^n$, so by Lemma 49 summed over all $t \in \mathcal{T}$,

$$\mathbb{P}_{\mathcal{D}}(A \text{ correct}) \leqslant \mathbb{P}_{\mathcal{D}}(T \notin \mathcal{T}) = 1 - \mathbb{P}_{\mathcal{D}}(T \in \mathcal{T}) \leqslant 1 - (1 - \epsilon^2)\,\mathbb{P}_{\mathcal{P}}(T \in \mathcal{T}) = 1 - (1 - \epsilon^2)p.$$

Therefore the success probability of $A$ on an input drawn from $\mu$ is

$$\frac{1}{2}p + \frac{1}{2}\Big(1 - (1 - \epsilon^2)p\Big) = \frac{1}{2} + \frac{\epsilon^2}{2}p \leqslant \frac{1}{2} + \frac{\epsilon^2}{2} = \frac{1}{2} + \frac{1}{8} = \frac{5}{8} < \frac{2}{3}.$$

So every deterministic algorithm with at most $\sqrt{N}/2$ queries errs with probability at least $3/8 > 1/3$ on $\mu$. By Lemma 47 (with $\delta = 1/3$), every randomized algorithm with at most $\sqrt{N}/2$ queries errs with probability greater than $1/3$ on some input, i.e., its success probability on that input is less than $2/3$. Hence an algorithm with success probability at least $2/3$ on every input must make more than $\sqrt{N}/2$ queries on some input. □

In words, as it was put in class: on the first distribution all answers are distinct and the algorithm cannot solve the problem quickly; on the second distribution the answers are also all distinct unless a collision occurs, which is unlikely with few queries, so the algorithm cannot quickly find any difference between the two distributions. The success probability $2/3$ is not important: the same proof with $\epsilon = \sqrt{2\gamma}$ shows that, for $\gamma \leqslant 1/8$, success probability $1/2 + \gamma$ needs more than $\sqrt{2\gamma N}$ queries.

**Corollary 50 ▶ Deterministic Lower Bound**

Every deterministic classical algorithm for Simon's problem makes more than $\sqrt{N}/2$ queries on some input. Hence the deterministic query complexity of Simon's problem is $\Theta\left(\sqrt{N}\right)$.

*Proof.* A deterministic algorithm that is always correct is a randomized algorithm (one that ignores its random bits) with success probability $1 \geqslant 2/3$ on every input, so Theorem 46 applies to it. The matching upper bound is Exercise 15. □

**Corollary 51 ▶ Classical Query Complexity of Simon's Problem**

The classical query complexity of Simon's problem is $\Theta\left(\sqrt{N}\right) = \Theta\left(\sqrt{2^n}\right)$, both for deterministic algorithms (Exercise 15, Corollary 50) and for randomized algorithms with constant success probability (Theorem 45, Theorem 46). Simon's quantum algorithm, with $\mathcal{O}(n)$ queries, is therefore exponentially faster than every classical algorithm.

We used the following elementary inequality.

**Lemma 52**

For $a_1, \ldots, a_m \in [0,1]$, $\prod_{i=1}^{m}(1-a_i) \geqslant 1 - \sum_{i=1}^{m} a_i$.

*Proof.* By induction on $m$; for $m = 1$ it is an equality. If it holds for $m-1$, then, multiplying the induction hypothesis by $1 - a_m \geqslant 0$ and using $a_m \sum_{i<m} a_i \geqslant 0$ in the last step,

$$\prod_{i=1}^{m}(1-a_i) \geqslant \left(1 - \sum_{i=1}^{m-1} a_i\right)(1-a_m) = 1 - \sum_{i=1}^{m-1} a_i - a_m + a_m \sum_{i=1}^{m-1} a_i \geqslant 1 - \sum_{i=1}^{m} a_i. \qquad \square$$

# 43 Simon's Algorithm Revisited

In class the quantum algorithm was gone through again (Figure 18 of the last lecture) with the measurement probabilities written out; the computations are those of Theorem 43, so we only record what is new. Just before the measurement the state is

$$\sum_{y\in\{0,1\}^n} \frac{1}{2^n} \sum_{x\in\{0,1\}^n} (-1)^{x\cdot y} |f(x)\rangle|y\rangle$$

(target register written first), so for a fixed $y$ the probability of measuring $y$ in the control register is the squared norm of the vector that multiplies $|y\rangle$,

$$\mathbb{P}(y) = \left\| \frac{1}{2^n} \sum_{x\in\{0,1\}^n} (-1)^{x\cdot y} |f(x)\rangle \right\|_2^2,$$

because the probability of a measurement outcome is the sum of the squared amplitudes of all basis states $|z\rangle|y\rangle$ with this $y$.

## 43.1 The 1-to-1 Case

In class the computation for a 1-to-1 function ended with a question mark: the "measure" (the probability of $y$) was computed as $\frac{1}{2^{2n}} \cdot 2^n = \frac{1}{2^n}$, and it was asked whether this is right and whether the total is 1.

**Exercise 16.** Suppose that $f$ is 1-to-1. What is the probability of measuring a given string $y$ in Simon's circuit, and do these probabilities add up to 1?

**Solution**. If $f$ is 1-to-1, the strings $f(x)$ for the $2^n$ different $x$ are $2^n$ different basis states, so we may replace $|f(x)\rangle$ by $|x\rangle$ (this only renames the basis states and does not change the norm). The vector multiplying $|y\rangle$ is then $\frac{1}{2^n}\sum_x (-1)^{x\cdot y}|x\rangle$, which has $2^n$ orthonormal components, each of absolute value $1/2^n$. Its squared norm is

$$\mathbb{P}(y) = \sum_{x\in\{0,1\}^n} \left(\frac{1}{2^n}\right)^2 = 2^n \cdot \frac{1}{2^{2n}} = \frac{1}{2^n}.$$

This is the squared norm, not the norm (the norm would be $1/\sqrt{2^n}$); the probability of an outcome is always the squared norm. Every one of the $2^n$ strings $y$ has the same probability $1/2^n$, so the measured $y$ is uniformly distributed over $\{0,1\}^n$, and the total probability is $2^n \cdot 1/2^n = 1$, as it must be. □

## 43.2 The 2-to-1 Case

For $f$ 2-to-1 with period $s \neq 0^n$, collecting the two preimages $x_z$ and $x_z \oplus s$ of every $z$ in the range $R$ of $f$ turns the vector into $\frac{1}{2^n}\sum_{z\in R}(-1)^{x_z\cdot y}(1+(-1)^{s\cdot y})|z\rangle$, whose squared norm is $2^{n-1}\cdot 4/2^{2n} = 1/2^{n-1}$ if $s\cdot y = 0$ and 0 if $s\cdot y = 1$ (Theorem 43, case 1). So we only ever measure strings $y$ with $s\cdot y = 0$, each of the $2^{n-1}$ such strings equally likely, and the total is $2^{n-1}\cdot 1/2^{n-1} = 1$.

## 43.3 Solving for $s$

Running the circuit $n-1$ times gives strings $y_1,\dots,y_{n-1}$, and in the 2-to-1 case the system of linear equations over $\{0,1\}$

$$s\cdot y_1 = 0, \quad s\cdot y_2 = 0, \quad \dots, \quad s\cdot y_{n-1} = 0.$$

We want the $y_i$ to be *linearly independent*: then the $(n-1)\times n$ coefficient matrix has rank $n-1$, and Gaussian elimination gives a one-dimensional solution space $\{0^n, s'\}$ with a unique nonzero solution $s'$. For the nonzero solution $s'$ of the system we test $f(0^n) = f(s')$ with two classical queries: if the test is true, then $f$ is 2-to-1 and $s = s'$ (by the promise, $f(0^n) = f(s')$ with $s' \neq 0^n$ means $s' = 0^n \oplus s$); if it is false, then $f$ is 1-to-1 (in the 2-to-1 case $s$ itself solves the system, so $s' = s$ and the test would be true). This is Algorithm 1 of the last lecture. It remains to show the claim about linear independence that was left open there.

**Exercise 17.** Show that the probability that the $y$'s are linearly independent is at least $1/4$.

In the 2-to-1 case the $y_i$ are independent uniform samples from $V = \{y \mid s\cdot y = 0\}$, which is a subspace of $\{0,1\}^n$ (over the field with two elements) of dimension $n-1$, with $2^{n-1}$ elements. The following theorem answers the exercise, and also covers the 1-to-1 case, where the $y_i$ are uniform samples from the whole space $\{0,1\}^n$ of dimension $n$.

**Theorem 53 ▶ Linear Independence of Random Vectors**

Let $V$ be a subspace of $\{0,1\}^n$ of dimension $d \geqslant 1$, so $|V| = 2^d$, and let $y_1, y_2, \dots$ be independent uniformly random elements of $V$.

1. $y_1,\dots,y_d$ are linearly independent with probability

$$\prod_{i=1}^{d}\left(1-\frac{1}{2^i}\right) = \left(1-\frac{1}{2}\right)\left(1-\frac{1}{2^2}\right)\left(1-\frac{1}{2^3}\right)\cdots\left(1-\frac{1}{2^d}\right) \geqslant \frac{1}{4}.$$

2. $y_1,\dots,y_{d-1}$ are linearly independent with probability $\prod_{i=2}^{d}(1-1/2^i) \geqslant 1/2$.

In particular, in the 2-to-1 case ($V = \{y \mid s\cdot y = 0\}$, $d = n-1$) the strings $y_1,\dots,y_{n-1}$ of Simon's algorithm are linearly independent with probability at least $1/4$, and in the 1-to-1 case ($V = \{0,1\}^n$, $d = n$) with probability at least $1/2$.

*Proof.* Suppose that $y_1,\dots,y_j$ are linearly independent, $0 \leqslant j < d$. Their span consists of the $2^j$ vectors $\sum_{i\leqslant j} c_i y_i$ with $c_i \in \{0,1\}$, all different (a nontrivial combination that gives 0 would be a

linear dependence), and it lies inside $V$ because $V$ is a subspace. The vector $y_{j+1}$ is uniform over the $2^d$ elements of $V$, independent of $y_1, \ldots, y_j$, and $y_1, \ldots, y_{j+1}$ are linearly independent exactly when $y_{j+1}$ is not in this span. So, given that $y_1, \ldots, y_j$ are independent, the probability that $y_1, \ldots, y_{j+1}$ are independent is

$$1 - \frac{2^j}{2^d} = 1 - \frac{1}{2^{d-j}}.$$

(This is the factor "$1 - 2^k/2^n$" written in class, with the $2^n$ replaced by the size $2^d$ of the space the $y$'s live in.) Multiplying these conditional probabilities for $j = 0, 1, \ldots, m-1$, the first $m$ vectors are linearly independent with probability

$$\prod_{j=0}^{m-1} \left(1 - \frac{1}{2^{d-j}}\right) = \prod_{i=d-m+1}^{d} \left(1 - \frac{1}{2^i}\right).$$

For $m = d$ this is $\prod_{i=1}^{d}(1 - 2^{-i})$ and for $m = d - 1$ it is $\prod_{i=2}^{d}(1 - 2^{-i})$.

For the lower bounds we use Lemma 52 and the geometric series $\sum_{i \geqslant 2} 2^{-i} = 1/2$:

$$\prod_{i=2}^{d} \left(1 - \frac{1}{2^i}\right) \geqslant 1 - \sum_{i=2}^{d} \frac{1}{2^i} \geqslant 1 - \frac{1}{2} = \frac{1}{2},$$

which proves the second claim, and then

$$\prod_{i=1}^{d} \left(1 - \frac{1}{2^i}\right) = \frac{1}{2} \prod_{i=2}^{d} \left(1 - \frac{1}{2^i}\right) \geqslant \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4},$$

which proves the first. The claims about Simon's algorithm follow from the measurement distributions computed above: in the 2-to-1 case each $y_i$ is uniform over the $2^{n-1}$ strings with $s \cdot y = 0$, which form a subspace of dimension $d = n - 1$ (the solution set of one nontrivial homogeneous linear equation), and we take $n - 1 = d$ samples; in the 1-to-1 case each $y_i$ is uniform over $\{0,1\}^n$, $d = n$, and we take $n - 1 = d - 1$ samples. □

Note that we need the first factor $1/2$ to be pulled out before using Lemma 52: the bound $1 - \sum_{i \geqslant 1} 2^{-i} = 0$ would be useless. The infinite product $\prod_{i \geqslant 1}(1 - 2^{-i}) = 0.2887\ldots$, so the constant $1/4$ cannot be improved much, and it does not depend on $n$. Together with Theorem 44 this completes the analysis of Simon's algorithm: $n - 1$ runs of the circuit plus two classical queries find $s$ with probability at least $1/4$, a constant number of repetitions makes the success probability any constant we like, and $\mathcal{O}(\log n)$ repetitions make it $1 - 1/n$.

**Lecture Date:** October 6, 2026

**Lecture Topic:** Grover's Search Algorithm, Phase Flip, Diffusion Operator, Rotation Analysis

In this lecture, we study *Grover's search algorithm*, which finds a solution of an unstructured search problem over $N = 2^n$ inputs with $\mathcal{O}\left(\sqrt{N}\right)$ quantum queries, whereas every classical algorithm needs $\Omega(N)$ queries. We first run the algorithm by hand on two qubits, then write it for $n$ qubits using two operators, the *phase flip* $Z_f$ and the *diffusion operator* $D$, and finally analyze it exactly: every Grover iteration is a rotation by a fixed angle $2\theta$ in a two-dimensional plane, which tells us precisely how many iterations to make.

## 44 Unstructured Search

**Problem 6 ▶ Unstructured Search**

Given $f\colon \{0,1\}^n \to \{0,1\}$ as a black box, find an $x$ with $f(x) = 1$.

Think of $f$ as a Boolean formula and of $x$ as an assignment: $f(x) = 1$ means that $x$ satisfies the formula. In *unique satisfiability* we are promised that there is exactly one $x$ with $f(x) = 1$. More generally, let

$$a = |\{x \mid f(x) = 1\}|, \qquad b = |\{x \mid f(x) \neq 1\}| = N - a$$

be the numbers of solutions and of non-solutions, where $N = 2^n$. The search is *unstructured*: we know nothing about $f$ except what the queries tell us, so classically the best we can do is to try inputs one by one. For $a = 1$ every classical algorithm, deterministic or randomized, needs $\Omega(N)$ queries (lectures of September 1 and 3: random sampling needs $N$ expected queries, and $\Omega(N)$ queries are necessary). Grover's algorithm needs only $\mathcal{O}\left(\sqrt{N}\right)$ queries, and more generally $\mathcal{O}\left(\sqrt{N/a}\right)$. (It is known, but not shown in this course, that $\Omega\left(\sqrt{N}\right)$ queries are necessary for every quantum algorithm, so Grover's algorithm is optimal.)

## 45 Grover's Algorithm on Two Qubits

Let $n = 2$, $N = 4$, and suppose that the unique solution is $x^* = 10$. We write a state of the two qubits as its vector of four amplitudes $(a_{00}, a_{01}, a_{10}, a_{11})$. The algorithm has four steps (Figure 20).

1. *Put the register in the uniform superposition with Hadamard gates:*

$$H^{\otimes 2}|00\rangle = \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle) = \left(\tfrac{1}{2}, \tfrac{1}{2}, \tfrac{1}{2}, \tfrac{1}{2}\right).$$

2. *Phase flip.* Recreate $f$ as a quantum circuit, the query $U_f\colon |x\rangle|c\rangle \mapsto |x\rangle|c \oplus f(x)\rangle$. With the target qubit in $\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$ this is the phase flip of the lecture of September 24,

$$U_f\colon |x\rangle \longmapsto (-1)^{f(x)}|x\rangle,$$

which flips the sign of the amplitude of every solution. One query gives

$$\left(\tfrac{1}{2}, \tfrac{1}{2}, -\tfrac{1}{2}, \tfrac{1}{2}\right).$$

3. *Grover diffusion operator.* Let $\mu = \frac{1}{N}\sum_x a_x$ be the mean of the amplitudes. The diffusion operator $D$ reflects every amplitude about the mean (*inversion about the mean*):

$$D\colon a_x \longmapsto 2\mu - a_x \qquad \text{for every } x.$$

Here $\mu = \frac{1}{4}\left(\frac{1}{2} + \frac{1}{2} - \frac{1}{2} + \frac{1}{2}\right) = \frac{1}{4}$, so the three non-solutions become $2 \cdot \frac{1}{4} - \frac{1}{2} = 0$ and the solution becomes $2 \cdot \frac{1}{4} + \frac{1}{2} = 1$:

$$\left(\tfrac{1}{2}, \tfrac{1}{2}, -\tfrac{1}{2}, \tfrac{1}{2}\right) \longmapsto (0, 0, 1, 0) = |10\rangle.$$

Measuring now gives the solution $x^* = 10$ with probability 1, after a single query.

4. In general, *repeat steps 2 and 3* $k$ times, where $k$ is about $\sqrt{N}$ (we compute the exact $k$ in Section 47); for $N = 4$ one iteration is enough.

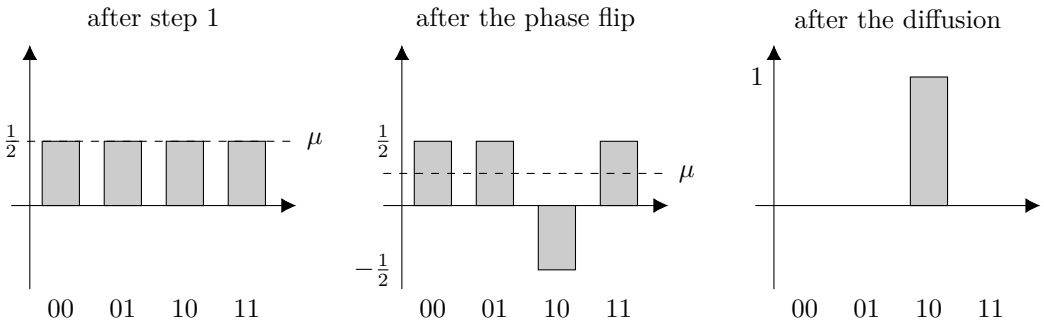



Figure 20: Grover's algorithm for $N = 4$ with the solution $x^* = 10$: the amplitudes after the Hadamard gates, after the phase flip (the mean drops to 1/4), and after the inversion about the mean. One iteration moves all the amplitude onto the solution.

The idea behind steps 2 and 3: the phase flip makes the solution's amplitude negative, which lowers the mean a little; the inversion about the mean then pushes the solution's amplitude up above the mean by the amount it was below, while all the other amplitudes shrink a little. Each iteration transfers amplitude from the non-solutions to the solution.

## 46 Grover's Algorithm on $n$ Qubits

For $n$ qubits and a unique solution $x^*$ the same steps apply to the vector $(a_x)_{x\in\{0,1\}^n}$ of the $N$ amplitudes. After the Hadamard gates every amplitude is $1/\sqrt{N}$, and the phase flip makes the amplitude of $x^*$ equal to $-1/\sqrt{N}$. The mean is then

$$\mu = \frac{1}{N}\left(\frac{N-1}{\sqrt{N}} - \frac{1}{\sqrt{N}}\right) = \frac{N-2}{N\sqrt{N}} \approx \frac{1}{\sqrt{N}},$$

so the diffusion operator maps the amplitude of $x^*$ to $2\mu + 1/\sqrt{N} \approx 3/\sqrt{N}$ and every other amplitude to $2\mu - 1/\sqrt{N} \approx 1/\sqrt{N}$. The next iteration takes $x^*$ to $-3/\sqrt{N}$ and then to about $5/\sqrt{N}$, the one after to about $7/\sqrt{N}$, and so on: as long as the solution's amplitude is small, the mean stays close to $1/\sqrt{N}$ and each iteration adds about $2/\sqrt{N}$ to it, so after about $\sqrt{N}$ iterations the amplitude of $x^*$ is a constant and a measurement finds $x^*$ with constant probability. The exact analysis in Section 47 confirms this and gives the right number of iterations.

### 46.1 The Operators $Z_f$, $Z_0$ and $D$

We now write the two steps of a Grover iteration as unitary operators on $n$ qubits.

**The phase flip $Z_f$.** The phase flip

$$Z_f\colon |x\rangle \longmapsto (-1)^{f(x)}|x\rangle$$

is implemented by the query $U_f$ with an extra qubit in the state $|1\rangle$ passed through a Hadamard gate (Figure 21): the extra qubit is in $H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$ when $U_f$ is applied, so $U_f$ multiplies $|x\rangle$ by $(-1)^{f(x)}$ and leaves the extra qubit unchanged, and the second Hadamard gate returns the extra qubit to $|1\rangle$. One application of $Z_f$ is one query.

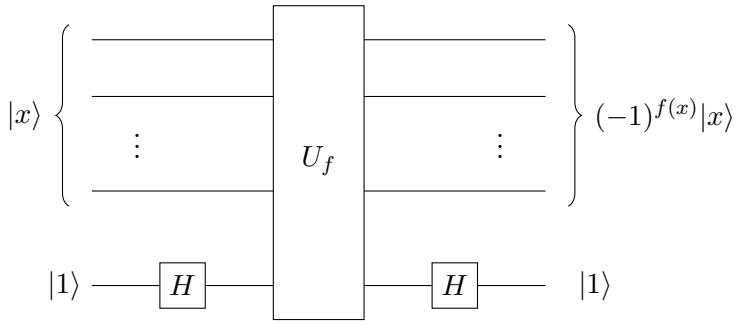



Figure 21: The phase flip $Z_f$: one query $U_f$ with the extra qubit in $H|1\rangle$ flips the sign of every solution $|x\rangle$. The extra qubit comes back to $|1\rangle$ and can be reused.

**The phase flip $Z_0$ about $0^n$.** Let $f_0\colon \{0,1\}^n \to \{0,1\}$ be the function with $f_0(x) = 1$ exactly for $x = 0^n$, i.e., $f_0(x) = \bar{x}_1 \wedge \bar{x}_2 \wedge \cdots \wedge \bar{x}_n$ where $\bar{x}_i = \neg x_i$. Its phase flip is

$$Z_0\colon |x\rangle \longmapsto \begin{cases} -|x\rangle & \text{if } x = 0^n, \\ |x\rangle & \text{otherwise,} \end{cases} \qquad \text{i.e.,} \qquad Z_0 = I - 2|0^n\rangle\langle 0^n|.$$

As a matrix, $|0^n\rangle\langle 0^n|$ is the $N \times N$ matrix with a single 1 in the top left corner, so

$$Z_0 = \begin{pmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & & \\ \vdots & & \ddots & \\ 0 & & & 1 \end{pmatrix} - 2\begin{pmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}\begin{pmatrix} 1 & 0 & \cdots & 0 \end{pmatrix} = \begin{pmatrix} -1 & 0 & \cdots & 0 \\ 0 & 1 & & \\ \vdots & & \ddots & \\ 0 & & & 1 \end{pmatrix}.$$

$Z_0$ is implemented exactly like $Z_f$, with the circuit for $f_0$ in place of the black box: the reversible gate $|x\rangle|c\rangle \mapsto |x\rangle|c \oplus (\bar{x}_1 \wedge \cdots \wedge \bar{x}_n)\rangle$ (a Toffoli gate with $n$ negated controls) and an extra qubit in $H|1\rangle$. It makes no query to $f$.

**The diffusion operator $D$.** Let $|h\rangle = H^{\otimes n}|0^n\rangle = \frac{1}{\sqrt{N}}\sum_x |x\rangle$ be the uniform superposition. The diffusion operator is

$$D = -H^{\otimes n} Z_0 H^{\otimes n}.$$

**Lemma 54 ▶ Inversion About the Mean**

$D = 2|h\rangle\langle h| - I$, and for every state $\sum_x a_x|x\rangle$ with mean amplitude $\mu = \frac{1}{N}\sum_x a_x$,

$$D \sum_{x\in\{0,1\}^n} a_x|x\rangle = \sum_{x\in\{0,1\}^n} (2\mu - a_x)|x\rangle.$$

*Proof.* Since $H^\dagger = H$ and $H^2 = I$, we have $H^{\otimes n}H^{\otimes n} = I$, and $(H^{\otimes n}|0^n\rangle)^\dagger = \langle 0^n|H^{\otimes n}$, i.e., $\langle 0^n|H^{\otimes n} = \langle h|$. Therefore

$$\begin{aligned} D &= -H^{\otimes n}(I - 2|0^n\rangle\langle 0^n|)H^{\otimes n} = -\big(H^{\otimes n}IH^{\otimes n} - 2H^{\otimes n}|0^n\rangle\langle 0^n|H^{\otimes n}\big) \\ &= -(I - 2|h\rangle\langle h|) = 2|h\rangle\langle h| - I. \end{aligned}$$

For the second claim, $\langle h|\psi\rangle = \frac{1}{\sqrt{N}}\sum_x a_x = \sqrt{N}\mu$ for $|\psi\rangle = \sum_x a_x|x\rangle$, so

$$2|h\rangle\langle h|\psi\rangle = 2\sqrt{N}\mu|h\rangle = 2\mu\sum_x|x\rangle, \qquad \text{and} \qquad D|\psi\rangle = 2\mu\sum_x|x\rangle - \sum_x a_x|x\rangle = \sum_x(2\mu - a_x)|x\rangle.$$

□

So $D$ is exactly the inversion about the mean of step 3, and it costs no query. (The notes write $D = I - 2|h\rangle\langle h|$, which is $-D$; the overall sign of $D$ only multiplies the state after $k$ iterations by $(-1)^k$, a global phase that changes no measurement probability. We keep the sign that makes $D$ the inversion about the mean.)

**The Grover iteration.** One iteration of steps 2 and 3 is the operator

$$G = DZ_f = -H^{\otimes n}Z_0H^{\otimes n}Z_f,$$

and Grover's algorithm is: prepare $|h\rangle = H^{\otimes n}|0^n\rangle$, apply $G$ $k$ times, measure (Figure 22). It makes $k$ queries, one per iteration; the Hadamard gates and $Z_0$ make no query.

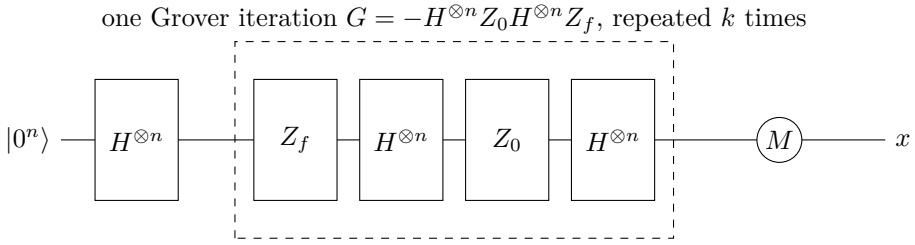



Figure 22: Grover's algorithm on $n$ qubits (the extra qubit used by $Z_f$ and $Z_0$ is not drawn). Each iteration makes one query.

## 47 Analysis: A Rotation in Two Dimensions

The exact analysis works for any number $a$ of solutions, $1 \leqslant a \leqslant N-1$; let $b = N - a$. The key observation is that the whole computation takes place in a two-dimensional plane: one dimension for the solutions and one for the non-solutions. Let

$$|A\rangle = \frac{1}{\sqrt{a}}\sum_{\substack{x\in\{0,1\}^n \\ f(x)=1}}|x\rangle, \qquad |B\rangle = \frac{1}{\sqrt{b}}\sum_{\substack{x\in\{0,1\}^n \\ f(x)=0}}|x\rangle$$

be the uniform superpositions of the solutions and of the non-solutions. They are unit vectors (each has $a$, resp. $b$, amplitudes equal to $1/\sqrt{a}$, resp. $1/\sqrt{b}$) and they are orthogonal (no basis state appears in both). The initial state is

$$|h\rangle = \frac{1}{\sqrt{N}}\sum_{x\in\{0,1\}^n}|x\rangle = \frac{1}{\sqrt{N}}\Big(\sqrt{a}|A\rangle + \sqrt{b}|B\rangle\Big) = \sqrt{\frac{a}{N}}|A\rangle + \sqrt{\frac{b}{N}}|B\rangle. \tag{10}$$

Since $\frac{a}{N} + \frac{b}{N} = 1$, there is a unique angle $\theta \in (0, \pi/2)$ with

$$\sin\theta = \sqrt{\frac{a}{N}}, \qquad \cos\theta = \sqrt{\frac{b}{N}}, \qquad \text{so that} \qquad |h\rangle = \sin\theta|A\rangle + \cos\theta|B\rangle.$$

For a unique solution, $\sin\theta = 1/\sqrt{N}$, so $\theta \approx 1/\sqrt{N}$ is a small angle: $|h\rangle$ is almost $|B\rangle$ (Figure 23).

**Lemma 55 ▶ One Grover Iteration**

$$G|A\rangle = \left(1 - \frac{2a}{N}\right)|A\rangle - \frac{2\sqrt{ab}}{N}|B\rangle = \cos 2\theta|A\rangle - \sin 2\theta|B\rangle,$$
$$G|B\rangle = \frac{2\sqrt{ab}}{N}|A\rangle - \left(1 - \frac{2b}{N}\right)|B\rangle = \sin 2\theta|A\rangle + \cos 2\theta|B\rangle.$$

*Proof.* The phase flip $Z_f$ negates every solution and fixes every non-solution, so $Z_f|A\rangle = -|A\rangle$ and $Z_f|B\rangle = |B\rangle$. By Equation (10) and orthonormality, $\langle h|A\rangle = \sqrt{a/N}$ and $\langle h|B\rangle = \sqrt{b/N}$. With $D = 2|h\rangle\langle h| - I$ (Lemma 54),

$$\begin{aligned} G|A\rangle = DZ_f|A\rangle = -D|A\rangle = -(2|h\rangle\langle h|A\rangle - |A\rangle) &= |A\rangle - 2\sqrt{\frac{a}{N}}|h\rangle \\ &= |A\rangle - 2\sqrt{\frac{a}{N}}\left(\sqrt{\frac{a}{N}}|A\rangle + \sqrt{\frac{b}{N}}|B\rangle\right) \\ &= |A\rangle - 2\left(\frac{a}{N}|A\rangle + \frac{\sqrt{ab}}{N}|B\rangle\right) = \left(1 - \frac{2a}{N}\right)|A\rangle - \frac{2\sqrt{ab}}{N}|B\rangle, \end{aligned}$$

and, by the same computation except that $Z_f$ leaves $|B\rangle$ unchanged, so that the leading minus sign of $-D$ is not there,

$$\begin{aligned} G|B\rangle = DZ_f|B\rangle = D|B\rangle = 2|h\rangle\langle h|B\rangle - |B\rangle &= 2\sqrt{\frac{b}{N}}\left(\sqrt{\frac{a}{N}}|A\rangle + \sqrt{\frac{b}{N}}|B\rangle\right) - |B\rangle \\ &= \frac{2\sqrt{ab}}{N}|A\rangle + \frac{2b}{N}|B\rangle - |B\rangle = \frac{2\sqrt{ab}}{N}|A\rangle - \left(1 - \frac{2b}{N}\right)|B\rangle. \end{aligned}$$

For the trigonometric form use $\sin^2\theta = a/N$, $\cos^2\theta = b/N$ and the double angle formulas:

$$\cos 2\theta = 1 - 2\sin^2\theta = 1 - \frac{2a}{N}, \qquad \cos 2\theta = 2\cos^2\theta - 1 = \frac{2b}{N} - 1 = -\left(1 - \frac{2b}{N}\right),$$
$$\sin 2\theta = 2\sin\theta\cos\theta = \frac{2\sqrt{ab}}{N}.$$

□

(The $|B\rangle$ term of $G|A\rangle$ is negative; on the board it was written with a + sign in a few places, but the line before it, $|A\rangle - 2(\dots)$, and the matrix written in class both have the minus sign.) Since $a + b = N$, both $1 - 2a/N$ and $2b/N - 1$ equal $(b - a)/N$. So, writing a state $\beta|B\rangle + \alpha|A\rangle$ of the plane as the column vector $(\beta, \alpha)^T$ in the basis $(|B\rangle, |A\rangle)$, the lemma says that $G$ acts as the $2 \times 2$ matrix

$$G = \begin{pmatrix} \frac{b-a}{N} & -\frac{2\sqrt{ab}}{N} \\ \frac{2\sqrt{ab}}{N} & \frac{b-a}{N} \end{pmatrix} = \begin{pmatrix} \cos 2\theta & -\sin 2\theta \\ \sin 2\theta & \cos 2\theta \end{pmatrix}, \tag{11}$$

whose columns are the coordinates of $G|B\rangle$ and $G|A\rangle$. The matrix on the right is the rotation matrix $R_\phi = \begin{pmatrix} \cos\phi & -\sin\phi \\ \sin\phi & \cos\phi \end{pmatrix}$ with $\phi = 2\theta$: one Grover iteration *rotates* the plane by the angle $2\theta$, from $|B\rangle$ towards $|A\rangle$. (Geometrically, $Z_f$ is the reflection about the line of $|B\rangle$ and $D$ is the reflection about the line of $|h\rangle$; two reflections about lines at angle $\theta$ compose to a rotation by $2\theta$.) In class the computation of $G|h\rangle$ was started and the rotation matrix was written down, but the computation was not finished; we complete it here.

**Exercise 18.** Compute $G|h\rangle = \sqrt{a/N}\,G|A\rangle + \sqrt{b/N}\,G|B\rangle$ and, more generally, $G^k|h\rangle$, and verify that the matrix of $G$ in Equation (11) is the rotation $R_{2\theta}$.

**Solution**. *The matrix is a rotation.* By Lemma 55 and $(b-a)/N = 1-2a/N = 2b/N-1$, the two columns of the matrix in Equation (11) are the coordinates of $G|B\rangle = \cos 2\theta|B\rangle + \sin 2\theta|A\rangle$ and $G|A\rangle = -\sin 2\theta|B\rangle + \cos 2\theta|A\rangle$, which are exactly the columns of $R_{2\theta}$. (It is $R_{2\theta}$, not $R_\theta$: the entries are $\cos 2\theta = 1-2\sin^2\theta$ and $\sin 2\theta = 2\sin\theta\cos\theta$, not $\cos\theta$ and $\sin\theta$.) As a check, the columns are unit vectors and orthogonal: $((b-a)/N)^2 + \left(2\sqrt{ab}/N\right)^2 = (b^2-2ab+a^2+4ab)/N^2 = (a+b)^2/N^2 = 1$.

*One iteration.* By the trigonometric form of Lemma 55 and the addition formulas,

$$G|h\rangle = \sin\theta\,(\cos 2\theta|A\rangle - \sin 2\theta|B\rangle) + \cos\theta\,(\sin 2\theta|A\rangle + \cos 2\theta|B\rangle) = \sin 3\theta|A\rangle + \cos 3\theta|B\rangle.$$

(In the $a, b$ form started in class, the coefficient of $|A\rangle$ is $\sqrt{a/N}\,(1-2a/N) + \sqrt{b/N}\cdot 2\sqrt{ab}/N = \sqrt{a/N}\cdot(3b-a)/N = \sin\theta(3\cos^2\theta - \sin^2\theta) = \sin 3\theta$ by the triple angle formula $\sin 3\theta = 3\sin\theta - 4\sin^3\theta$.)

*$k$ iterations.* We claim that for every $k \geqslant 0$,

$$G^k|h\rangle = \sin((2k+1)\theta)|A\rangle + \cos((2k+1)\theta)|B\rangle.$$

This holds for $k=0$ by Equation (10). If it holds for $k$, then by Lemma 55 and the addition formulas, with $\phi = (2k+1)\theta$,

$$\begin{aligned} G^{k+1}|h\rangle &= \sin\phi(\cos 2\theta|A\rangle - \sin 2\theta|B\rangle) + \cos\phi(\sin 2\theta|A\rangle + \cos 2\theta|B\rangle) \\ &= (\sin\phi\cos 2\theta + \cos\phi\sin 2\theta)|A\rangle + (\cos\phi\cos 2\theta - \sin\phi\sin 2\theta)|B\rangle \\ &= \sin(\phi+2\theta)|A\rangle + \cos(\phi+2\theta)|B\rangle = \sin((2k+3)\theta)|A\rangle + \cos((2k+3)\theta)|B\rangle, \end{aligned}$$

which is the claim for $k+1$. So after $k$ iterations the state makes the angle $(2k+1)\theta$ with $|B\rangle$: it starts at $\theta$ and every iteration adds $2\theta$ (Figure 23). □

## 47.1 How Many Iterations

Measuring the state $G^k|h\rangle = \sin((2k+1)\theta)|A\rangle + \cos((2k+1)\theta)|B\rangle$ in the computational basis gives a solution with probability $\sin^2((2k+1)\theta)$: the amplitude of each of the $a$ solutions is $\sin((2k+1)\theta)/\sqrt{a}$, and these $a$ squared amplitudes add up to $\sin^2((2k+1)\theta)$. We want $(2k+1)\theta$ to be as close to $\pi/2$ as possible.

**Theorem 56 ▶ Grover's Algorithm**

Let $f$ have $a$ solutions, $1 \leqslant a \leqslant N/2$, let $\theta \in (0, \pi/2)$ with $\sin\theta = \sqrt{a/N}$, and let $k = \lfloor \pi/4\theta \rfloor$. Then Grover's algorithm with $k$ iterations makes $k \leqslant \frac{\pi}{4}\sqrt{N/a}$ queries and outputs a solution with probability at least $1 - a/N \geqslant 1/2$. For a unique solution ($a=1$) it makes about $\frac{\pi}{4}\sqrt{N}$ queries and succeeds with probability at least $1 - 1/N$.

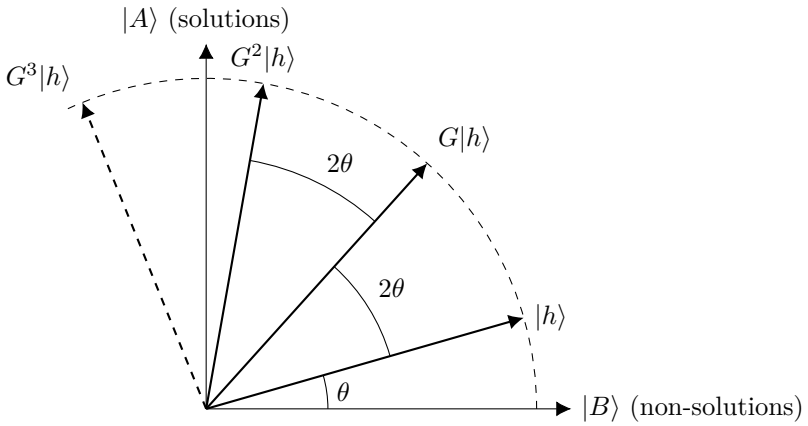



Figure 23: Grover's algorithm in the plane spanned by $|A\rangle$ and $|B\rangle$. The initial state $|h\rangle$ makes the small angle $\theta$ with $|B\rangle$, and every iteration rotates the state by $2\theta$ towards $|A\rangle$: after $k$ iterations the angle is $(2k+1)\theta$. Here $\theta = 16°$, so $k = \lfloor \pi/4\theta \rfloor = 2$ iterations are best (angle $80°$, success probability $\sin^2 80° \approx 0.97$), and a third one (dashed, angle $112°$) overshoots past $|A\rangle$ and lowers the probability to $\sin^2 112° \approx 0.86$.

*Proof.* By the definition of $k$, $\pi/4\theta - 1 < k \leqslant \pi/4\theta$, so multiplying by $2\theta$ and adding $\theta$,

$$\frac{\pi}{2} - \theta < (2k+1)\theta \leqslant \frac{\pi}{2} + \theta.$$

The sine is increasing on $[0, \pi/2]$ and decreasing on $[\pi/2, \pi]$, so on the interval $[\pi/2-\theta, \pi/2+\theta] \subseteq [0, \pi]$ it is at least its value at the endpoints, $\sin(\pi/2 \pm \theta) = \cos\theta$. Hence the success probability is

$$\sin^2((2k+1)\theta) \geqslant \cos^2\theta = 1 - \sin^2\theta = 1 - \frac{a}{N} \geqslant \frac{1}{2}.$$

For the number of queries, $\theta \geqslant \sin\theta = \sqrt{a/N}$ (as $\sin\theta \leqslant \theta$ for $\theta \geqslant 0$), so $k \leqslant \pi/4\theta \leqslant \frac{\pi}{4}\sqrt{N/a}$. For $a = 1$, $\sin\theta = 1/\sqrt{N}$, so $\theta \approx 1/\sqrt{N}$ and $k \approx \frac{\pi}{4}\sqrt{N}$. □

This is the precise form of "repeat $\sqrt{N}$ times": the number of iterations is $\Theta\left(\sqrt{N}\right)$ for a unique solution and $\Theta\left(\sqrt{N/a}\right)$ for $a$ solutions, and the success probability is a constant close to 1. The analysis also explains the two-qubit example: for $N = 4$ and $a = 1$, $\sin\theta = 1/2$, so $\theta = \pi/6$ and $k = \lfloor 3/2 \rfloor = 1$ iteration gives the angle $3\theta = \pi/2$, i.e., the state $|A\rangle = |x^*\rangle$ exactly and success probability $\sin^2(\pi/2) = 1$, as computed in Figure 20.

**Do not overshoot.** Unlike a classical search, more iterations do not help: the state keeps rotating, and after $k$ iterations with $(2k+1)\theta > \pi/2$ the success probability $\sin^2((2k+1)\theta)$ decreases again (the dashed vector in Figure 23). After about $\pi/2\theta$ iterations the state is back near $|B\rangle$. So the number of iterations has to be chosen from $a$ and $N$; this is why the algorithm needs to know (or to estimate) the number of solutions.